

ТЕХНОЛОГИЧНО УЧИЛИЩЕ ЕЛЕКТРОННИ СИСТЕМИ
към ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - СОФИЯ

ДИПЛОМНА РАБОТА

Тема: Система за управление на клиенти

Дипломант:

Емил Дойчинов

Научен ръководител

Станимир Чакалов

СОФИЯ

2024

Увод

Няма човек, който в днешно време да не използва интернет - това е станало част от нашия всекидневен живот. Естествено тази огромна платформа се използва за всевъзможни дейности - за търсене на информация по конкретни теми, за а игри, основно и за пазаруване. Онлайн пазаруването набира рязка популярност в последните години. Според статистики eCommerce продажбите до 2023 година са се увеличили с почти 8 пъти спрямо 2010 година, а за 2023 е известно, че има около 2.63 милиарда пазаруващи онлайн. Всеки бизнес се опитва да прокара в тази среда, за която е прогнозирано само разрастване в близките няколко години.

Според най-успешните бизнесмени клиентът винаги има право. Всички компании, независимо големи или малки, се стремят да подобрят максимално изживяването му във всяка една стъпка от използването на техния продукт. Причината за това е, че колкото по-добро е потребителското изживяване, то толкова по-вероятно е клиентът да остане доволен, което на свой ред довежда до по-голяма вероятност продуктът да бъде препоръчан на други хора. Така чрез осигуряване на качеството на приложението за клиента един бизнес може да се развива успешно. Тук на свой ред идват и системите за управление на клиенти.

Customer relationship management системите или CRM системите на кратко, са средства, които открай време се използват от малки бизнеси до големи корпоративни среди като основни платформи за управление на взаимодействието с клиентите, помагачи за администриране на потребителите, за наблюдения над транзакции и продажби, за изготвяне на статистики за активността в приложението, за вземане на важни решения в бизнеса, които да допринесат за неговия успех, за развитие и цялостно

наблюдение над потребителското изживяване и създаване на нови начини за подобряването му.

Тази дипломна работа има за цел да даде решение за CRM система, чрез имплементация на приложно-програмен интерфейс, която може да бъде приложена както в малки среди, така и в корпоративни. Целта на системата е да покрива основните нужди на един разрастващ се бизнес и да може да бъде разширена в бъдеще, така че да бъде способна да асистира потребителите на нейните клиенти във всяка една част от използването на техните уеб приложения. Всеки клиент на системата ще бъде снабден със своя изолирана инстанция, в която нейните потребители, наречени оператори, ще могат според зададените от него права да управляват клиентите му и техните данни, както и да проследяват на дълбоко ниво взаимоотношенията с тях. Системата ще предоставя възможността за лесна интеграция в уеб апликации с цел улесняването на тяхната разработка и лесно изготвяне на статистики за клиентите, като например за тяхната активност, техните най-често посещавани страници, техните поръчки и други.

Глава I.

Подходи за създаване на системата и проучване на подобни решения

1.1 Методи за разработка на уеб приложения

Съществуват различни методи за разработка на уеб приложения. Тези методи включват: server-side, client-side и full-stack разработка.

Server-side[1] разработката представлява изграждане на уеб приложения с помощта на езици за програмиране като Node.js, PHP, Java, Python или Ruby, заедно с рамки като NestJS, Laravel, Spring Boot, Django или Ruby on Rails. При този подход сървърът обработва по-голямата част от логиката на приложението и обработката на данни, като генерира динамично съдържание, което след това се изпраща към уеб браузъра на клиента. Този метод предлага мащабируемост и сигурност, което го прави предпочитан избор при разработката на приложения от корпоративен клас.

Client-side[1] разработката се фокусира върху създаването на динамични и интерактивни потребителски интерфейси с помощта на front-end технологии като HTML, CSS и JavaScript. Рамки и библиотеки като Svelte, Angular, React, Vue.js и SolidJS са широко използвани за изграждане на single page приложения (SPA), които се изпълняват изцяло в уеб браузъра на клиента. Client-side разработката набляга на гъвкавостта, производителността и гладкото потребителско изживяване, като използва възможностите на съвременните уеб браузъри, за да предоставя богати и интересни уеб приложения.

Full-stack разработката съчетава както server-side, така и client-side разработката, за да се създаде цялостно уеб приложение. Рамки като Node.js дават възможност на разработчиците да използват JavaScript или TypeScript както за разработване на сървъра, така и на клиента, като по този начин бива постигната съгласуваност на целия стек от използвани технологии. Full-stack разработката предлага гъвкавост, ефективност и възможност за изграждане на мащабируеми и богати на функционалности уеб приложения.

1.2 Технологии и развойни средства за разработка на уеб приложения

1.2.1 Rust

Макар и да не е толкова разпространен спрямо езици като JavaScript или Python в контекста на уеб разработване, Rust [2] предлага уникални предимства за изграждане на сигурни уеб приложения. Една от ключовите характеристики на Rust е безопасността на паметта и на нишките, без това да е за производителността на приложението. Строгите проверки на компилатора осигуряват безопасност на паметта, като предотвратяват често срещани капани като препращане към нулев указател и загуба данни, което го прави добър за изграждане на стабилни и надеждни уеб приложения. Характеристиките на производителността на Rust са на ниво с low-level езици като C и C++. Това прави Rust подходящ за извършване на интензивни изчислителни операции, които често се срещат в съвременните уеб приложения. За разработката на уеб приложения Rust предлага разнообразни рамки и библиотеки. Рамки като Rocket и Actix дават възможност за разработка на високопроизводителни и лесни за поддръжка приложно-програмни интерфейси.

1.2.2 Python

Python[3] е един от най-популярните езици за програмиране, известен със своята простота, разбираемост и гъвкавост. За разработване на уеб приложения Python предлага богата екосистема от рамки и библиотеки като например `django` и `flask`.

Django предоставя солиден набор от инструменти за бързо изграждане на уеб приложения. Вграденият Object-relational mapper опростява взаимодействието с бази данни, а вградените функции като удостоверяване и маршрутизиране на URL адреси улесняват процеса на разработка.

Flask е лека и гъвкава микрорамка.. Позволява по-бързо и опростено изграждане на уеб приложения, което го прави добър избор за малки проекти или за създаване на прототипи.

1.2.3 JavaScript

JavaScript[4] е универсален инструмент за изграждане на мащабируеми и ефективни full-stack уеб приложения. Node.js - среда за изпълнение на JavaScript, предоставя стабилен набор от функционалности и модули за server-side разработка. Богатата екосистема на JavaScript от модули , Node Package Manager, допринася към уеб разработката. Популярни модули са TypeORM и Prisma, използвани за object-relational mapping, както и PassportJS - мидълуер, използван за автентикация. Съвместимостта на JavaScript с JSON (JavaScript Object Notation) опростява обмена на данни между клиента и сървъра, позволявайки безпроблемна комуникация през RESTful приложно-програмни интерфейси и WebSocket връзки.

1.2.4 TypeScript

TypeScript[5] надгражда JavaScript, като добавя статична типизация. Езикът може да бъде изпълнен на всяка една платформа, която поддържа JavaScript и всички JavaScript модули могат да бъдат използвани и тук. В сферата на server-side разработката, TypeScript предлага няколко предимства пред JavaScript, като интерфейси, generics, декоратори, както и по-добро type safety. С модули и рамки като NestJS, Express.js, Koa и Fastify, ефективно могат да се създават RESTful приложно-програмни интерфейси както на JavaScript, така и на TypeScript.

1.2.5 Java

Java[6] се използва в разработката на уеб приложения от десетилетия. Благодарение на голямата си екосистема и развитите рамки, този език все още е добър избор за изграждане на надеждни уеб приложения. Обширната екосистема от рамки на Java, включително Spring Boot, Jakarta EE и Play Framework, предоставя инструменти за изграждане на уеб приложения с различна сложност. Тези рамки предлагат функционалности като аспектно-ориентирано програмиране и ORM. Възможността за многопоточност и паралелност позволява създаването високопроизводителни уеб приложения, способни да обработват голям трафик и едновременни заявки. Вградената поддръжка на Java за сървлети, JSP (JavaServer Pages) и уеб контейнери като Apache Tomcat и Jetty осигурява солидна основа за изграждане на мащабируеми и ефективни уеб сървъри.

1.2.6 C#

C#[7] е широко разпространен език за разработка на уеб приложения, особено в рамките на .NET екосистемата. Именно интеграцията с рамката .NET е едно от ключовите предимства на C#, тъй като предоставя богат набор от библиотеки, приложно-програмни интерфейси и услуги. ASP.NET Core, междуплатформена рамка, изградена върху .NET Core, позволява изграждането на високопроизводителни уеб приложения, които могат да работят в Windows, Linux и macOS среди. C# лесно може да се интегрира с Microsoft Azure, което позволява лесен deployment и мащабиране. Благодарение на Azure App Service, Azure Functions и Azure SQL Database, уеб приложенията лесно могат да бъдат управлявани в облака.

1.2.7 Go

Go[8] е статично типизиран, компилируем език за програмиране, създаден от Google. В последните години, благодарение на производителността си набира популярност за изграждане на уеб приложения, приложно-програмни интерфейси и микроуслуги. Едно от предимствата на Go за уеб разработка е изградената поддръжка за паралелност чрез goroutines и канали. Goroutine-ите позволяват едновременното изпълняване на няколко процеса, докато каналите улесняват комуникацията и синхронизацията между тях. Стандартната библиотека на Go предоставя набор от пакети за изграждане на уеб сървъри, като например net/http, обработка на HTTP заявки и работа с JSON. Рамки и библиотеки като Gin, Echo и Fiber предоставят допълнителни функции и помощни програми за изграждане на уеб приложения и приложно-програмни интерфейси. Съвместимостта на Go с

Docker и Kubernetes го прави подходящ за изграждане на контейнеризирани приложения.

1.2.8 Ruby

Ruby[9] е обектно-ориентиран динамичен език за програмиране, привлекателен с рамката на Ruby Rails. Rails предоставя набор от конвенции и най-добри практики за ефективно изграждане на уеб приложения. Предлагайки функционалности като MVC или Model-View-Controller архитектура и ActiveRecord ORM, рамката добре абстрахира някои от сложностите на уеб разработката. Динамичната природа на езика позволява бързото създаване на прототипи и итеративно разработка на приложението. Ruby разполага с богата екосистема от gems, тоест библиотеки, предоставящи функционалности от удостоверяване и оторизация до тестване и deployment, които допълнително подпомагат разработката на уеб приложения чрез Rails.

1.2.9 PHP

PHP[10] (Hypertext Preprocessor) е широко използван скриптов език с отворен код, създаден специално за уеб разработка. Със своята простота, гъвкавост и обширна екосистема от рамки и библиотеки, PHP е един от най-популярните избори за изграждане на динамични и интерактивни уеб приложения. Обширната стандартна библиотека на PHP предоставя широк набор от функционалности и модули за уеб разработка, включително обработка на файлове, достъп до база данни и обработка на формуляри. PHP разполага с множество рамки и Content Management System (CMS, система за управление на съдържанието) платформи. Рамки като Laravel, Symfony и CodeIgniter предлагат функционалности като маршрутизиране, шаблониране, удостоверяване и ORM, ускорявайки процеса на разработка.

1.2.10 PostgreSQL

PostgreSQL[11] е RDBMS (Relational Database Management System, система за управление на релационни бази данни) с отворен код, известна със своите стабилни функционалности, надеждност и разширяемост. Поддържа широк набор от типове данни, включително JSON, XML и масиви. PostgreSQL предлага разширени функционалности като съответствие с ACID (atomicity, consistency, isolation, and durability), транзакции и поддръжка за сложни заявки и join-ове. Разширяемостта позволява на създаването на персонализирани функции и типове данни. PostgreSQL използва MVCC (Version Concurrency Control, за да гарантира, че много потребители могат да имат достъп до едни и същи данни едновременно без конфликти).

1.2.11 MSSQL

MSSQL[12] е собствена RDBMS, разработена от Microsoft, широко използвана в уеб приложения и организации на корпоративно ниво, използващи технологичния стек на Microsoft. Предлага цялостни функционалности за управление на данни, сигурност и мащабируемост, което прави тази база данни подходяща за mission-critical приложения. MSSQL осигурява безпроблемна интеграция с други продукти и услуги на Microsoft, включително Windows Server, Active Directory и Azure. Функционалностите за сигурност включват:

- криптиране,
- row-level security
- одит

MSSQL предлага решения за high availability и възстановяване след бедствие, като Always On Availability Groups и дублиране на бази данни.

1.2.12 Redis

Redis[13] е популярно хранилище за данни в паметта с отворен код, известно със своята скорост, гъвкавост и производителност. В уеб разработката Redis служи като мощен инструмент за :

- Кеширане: Redis кешира на често достъпвани данни в паметта, като намалява натоварването на базата данни и подобрява производителността на приложението. Съхранявайки key-value двойки в паметта, Redis позволява бързо извличане на кеширани данни.
- Управление на сесии: Redis предоставя ефективни възможности за управление на сесии. Могат да бъдат имплементирани мащабируеми сесийни хранилища, които поддържат милиони потребители и осигуряват безпроблемна поддръжка на изтичането, както и постоянството на сесиите.
- Анализ в реално време: Структурите на данни на Redis, като сортирани набори и хиперлогове, позволяват анализи в реално време и обработка на данни.
- Pub/Sub съобщения: Redis поддържа pub/sub модели, което позволява комуникация между различни компоненти на уеб приложение или между множество приложения. С Redis Pub/Sub разработчиците могат да имплементират event-based архитектури, известия в реално време и опашки за съобщения, подобрявайки скалируемостта и скоростта на приложенията.
- Атомарни операции и транзакции: Redis предлага атомарни операции и поддръжка на транзакции, като гарантира целостта на данните и последователност при многостепни операции.

- Персистентност: Redis предоставя различни опции за персистентност, включително моментна снимка и append-only file (AOF), за да гарантира дълготрайност и възможност за възстановяване на данните..

1.2.13 Docker

Docker[14] е популярна платформа, която позволява създаване, доставяне и изпълнение на приложения в леки, преносими контейнери. Със своята технология за контейнеризация Docker опростява процеса на deployment и управление на уеб приложения в различни среди, от такива за разработка до производствени. Docker е привлекателен за уеб разработка, поради способността за енкапсулиране на приложенията и техните зависимости в самостоятелни контейнери. Контейнерите “пакетират” кода на приложението, библиотеките, runtime-а и зависимостите, осигурявайки консистентност и възпроизводимост в различни среди. Docker предоставя декларативен синтаксис, известен като Dockerfile, който се използва за дефиниране на конфигурацията и зависимостите на конкретното приложение. Dockerfile-овете се използват за създаване на персонализирани изображения на контейнери, съобразени с техните специфични изисквания, подпомагайки процеса на deployment и намалявайки проблемите със съвместимостта. Ефективният модел за контейнеризация предоставя възможност за мащабируемост на уеб приложенията. Множество контейнери могат да вървят едновременно, всеки от които изпълнява определен компонент или услуга на приложението, и да бъдат организирани, чрез оркестратори като Kubernetes. Docker екосистемата разполага с Docker Compose - инструмент за дефиниране и управление на многоконтейнерни приложения. Docker се интегрира безпроблемно с pipeline-ове за Continuous Integration и

Continuous Deployment (CI/CD, непрекъсната интеграция и непрекъснато внедряване).

1.2.14 Kubernetes

Kubernetes[15], още познат като K8s, е платформа за оркестриране на контейнери с отворен код, предназначена да автоматизира deployment-a, мащабирането и управлението на контейнерни приложения. Тъй като уеб приложенията стават все по-сложни и разпределени, Kubernetes предоставя стабилна рамка за оркестриране и управление на натоварвания в мащаб. В основата на Kubernetes е неговата декларативна API-ориентирана архитектура, която позволява дефиницията на желаното състояние на приложението, използвайки YAML файлове. Контролерите на Kubernetes непрекъснато наблюдават състоянието на клъстера и гарантират, че действителното състояние съответства на желаното състояние, като автоматично коригира ресурсите и конфигурациите, ако е необходимо. Една от ключовите характеристики на Kubernetes е способността му да управлява контейнеризирани приложения в клъстери от виртуални или физически машини. Kubernetes предоставя вградени функции за service discovery, load balancing и автоматично мащабиране, което го прави много подходящ за изграждане на устойчиви уеб приложения. Абстракцията на услугите на Kubernetes позволява безпроблемна комуникация между микроуслугите в рамките на клъстера, докато неговите възможности за хоризонтално и вертикално автоматично мащабиране гарантират, че приложенията могат да се справят с натоварения трафик. Kubernetes се интегрира безпроблемно с други облачни технологии, като Prometheus за наблюдение (monitoring), Grafana за визуализация и Istio за мрежова функционалност на услугата.

1.2.15 Postman

Postman[16] е основен инструмент в уеб разработката, предлагащ удобен за потребителя интерфейс за тестване и отстраняване на грешки в приложно-програмния интерфейс на уеб приложението или по-точно в сървърната част. Основните му характеристики включват възможността за изпращане на HTTP заявки, преглед на отговорите и анализ на данни в различни формати като JSON и XML. Postman предлага възможности за автоматизирано тестване, което позволява създаването и изпълняването на тестови пакети, за да бъде гарантирано, че крайните точки на приложно-програмния интерфейс функционират според очакванията.

1.3 Оглед на съществуващи решения

1.3.1 Salesforce

Salesforce е система за управление на клиенти (CRM), базирана на облак, която дава възможност на бизнесите да изгражда добри взаимоотношения с клиентите си и съответно да стимулира продажбите. Със своя изчерпателен набор от функции и интуитивен потребителски интерфейс. Salesforce разполага с набор от функционалности като :

- Управление на потребителите: Salesforce управление на потребителите, което позволява на администраторите да създават и управляват потребителски акаунти, роли и права. Чрез централизирано табло за управление администраторите могат да дефинират нива на потребителски достъп, да присвояват роли и да контролират видимостта на данните.

- Управление на потребителските права: Администраторите имат контрол върху потребителските права, което им позволява да дефинират персонализирани контроли за достъп и разрешения, съобразени с конкретни потребителски роли.
- Управление на клиентски данни: Salesforce предоставя мощни инструменти за управление на клиентски данни, включително изчерпателни клиентски профили, информация за контакт и хронология на взаимоотношенията.
- Проследяване на взаимоотношения с клиенти: Salesforce предлага разширени функционалности за проследяване на взаимоотношения с клиенти, включително проследяване на активността и точкуване на възможни клиенти. С табла за управление и отчети, които могат да се персонализират, екипите по продажбите могат да получат ценна информация за взаимодействията с клиентите, да идентифицират възможностите за продажби и да приоритизират ефективно потенциалните клиенти.
- API интеграция: Salesforce поддържа безпроблемна API интеграция с широк набор от приложения и услуги, което позволява на организациите да разширят функционалността на своята CRM платформа и да се интегрира в third-party системи. Чрез стабилните си REST и SOAP API, Salesforce улеснява обмена на данни, автоматизацията и синхронизирането между различни системи и платформи.
- Einstein AI:
 - Оценяване на потенциални клиенти: Einstein Lead Scoring анализира исторически данни и взаимодействия с клиенти, за да предвиди кои потенциални клиенти да бъдат приоритизирани.

- Opportunity Insights: Einstein Opportunity Insights предоставя прогнозни анализи, за да помогне на търговските представители да разберат вероятността от спечелване на сделки и идентифицира ключови фактори, влияещи върху резултатите от сделката.
- Автоматизирани имейл отговори: Einstein Email Insights анализира имейлите на клиентите и предлага подходящи отговори, което позволява на търговските представители да се ангажират с клиентите по-ефективно.
- Предсказуемо прогнозиране: Einstein Forecasting използва алгоритми за машинно обучение, за да генерира точни прогнози за продажби въз основа на исторически данни.

1.3.2 Zoho CRM

Zoho CRM предлага набор от функционалности, подобни на изброените в Salesforce CRM, което го прави популярен избор за фирми, които търсят цялостни решения за управление на взаимоотношенията с клиенти. За сметка на това, че Zoho CRM не разполага с изкуствен интелект като Salesforce Einstein, тази система е снабдена с функционалности за автоматизация на работния процес, които Salesforce няма готови. Zoho CRM позволява на потребителите да създават персонализирани работни потоци, да автоматизират задачи и да задействат механизми въз основа на предварително зададени критерии.

1.3.3 HubSpot CRM

HubSpot CRM предлага набор от функционалности за управление на взаимоотношенията с клиенти, продажби и маркетинг като :

- Управление на контакти: HubSpot CRM предоставя инструменти за организиране и управление на информация за контакти, история на комуникацията и взаимодействия с потенциални клиенти и клиенти.
- Управление на канали: Потребителите могат да проследяват сделки и канали за продажби, което позволява на екипите по продажбите да приоритизират потенциални клиенти, да прогнозират приходи и да управляват ефективно продажбите.
- Имейл интеграция: HubSpot CRM се интегрира безпроблемно с имейл платформи, позволявайки на потребителите да проследяват взаимодействията с клиентите по имейл, да планират последващи действия и да автоматизират имейл кампании директно от CRM интерфейса.
- Маркетингова автоматизация: HubSpot CRM предлага функционалности за автоматизация, позволявайки на потребителите да създават персонализирани маркетингови кампании и да поддържат потенциални клиенти чрез автоматизирани работни процеси.
- Персонализирано отчитане и анализ: Потребителите могат да генерират персонализирани отчети, анализи, да наблюдават дейностите по продажби и да измерват ефективността на маркетинговите кампании.
- Централизирана на всички комуникации с клиенти : Това бива постигнато, чрез Conversations inbox. Чрез него биват централизирани всички комуникации с клиенти, включително имейли, чатове на живо и съобщения в социалните медии, в една обединена входяща кутия. Тази функционалност позволява на потребителите да управляват и отговарят на

клиентски запитвания по множество канали от един интерфейс.

Глава II.

Функционални изисквания на системата. Избор на развойни средства. База данни. Структура на системата.

2.1 Функционални изисквания

- Системата да поддържа различни инстанции за нейните клиенти, наречени системни клиенти :
 - Всяка инстанция разполага с отделни потребители, наречени оператори, които управляват клиентите и техните данни
 - Всяка инстанция има отделни клиенти
- Управление на потребителите :
 - Администратори в системата, които могат да създават, променят, изтриват оператори (акаунти) за отделните системни клиенти.
 - Оператори, които могат да създават, променят, изтриват акаунти за конкретната инстанция.
 - Възможност за вписване ; оторизация ; сигурност на акаунта :
 - Генериране на JWT Token

- Генериране на Refresh Token
 - Хеширане на паролите в базата данни
- Управление на потребителските права:
 - Администраторска роля, която има контрол над :
 - Системни клиенти :
 - Създаване
 - Променяне
 - Изтриване
 - Всички оператори (потребители), независимо от инстанцията
 - Всички клиенти, независимо от инстанцията
 - Всички данни и статистики, независимо от инстанцията
 - Всички роли и права, , независимо от инстанцията
 - Възможност за създаване на роли с права за :
 - Потребителски акаунт :
 - Създаване
 - Променяне
 - Изтриване
 - Обвързване с оператор в конкретната инстанция
 - Оператори :
 - Създаване
 - Променяне
 - Изтриване
 - Обвързване с потребителски акаунт
 - Обвързване с конкретния системен клиент
 - Клиенти :

- Създаване
- Променяне
- Изтриване
- Изпращане на съобщения
- Проследяване на взаимоотношения

■ Роли :

- Създаване
- Променяне
- Изтриване
- Обвързване с права

■ Права :

- Създаване
- Променяне
- Изтриване
- Обвързване с роли

- Управление на данни за клиенти :
 - Наличие на данни (например имейли, имена, телефонни номера) за клиентските акаунти в конкретната инстанция.
- Управление на клиентските акаунти.
 - Възможност за регистрация и вписване ; оторизация ; сигурност на акаунта :
 - Генериране на JWT Token
 - Хеширане на паролите в базата данни
 - Управление на статуса на акаунта. Статусите могат да бъдат :
 - Активен - клиентът има достъп до приложението на системния клиент.
 - Деактивиран - клиентът няма достъп до приложението на системния клиент ; акаунтът му ще бъде изтрит в срок

от 30 дни. Ако в рамките на това време се впише в акаунта си, то той отново ще бъде активиран.

- Блокиран - достъпът на клиента до приложението на системния клиент е временно преустановен.
- Изчакващ активация - предстои одобрение за достъп до приложението на системния клиент.

- Проследяване на взаимоотношенията с клиентите :
 - Водене на записки за клиентите в конкретната инстанция
 - Изпращане на имейл при създаване на акаунт в приложението на системния клиент
 - Изпращане на имейл при промяна на статуса на акаунта.
- Възможност за съвместна работа между приложно-програмния интерфейс и third-party приложно-програмен интерфейс / клиент интерфейс, принадлежащи на системния клиент :
 - Ключ за достъп до приложно-програмния интерфейс, с който ще бъде снабден всеки нов системен клиент
- Скалируемост и висока отказоустойчивост на системата.

2.2 Избор на развойни средства

2.2.1 Visual Studio Code

Visual Studio Code или накратко VS Code е един от най-честите избори за интегрирана среда за обработка, когато става въпрос за създаване на уеб приложения. Причините за избор на тази среда са :

- Гъвкавост: VS Code поддържа широк набор от езици за програмиране, често използвани в уеб разработката, включително HTML, CSS, JavaScript, TypeScript и др. Неговата

гъвкавост позволява работата върху различни проекти в рамките на една интегрирана среда за разработка.

- Разширяемост: Благодарение на огромна колекция от разширения, налични в Visual Studio Code Marketplace, средата за разработка може да бъде персонализирана. Те могат да предоставят възможност за подчертаване на синтаксис и кодови фрагменти, както и инструменти за отстраняване на грешки и интеграция за контрол на версиите.
- IntelliSense - набор от функции, включващ завършването на код, информация за параметри на функции и проверка за грешки в реално време.

2.2.2 NestJS

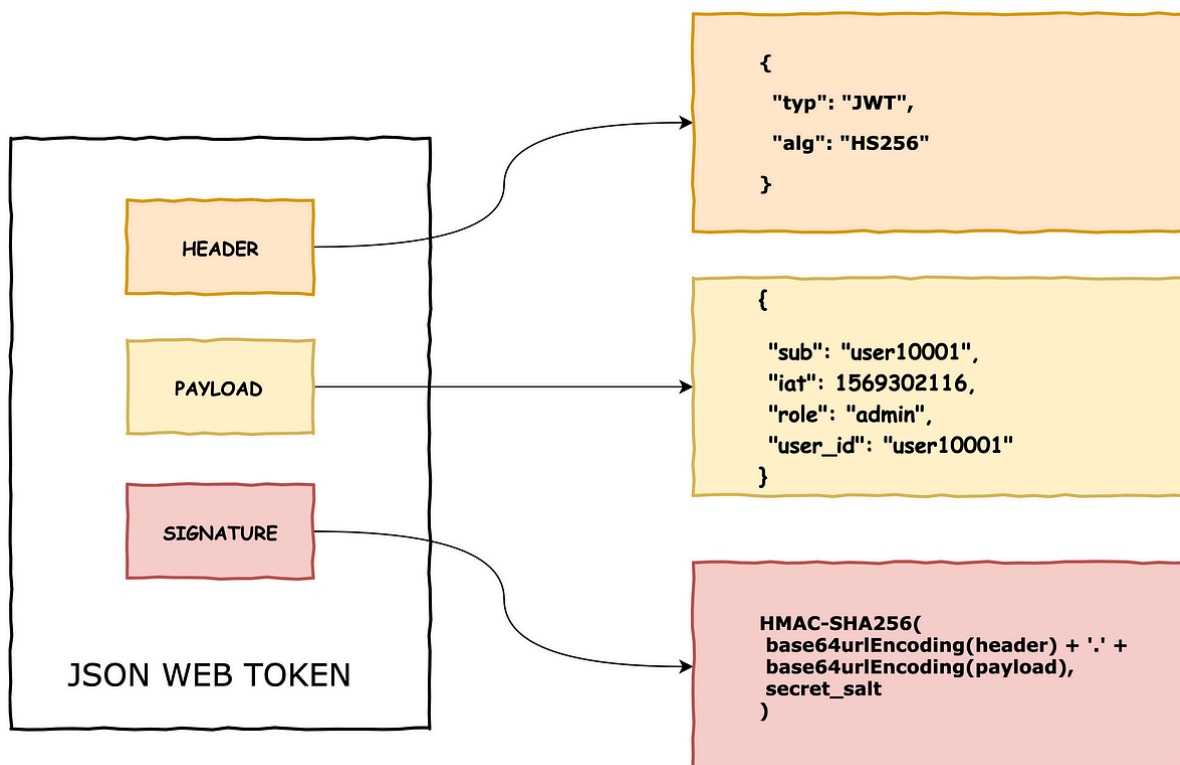
За разработка на приложно-програмният интерфейс ще бъде използван NestJS. Това е рамка, базирана на Node.js, предназначена за създаване на скалируеми, надеждни приложно-програмни интерфейси. Причините за избор са :

- Съвременна поддръжка на TypeScript: NestJS е изграден чрез TypeScript, който предлага типизиране и модерни функции на ECMAScript. Благодарение на TypeScript е предоставено по-добро type safety в системата.
- Модулна архитектура, вдъхновена от Angular, която позволява организацията на приложенията в модули, контролери, услуги и междинен софтуер за многократна употреба. Модулният подход насърчава преизползването на кода и мащабируемостта, което улеснява поддържането и разширяването на по-сложни приложения.

- Dependency injection система, която опростява управлението на зависимостите в различните модули, което на свой ред води до лесно тестваем код.
- Безпроблемна интеграция с популярни рамки за уеб сървъри като Express и Fastify.
- Богата екосистема и набор от модули, поддържащи всевъзможни функционалности.
- Поддръжка за използването на популярни рамки като Jest, което улеснява тестването на отделните модули, гарантирайки надеждност в уеб приложението през целия цикъл на разработка.

2.2.3 JWT

JSON уеб токенът (*фиг. 2.1*), известен под съкращението JWT, представлява отворен стандарт (RFC 7519) за сигурно предаване на информация между две комуникиращи среди като JSON обект. Бива дигитално подписан с помощта на частен и/или чифт публични ключове и криптографски алгоритми като HMAC, RSA или ECDSA.. Целта на JWT не е скриването на данни, а по-скоро е да осигури автентичността на предаваните данни.



фиг. 2.1 Структура на Json Web Token

JWT се използва като stateless механизъм за автентикация, базиран на токени. Тъй като е stateless клиентска сесия, сървърът не трябва да разчита напълно на база данни, за да запази информацията за нея. Автентикацията е stateless, което създава минимален overhead, правейки JWT добро решение в контекста на скалируемостта. Токенът е компактен, което позволява лесното му предаване чрез HTTP заявки или URL параметри. NestJS разполага с вградена поддръжка за автентикация и оторизация, базирана на JWT, което го прави добър избор в стека на работа с приложението.

2.2.4 CASL

Common Ability Schema Language или CASL накратко е JavaScript библиотека за авторизация, широко използвана в контекста на RBAC и ABAC. Предоставя гъвкав начин за дефиниране и проверка на права в

рамките на приложението, така че да могат ясно да бъдат разделени възможностите на отделните потребители според ролите им, атрибутите им и други. CASL предоставя възможността за дефиниране на динамични права, което означава създаване на правила, които се изменят по време на runtime-а. Потребителските права в контекста на библиотеката се наричат ability-та. Всяко ability разполага с :

- subject - обект (например клас от базата данни), над който потребителят може да извърши определени операции.
- action - операцията, която потребителят може да извърши над обекта
- conditions - условията, под които операцията може да бъде извършена

Ability се дефинира по следния начин :

```
defineAbility((can, cannot) => {  
  can('read', 'Post');  
  can('update', 'Post');  
  can('read', 'Comment');  
  can('update', 'Comment');  
});
```

фиг. 2.2 дефиниране на ability в CASL

Ability се проверява по един от тези начини :

```
//дали потребителят има конкретното ability  
ability.can('update', 'Post')  
  
//дали потребителят няма конкретното ability  
ability.cannot('update', 'Post')
```

```
//хвърли exception, освен ако потребителя има  
конкретното ability  
ability.throwUnlessCan('update', 'Post')
```

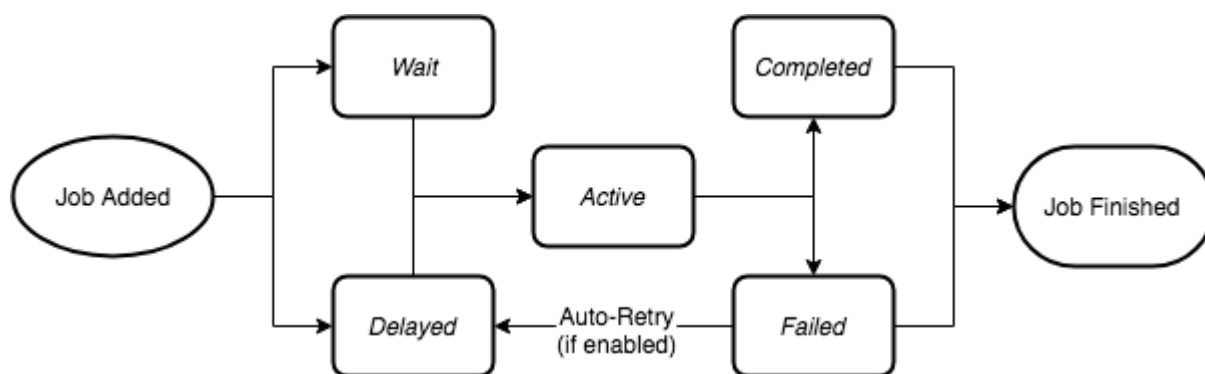
фиг. 2.3 проверка на ability в CASL

Използвайки междинния софтуер и декораторите, предоставени от NestJS, CASL безпроблемно може да се интегрира като средство за проверка на потребителските права в рамките на системата, което го прави добър избор за конфигурирането на RBAC.

2.2.5 Bull

Bull е надеждна опашка за процеси за Node.js. Дава възможността за асинхронно обработване на фонові процеси и задачи, обвързани с тях, наречени job-ове. Така се постига “разтоварването” на времеемки или ресурсоемки процеси от основната нишка на приложението, позволявайки ефективното извършване на фоновите процеси, като например изпращане на имейли, обработка на изображения или изпълнение на операции с бази данни. Използването на опашка за процеси е подход, който допълнително осигурява скалируемостта на приложението. Bull предоставя контрол над едновременната обработка на процесите, като по този начин се осигурява безпроблемна работа при по-високи натоварвания. Процесите в опашката могат да бъдат приоритизирани, както и да им бъде зададено отложено изпълнение. Едно от главните предимства и причини за използване на Bull е устойчивостта. Тъй като използва Redis като хранилище, Bull осигурява трайност на данните за процесите, дори и при рестартиране или срыв на приложението. Bull предоставя функционалности за мониторинг и метрики, които позволяват проследяването на показателите за изпълнение

на процесите и следене на състоянието на опашките. NestJS предоставя пакета `@nestjs/bull` като абстракция/обвивка върху Bull. Пакетът улеснява интегрирането на опашките по удобен за Nest начин в приложението. Начинът на работа на опашката за процеси е описан във фиг. 2.4.



фиг. 2.4 Начин на работа на опашка за процеси

2.2.6 Redis

В рамките на разработка на системата ще бъде използван Redis. Това представлява key-value хранилище за данни. Причината за използването на Redis е работата с Bull при разработката на някои от функционалностите.

2.2.7 Handlebars

Handlebars е език за шаблониране без логика, предназначен за генериране на динамично съдържание в уеб приложения. Той използва синтаксис, подобен на Mustache, като капсулира заместителите в двойни къдрави скоби (`{{}}`) и поддържа HTML по подразбиране. Тези заместители или променливи се заменят с действителни стойности по време на изобразяването на шаблона. Освен това Handlebars поддържа помощни функции за въвеждане на логика в шаблоните, като например условни оператори и цикли. Handlebars може да се използва както от

страна на клиента, така и от страна на сървъра, като са налични библиотеки за различни среди. В рамките на разработката на системата Handlebars ще бъде използван за шаблониране на имейлите.

2.2.8 JSON и DTO

JSON, или JavaScript Object Notation, е олекотен формат за обмен на данни, който обикновено се използва за предаване на данни между сървър и клиент. Той представя данните в двойки ключ-стойност и масиви, което улеснява четенето и писането, анализирането и генерирането (фиг. 2.5). Обектите за прехвърляне на данни (Data Transfer Object; DTO) са обекти, използвани за прехвърляне на данни между различни слоеве на приложението, например между клиента и сървъра или между различни части на сървъра. Те служат като структуриран начин за капсулиране и пренасяне на данни. Обектите за прехвърляне на данни улесняват валидирането и трансформирането на данните, гарантирайки целостта им. NestJS разполага с вградени механизми за валидиране и сериализация, което улеснява дефинирането и използването на обекти за прехвърляне на данни в контролери и service-и.

```
{
  "type": "basket",
  "beans": 47,
  "apples": 7,
  "oranges": 23,
  "brand": "ConvertSimple",
  "ratio": 33.9,
  "fees": {
    "cleaning": "$4.50",
    "baking": "$27.30",
    "commission": "$93.10"
  },
  "descriptors": ["clean", "fresh", "juicy", "delicious"]
}
```

фиг 2.5 JSON пример

2.2.9 Docker

Docker позволява създаване, доставяне и изпълнение на приложения в леки, преносими контейнери. Ще бъде използван за контейнеризация на базата данни, на Redis хранилището, както и на самото приложение.

2.2.10 Kubernetes

K8s позволява оркестрирането на контейнери, автоматизиране на deployment-a, мащабирането и управлението на контейнерни приложения. Ще бъде използван за deployment и скалиране на цялата система

2.2.11 Postman

Postman ще бъде използван за тестване на всяка една от функционалностите на приложението без готово имплементиран потребителски интерфейс.

2.3 База данни

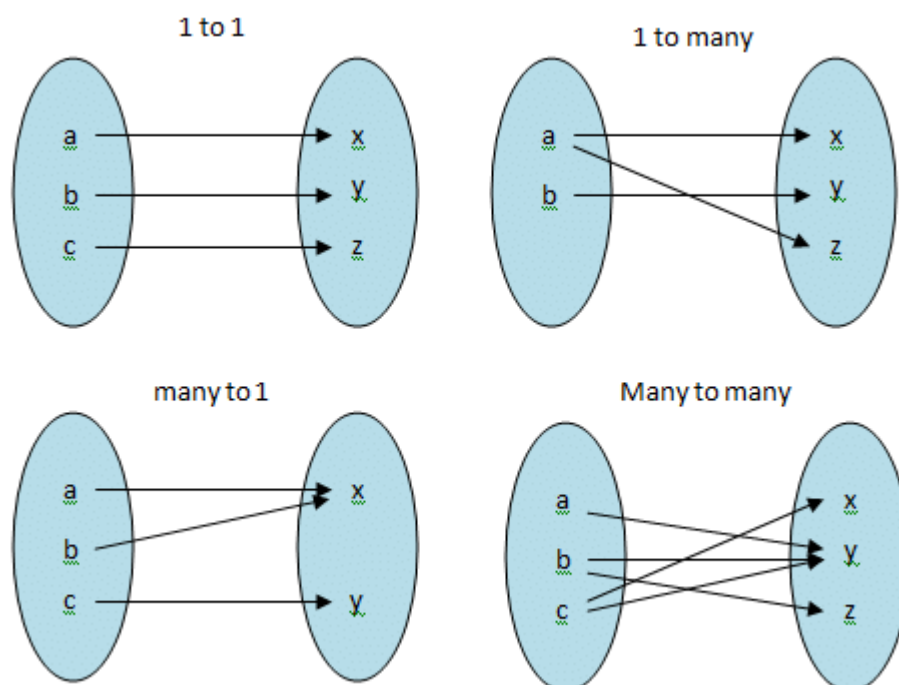
Базата данни е една от най-важните части за един приложно-програмен интерфейс. Тя служи като гръбнак на приложението, улеснявайки съхранението, извличането и управлението на огромни количества структурирани и неструктурирани данни. Има два основни типа бази данни :

2.3.1 Релационни бази данни

Релационните БД са вид СУБД, която организира данните в таблици, състоящи се от редове и колони, като всеки ред представлява запис, а всяка колона - конкретен атрибут или поле. Данните са структурирани и всяка таблица има предварително дефинирана схема. Релационните БД

позволяват установяването на връзки между таблиците чрез ключове, например първични ключове и външни ключове. Тези връзки налагат целостта и последователността на данните, като гарантират, че данните са точно свързани и се поддържат в множество таблици. Те се придържат към ACID (Atomicity, Consistency, Isolation, Durability) принципите, за да гарантират целостта и надеждността на заявките. Релационните бази данни използват SQL като стандартен език. Видовете релации (фиг. 2.6) между отделните таблици са :

- One to one - към един запис от таблица А строго е обвързан един от таблица В и обратното.
- One to many - към един запис от таблица А са обвързани един или повече записи от таблица В
- Many to one - обратното на one to many
- Many to many - към един или повече записи от таблица А са обвързани един или повече записи от таблица В



фиг 2.6 Видове релации

2.3.1 Нерелационни бази данни

Нерелационните или NoSQL БД са вид СУБД, която е предназначена, за съхраняване и управление на големи обеми неструктурирани или полуструктурирани данни. За разлика от релационните бази данни, които организират данните в таблици с предварително дефинирани схеми, нерелационните бази данни предлагат по-голяма гъвкавост при моделирането и съхранението на данни. Те са проектирани да са хоризонтално скалируеми, което означава, че могат да обработват големи обеми данни и високи нива на трафик чрез разпределяне на данните между множество сървъри или възли в клъстер. При повечето модерни NoSQL БД данните се репликират в няколко възела в клъстера, което гарантира, че системата ще продължи да функционира дори в случай на хардуерни повреди или мрежови проблеми. Това осигурява висока отказоустойчивост и толеранс към грешки.

2.3.2 Избор на база данни

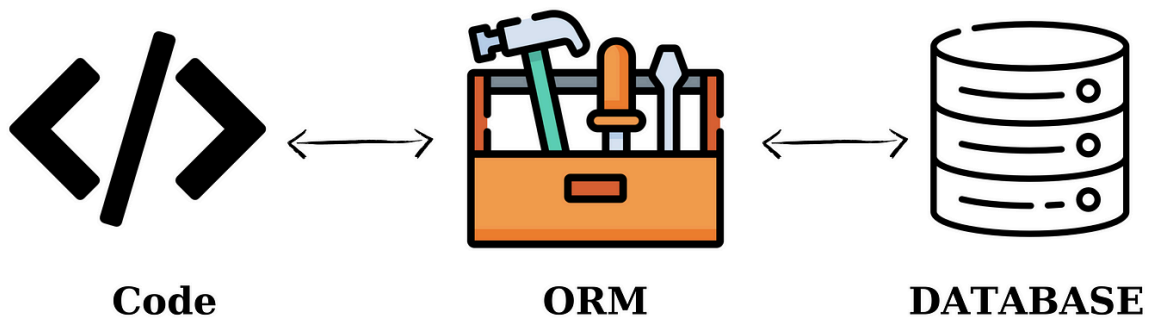
Според функционалните изисквания на системата следва, че данните трябва да бъдат структурирани, както и че ще има връзки между отделните таблици. Заради това за разработка на приложно-програмния интерфейс е избрана системата за управление на релационни бази данни PostgreSQL.

2.3.3 Осъществяване на връзка с база данни

Двата най-често срещани метода за осъществяване на връзка с база данни са чрез драйвери за конкретната база (например прм пакетите cassandra-driver, mysql, postgres) или чрез Object Relational Mapper (ORM; обектно-релационен картограф). ORM са софтуерни библиотеки, които улесняват връзката между обектно-ориентираните езици за програмиране

и релационните бази данни (фиг. 2.7). Преносът на данни между езикът за програмиране и ORM се осъществява чрез Entity класове. Entity-тата представляват класове-инстанции на таблици от базата данни. При извличане на информация чрез ORM-а бива върнат обект от типа съответния Entity клас. Използването на ORM е добра практика, тъй като предоставя ниво на абстракция пред писането на директни заявки към базата данни. Чрез заявки, написани “на ръка”, се рискува компроментирането на данните, тъй като вероятността за SQL инжекция е по-голяма. ORM-ите предоставят вградени функции, които изграждат сигурни заявки към базата. Повечето ORM-и поддържат и конструктори за заявки, чрез които могат да се правят по-сложни заявки, често при извличане на данни с много релации. Конструкторите абстрахират основния синтаксис на SQL и позволяват операции в по-четим и лесен за поддържане формат.

В рамките на разработката на системата ще бъде използван TypeORM за връзка с базата данни. Това е най-зрялата ORM за TypeScript, която поддържа различни системи за бази данни като PostgreSQL, MySQL, MariaDB, SQLite и SQL Server. Предоставя вградена поддръжка за миграции, изготвяйки автоматично скриптовете за миграция въз основа на класовете на таблиците. Има възможност за синхронизация между кода на приложението и схемата на базата данни. NestJS предоставя out-of-the-box интеграция с TypeORM, което го прави подходящ избор за използване в рамките на разработката на системата.



фиг. 2.7 Връзка между програмен език, ORM и база данни

2.3.4 Структура на базата данни

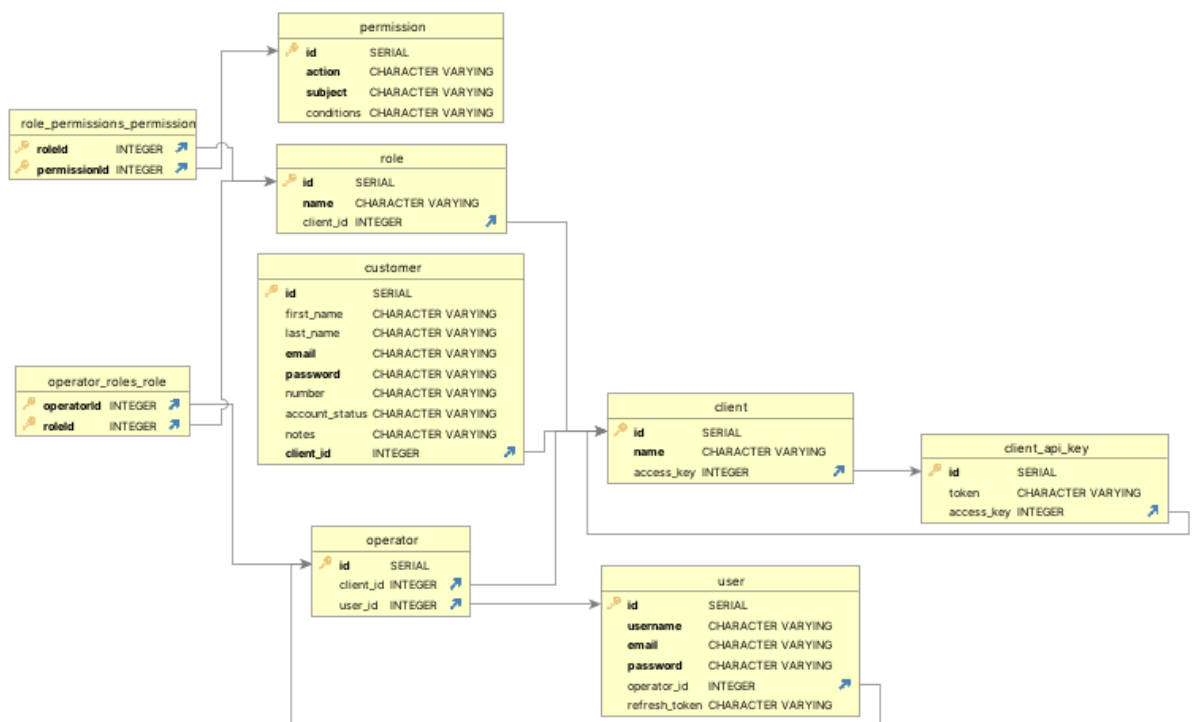


Таблица на системния клиент (фиг. 2.8)

```
@Entity()
@Unique(['name'])
```

```

export class Client {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({nullable: false})
  name: string;

  @OneToMany(() => Operator, (operator) => operator.client, {nullable:
true})
  operators?: Operator[];

  @OneToMany(() => Customer, (customer) => customer.client, {nullable:
true})
  customers?: Customer[];

  @OneToMany(() => Role, (role) => role.client, {nullable: true})
  roles?: Role[];

  @OneToOne(() => ClientApiKey, (api_key) => api_key.client, {nullable:
true, eager: true})
  api_key: ClientApiKey;
}

```

фиг. 2.8 таблица на системния клиент

Във фигура 2.8 е показан обектът зад таблицата на системния клиент. Системният клиент обособява конкретната инстанция на приложението. Една инстанция има много оператори (потребители), асоциирани с нея, които според ролите си управляват ролите, клиентите, както и другите потребители в тази инстанция. Тъй като ролите не са глобални, а се създават от операторите и системните администратори за конкретната инстанция, те са обвързани чрез One To Many релация със системния клиент. Customers представляват клиентите (потребителите) на third party приложението на системния клиент ; много customer-и принадлежат точно към един клиент. API key, представлява ключът за достъп до приложно-програмния интерфейс, като една инстанция е обвързана точно с един ключ. Може да се създаде инстанция, която все още не е изградена,

тоест. все още няма оператори, клиенти или роли. Всеки клиент има уникално име.

Таблица на клиентски ключ за достъп до приложно програмен интерфейс (фиг. 2.9)

```
export class ClientApiKey {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({nullable: true})
  token: string;

  @OneToOne(() => Client, (client) => client.api_key, {nullable: true,
onDelete: 'CASCADE'}, )
  @JoinColumn({ name: 'client_id' })
  client: Client;
}
```

фиг. 2.9 таблица на клиентски ключ за достъп до приложно програмен интерфейс

Във фигура 2.9 е показан обектът зад таблицата на клиентския ключ за достъп до приложно програмния интерфейс. Всеки ключ има уникален token.

Таблица на оператор (потребител в инстанцията) (фиг. 2.10)

```
@Entity()
export class Operator {
  @PrimaryGeneratedColumn()
  id: number;

  @ManyToOne(() => Client, (client) => client.operators, {nullable: true})
  @JoinColumn({ name: 'client_id' })
  client?: Client;

  @OneToOne(() => User, (user) => user.operator, {nullable: true, eager:
true})
  user?: User;

  @ManyToMany(() => Role, (role) => role.operators, {nullable: true})
```

```

@JoinTable()
roles?: Role[];
}

```

фиг. 2.10 таблица на оператор (потребител в инстанцията)

Във фигура 2.10 е показан обектът за таблицата на оператора (потребителя в инстанцията). Всеки оператор може да има много роли, принадлежащи на инстанцията, като различните оператори могат да имат еднакви роли. Всеки оператор е обвързан с точно един потребителски акаунт.

Таблица на потребителски акаунт (фиг. 2.11)

```

@Entity()
@Unique(['username'])
@Unique(['email'])
export class User {
    @PrimaryGeneratedColumn()
    id: number;

    @Column({nullable: false})
    username: string;

    @Exclude()
    @Column({nullable: false})
    email: string;

    @Exclude()
    @Column({nullable: false})
    password: string;

    @OneToOne(() => Operator, (operator) => operator.user, {nullable: true,
onDelete: "CASCADE"})
    @JoinColumn({ name: 'operator_id' })
    operator?: Operator;

    @Column({nullable: true})
    refresh_token?: string;
}

```

фиг. 2.11 таблица на потребителски акаунт

Във фигура 2.11 е показан обектът зад таблицата на потребителски акаунт. Всеки потребителски акаунт има уникални потребителско име и имейл. С оглед на сигурността не всички данни, които се съдържат в един entity обект от базата данни, трябва да бъдат върнати в отговор на заявка. Например не би било благоприятно, ако потребителските парола и имейл биват върнати в отговор на заявка за вземане на потребител. Заради това в entity-тата може да бъде наблюдавано използването на декоратора `Exclude` от библиотеката `"class-transformer"`. Декораторът индикира изключването на конкретното поле от отговора на заявката. Чрез слагане на флага `eager` на релацията между оператор и потребител се осигурява, че при `find` метод тази релация винаги ще бъде заредена. Всеки акаунт е снабден с токен за опресняване при влизане в системата. Използването му ще бъде разгледано в глава III.

Таблица на роля (фиг. 2.12)

```
@Entity()
export class Role {
  @PrimaryGeneratedColumn()
  id: number;

  @Column({nullable: false})
  name: string;

  @ManyToMany(() => Permission, (permission) => permission.roles, {eager:
true, nullable: true})
  @JoinTable()
  permissions?: Permission[];

  @ManyToMany(() => Operator, (operator) => operator.roles, {nullable:
true})
  operators?: Operator[];

  @ManyToOne(() => Client, (client) => client.operators, {nullable: true})
  @JoinColumn({ name: 'client_id' })
  client?: Client;
}
```

фиг. 2.12 таблица на роля

Във фигура 2.12 е показан обектът зад таблицата за роля. Всяка роля има уникално име в контекста на инстанцията. Това е реализирано на логическо ниво извън базата и ще бъде разгледано в глава III. Една роля може да има много права и съответно две различни роли могат да имат едно и също право.

Таблица на правата (фиг. 2.13)

```
@Entity()
export class Permission {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  action: string;

  @Column()
  subject: string;

  @Column({nullable : true})
  conditions?: string;

  @ManyToMany(() => Role, (role) => role.permissions)
  roles: Role[];
}
```

фиг. 2.13 таблица за права

Във фигура 2.13 е показан обектът зад таблицата на правата, с които са снабдени потребителските роли. За разлика от ролите, правата са глобални и могат да бъдат създавани само от системни администратори и разработчици. Това е реализирано на логическо ниво и ще бъде разгледано в глава III. Причината е, че съществуват ограничен набор от права, които може да има една роля, но различните инстанции на системата могат да имат роли с различни наименования и различни права според логиката на тези наименования. Например ролята “Admin” в една инстанция може да има различни права от роля със същото име в друга инстанция.

Таблица на клиентите на приложението на инстанцията (фиг. 2.14)

```
@Entity()
export class Customer {
  @PrimaryGeneratedColumn()
  id: number;

  @Exclude()
  @Column({nullable: true})
  first_name: string;

  @Exclude()
  @Column({nullable: true})
  last_name: string;

  @Exclude()
  @Column({nullable: false})
  email: string;

  @Exclude()
  @Column({nullable: false})
  password: string;

  @Exclude()
  @Column({nullable: true})
  number: string;

  @Column({nullable: true})
  account_status: string;

  @Column({nullable: true})
  notes: string;

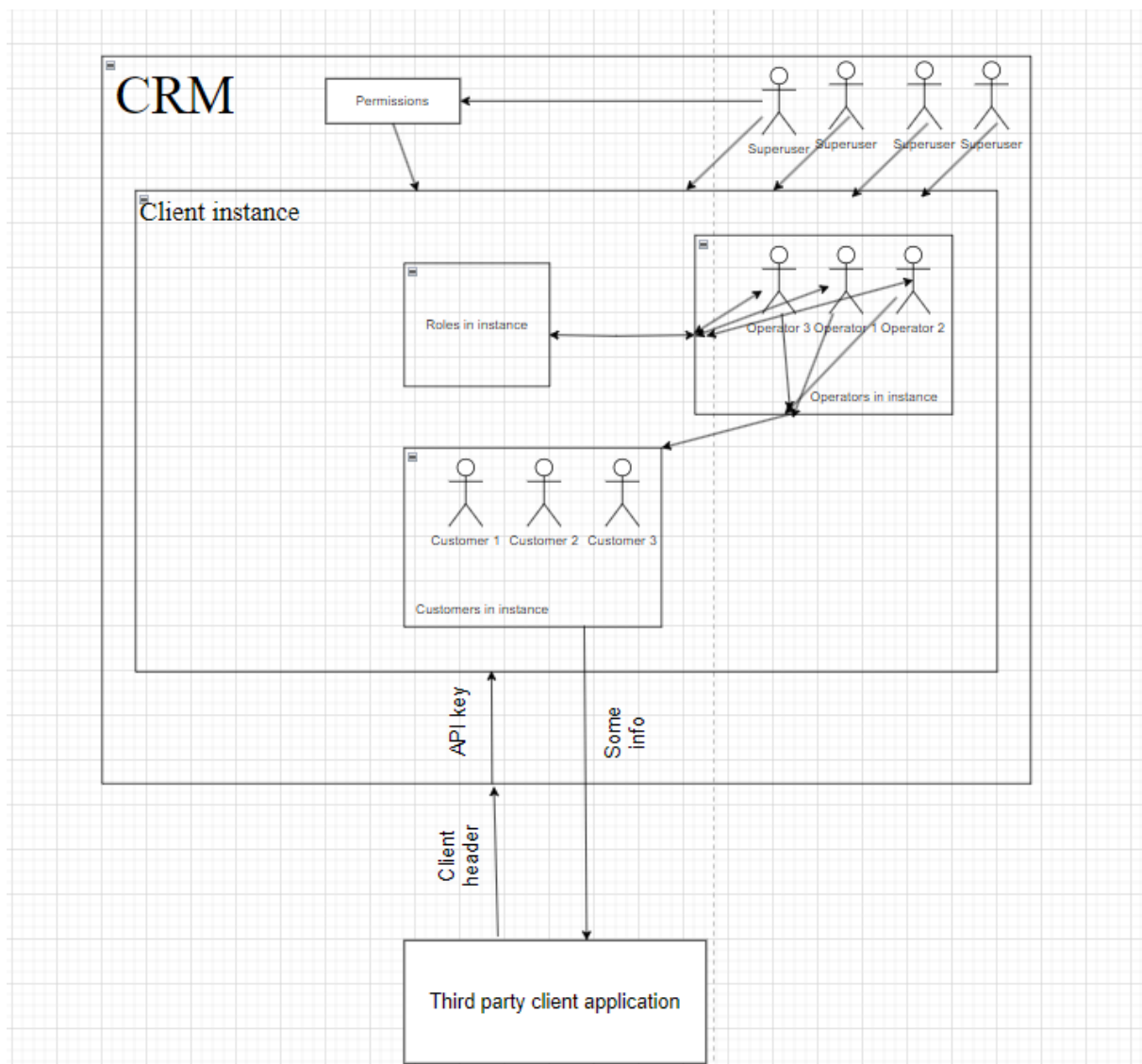
  @ManyToOne(() => Client, (client) => client.customers, {nullable:
false})
  @JoinColumn({ name: 'client_id' })
  client: Client;
```

}

фиг. 2.14 таблица на клиентите на приложението на инстанцията

Във фигура 2.14 е показан обектът зад таблицата на клиента на приложението на инстанцията. Той разполага с полета за имейл и парола с цел предоставяне на готови функционалности на приложно-програмния интерфейс за оторизация и автентикация на потребители за third party приложението на системния клиент. Това ще бъде разгледано по-обстойно в глава III.

2.4 Структура на приложението



фиг. 2.14 Схема на структура на приложението

Във фигура 2.14 е показана схемата на структурата на приложението. Всеки клиент на CRM системата има обособена за него инстанция. В тази инстанция има потребители, наречени оператори, които отговарят за работата с клиентите. Операторите имат различни роли, според които могат да извършват различни дейности. Всяка инстанция има различни създадени роли. Правата, които ролите могат да имат са едни и същи, независимо от инстанцията и заради това са глобални. При създаване на

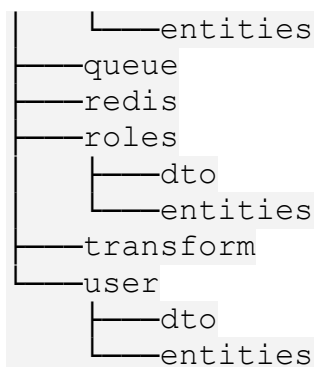
нова инстанция на системата, новият клиент е снабден с ключ за достъп до приложно-програмния интерфейс на CRM системата. При заявка трябва да бъде наличен header x-client - името на инстанцията, заедно с ключа за достъп до нея. Ако е намерена инстанция и ключът е валиден, то заявката бива обработена и съответно се връща информация на third party приложението. Така примерно системата може да бъде използвана като готов модул за регистрация и вписване на клиенти в приложението на системния клиент, като ако клиент не съществува в инстанцията в системата, то той ще бъде създаден. Това ще бъде разгледано по-обстойно в глава III. Superuser-ите представляват админските акаунти в системата. Те имат права за всичко, освен създаването на други superuser-и - това могат да правят само разработчиците. Superuser-ите работят със системните клиенти. Те създават инстанциите, създават началните роли и оператори и създават начални клиенти, ако трябва да има такива. Superuser-ите отговарят за създаването на различни права и могат да модерират цялата система.

Глава III.

Реализация на системата

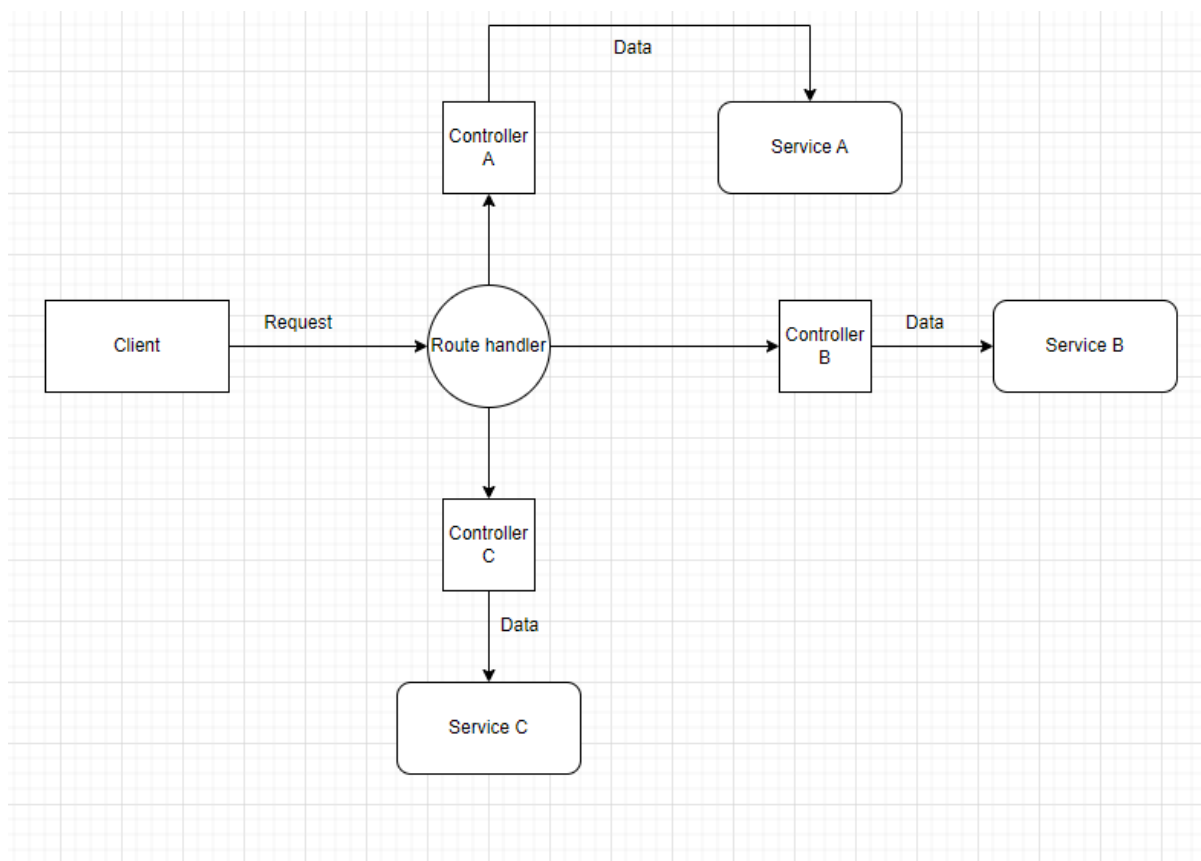
3.1 Файлова структура

```
src
├── auth
│   └── dto
├── client
│   ├── dto
│   └── entities
├── client-api-key
│   ├── dto
│   └── entities
├── customer
│   ├── dto
│   └── entities
├── customer-status
│   └── dto
│       └── validation
├── decorators
│   ├── ability
│   ├── allow-unauthorized-request
│   ├── require-api-key
│   └── require-superuser
├── enums
├── guards
│   ├── ability
│   ├── auth
│   ├── client
│   │   └── api-key
│   └── superuser
├── interfaces
│   └── requests
├── mail
│   ├── dto
│   └── templates
├── migrations
├── operator
│   ├── dto
│   └── entities
├── permissions
│   └── dto
```



Това е файловото дърво на папката `"src"` в системния код. В тази папка се намира почти всичкият сорс код, използван за реализация на приложението. С оглед на количеството файлове, са представени само папките и подпапките на файловото дърво. Следван е моделът на NesJS за файлова структура. Отделните модули, заедно с техните контролери, service-и, listener-и и така нататък са енкапсулирани в една папка.

Разделянето на модули подпомага организирането на приложението на по-малки, управляеми части. Всеки модул капсулира свързани компоненти, като контролери, провайдъри (service-и, репозитории) и други свързани модули. Контролерите представляват онези компоненти на модула, които са отговорни за обработването на заявките от клиента. Service-ите са онези компоненти от модула, които отговарят за логиката зад контролерите. Те капсулират бизнес логиката на приложението. Отговарят за операции, свързани с данни, за взаимодействие с бази данни, third party приложно-програмни интерфейси или други външни ресурси и за изпълнение на задачи, които се изискват от модулите или контролерите на приложението. Когато сървърът получи заявка, той я обработва и я изпраща към съответния контролер. На крайната точка на съответния контролер нужните данни от заявката биват предадени на функция от service-а. Така се осъществява изпълнението на логиката зад контролера (фиг. 3.1).



фиг. 3.1 Обработка на заявка ; връзка между контролер и service

3.2 Помощни константи, еnumерации, интерфейси

3.2.1 Константи

```
export const SUPERUSER = 'superuser';
export const CHANGE_STATUS_LENGTHS = {
  MINUTE: 60 * 1000,
  TEN_MINUTES: 10 * 60 * 1000,
  HOUR: 60 * 60 * 1000,
  TEN_HOURS: 10 * 60 * 60 * 1000,
  DAY: 24 * 60 * 60 * 1000,
  WEEK: 7 * 24 * 60 * 60 * 1000,
  MONTH: 30 * 24 * 60 * 60 * 1000,
  HALF_YEAR: 6 * 30 * 24 * 60 * 60 * 1000,
  YEAR: 365 * 24 * 60 * 60 * 1000
};
export const DURATION_WORD_KEYS = {
  MINUTE: '1 minute',
  TEN_MINUTES: '10 minutes',
```

```
HOUR: '1 hour',  
TEN_HOURS: '10 hours',  
DAY: '1 day',  
WEEK: '1 week',  
MONTH: '1 month',  
HALF_YEAR: '6 months',  
YEAR: '1 year'  
};
```

фиг. 3.2 Константи, използвани в приложението

Във фигура 3.2 са разгледани всички константи, използвани за реализация на приложението. Използването на константи подобрява качеството на кода, улеснявайки неговата поддръжка, тъй като при нужда от промени в стойности, които се ползват на много места, промяната е само една. Първата константа представлява резервираното име на ролята на админите в приложението. Роли с име тази константа не могат да бъдат създавани - тя е само една, глобална за цялото приложение. Следващите четири константи представляват “флагове”, които декораторите в приложението добавят като метаданни на контекста на заявката. Константата `CHANGE_STATUS_LENGTHS` представлява съвкупност от ключове и стойности за различни времеви периоди, изразени в милисекунди. По-добре разписана, формулата за превръщане на минути в милисекунди е: `Milliseconds = Minutes × 60 × 103`. Тези стойности ще бъдат използвани при промяна на статуса на клиентите. Константата `DURATION_WORD_KEYS` представлява съвкупност от ключове и стойности за различни времеви периоди и думовото оформление, което отговаря за тях. Тези стойности ще бъдат използвани при изпращане на имейли за промяна на статуса на клиентите.

3.2.2 Енумерации

Всички енумерации, използвани за реализация на приложението се намират в папката `enums` :

```
export enum AccountStatus {  
    PENDING_ACTIVATION = 'Pending',  
    ACTIVE = 'Active',  
    BANNED = 'Banned',  
    DEACTIVATED = 'Deactivated',  
}
```

фиг. 3.3 енумерация за статус на клиентския акаунт

Енумерацията във фигура 3.3 ще бъде използвана при промяна на статуса на акаунта на клиентите на инстанцията. Статусите могат да бъдат :

- `PENDING_ACTIVATION` - изчакващ активация ; този статус е даден на акаунт, който е току - що създаден, или който преди това е бил със статус `BANNED`, тоест блокиран. При създаване на акаунт отнема 1 минута за автоматичната му активация и изпращане на имейл за конкретната промяна.
- `ACTIVE` - активен
- `BANNED` - блокиран
- `DEACTIVATED` - деактивиран ; когато този статус е зададен на акаунт, по желание на клиента на инстанцията или по желание на администраторите на инстанцията, акаунтът е насрочен за изтриване 30 дни след началната деактивация. Ако в рамките на тези 30 дни потребителят влезе в акаунта си, процесът по деактивация е спрял.

```
export enum DecoratorMetadata{  
    CHECK_ABILITY = 'checkAbility',  
    UNAUTHORIZED_REQUEST_DECORATOR = 'allowUnauthorizedRequest',  
    REQUIRE_API_KEY = 'requireApiKey',  
    REQUIRE_SUPERUSER_ROLE = 'requireSuperuserRole'  
}
```

фиг. 3.4 енумерация за метаданни на декоратори

Енумерацията във фигура 3.4 ще бъде използвана за добавяне на метаданни, “флагове”, от съответните декоратори на крайната точка към контекста на заявката. Според тези метаданни ще бъдат извършвани функции и проверки, обвързани със смисъла на всеки един от декораторите :

- `CHECK_ABILITY` - това представляват метаданните, които ще бъдат добавяни от декоратора за проверка на потребителските права.
- `UNAUTHORIZED_REQUEST_DECORATOR` - това представляват метаданните от декоратор за заявки, които не изискват потребителя да е оторизиран, тоест да не е вписан. Такава заявка е заявката за вписване в системата.
- `REQUIRE_API_KEY` - това представляват метаданните от декоратор за заявки, които изискват ключ за достъп до приложно-програмния интерфейс, който се предоставя при създаване на инстанция. Той е валиден за един месец, след което може да бъде опреснен от админите на системата. Такива заявки са предназначени за използване от third-party потребителски или приложно-програмен интерфейс на системния клиент. Заявки, изискващи ключ за достъп са :
 - За вписване на клиент на системния клиент
 - За регистрация на клиент на системния клиент
 - Деактивиране на акаунта на клиент на системния клиент
- `REQUIRE_SUPERUSER_ROLE` - това представляват метаданните от декоратор за заявки, до които имат достъп единствено и само админските акаунти системата. Тези акаунти имат достъп до всички инстанции и имат всички права за операции в тях. Заявки, които изискват админски права са :

- Всички операции със системния клиент
- Създаване и опресняване на ключ за достъп на системния клиент до приложно-програмния интерфейс
- Вземане на информация за всички клиенти от конкретна инстанция по нейния идентификационен номер
- Вземане на информация за всички оператори от конкретна инстанция по нейния идентификационен номер
- Всички операции, свързани с правата, които могат да имат ролите
- Вземане на информация за всички роли от конкретна инстанция по нейния идентификационен номер
- Вземане на информация за всички потребителски акаунти от конкретна инстанция по нейния идентификационен номер

```
export enum EntityService {
  OPERATOR = 'operator',
  PERMISSION = 'permission'
}
```

фиг. 3.5 енумерация за service, използван от listener

Ключовете на енумерацията във фигура 3.4 ще бъдат използвани от класа `RolesListener`, като флагове за избиране дали ще бъде използвана функционалността за добавяне на роля от service-ът на операторите или този на правата.

```
export enum SubjectActions {

    CREATE = 'create', //can create entity
    READ = 'read', //can get entity
    UPDATE = 'update', //can update entity
    DELETE = 'delete' //can delete entity
}
```

фиг. 3.6 еnumерация за действия, които могат да бъдат извършвани върху обектите в базата

Ключовете на еnumерацията във фигура 3.5 ще бъдат използвани в декоратора за права и съответният му guard за обозначаване на правата за операции върху конкретния обект, които са нужни за достъп до конкретната крайна точка.

3.2.3 Интерфейси

Всички интерфейси, използвани за реализация на приложението се намират в папката `interfaces`. За момента е нужен един интерфейс :

```
export interface UserRequest extends Request{
    user : any;
}
```

фиг. 3.7 интерфейс за заявка с потребител

Тъй като интерфейсът Request от приложно-програмния интерфейс Fetch, предоставен от JavaScript за манипулация на заявки и отговори, няма поле user, а това поле ще бъде нужно в много от заявките и всички guard-ове, е направен интерфейс, който наследява от Request и добавя това поле.

3.3 Помощни функционалности

3.3.1 Връзка с базата данни

Модулят `app.module.ts` служи като основен модул в приложението. Той определя зависимостите и структурата на системата. В този модул се осъществява връзката с базата данни (фиг. 3.8).

```
TypeOrmModule.forRootAsync({
  inject: [ConfigService],
  useFactory: async (configService: ConfigService) => ({
    type: 'postgres',
    host: configService.get('DB_HOST' /*'DB_DEPLOYMENT_HOST'*/),
    port: configService.get<number>('DB_PORT'),
    username: configService.get<string>('DB_USER'),
    password: configService.get<string>('DB_PASSWORD'),
    database: configService.get<string>('DB_NAME'),
    entities: ['dist/**/*.entity{.ts,.js}'],
    synchronize: true,
  }),
}),
```

фиг. 3.8 осъществяване на връзка с базата данни

С оглед на сигурността, данните за базата са запазени в `.env` файл. `ConfigService` е service от `@nestjs/config`, чрез който може да бъде извлечена информацията от `.env` файла. За хост на базата се взима или конфигуриран за локално пускане, или за такъв в мрежа от контейнери, от които са част сървърът и базата. `Entities` представлява локацията на `entity` класовете, с които са обвързани таблиците в базата данни. `Synchronize` флага определя дали да бъдат синхронизирани таблиците с `entity` класовете в приложението. Когато този флаг е `true`, TypeORM автоматично ще създава таблици в базата данни, базирани на класовете. Конфигурацията за връзка с базата при осъществяване на миграции се намира във файла `typeorm.config.mjs` (фиг. 3.9)

```
import { DataSource } from 'typeorm';
import { ConfigService } from '@nestjs/config';
import { config } from 'dotenv';

config();

const configService = new ConfigService();

export default new DataSource({
  type: 'postgres',
  host: configService.get('DB_HOST' /*'DB_DEPLOYMENT_HOST'*/),
  port: configService.get('DB_PORT'),
  username: configService.get('DB_USER'),
  password: configService.get('DB_PASSWORD'),
  database: configService.get('DB_NAME'),
  entities: ['dist/**/*.entity{.ts,.js}'],
  synchronize: false,
  migrations: ['dist/migrations/*.js'],
});
```

фиг. 3.8 конфигурация за връзка с базата данни при миграции

`Migrations` представлява локацията на миграциите към базата данни. Миграция представлява съвкупност от генерирани от ORM-а заявки, които променят таблици, ограничения, релации и други. Тук `synchronize` флага е `false`, защото тази конфигурация се използва за ръчни миграции и не трябва да се изпълняват автоматично.

3.3.2 Връзка със склада данни

Складът данни е нужен за правилното функциониране на Bull опашката. Запазването на процесите в него осигурява безпроблемното им

протичане дори и след рестартиране или срыв на системата. Връзката (фиг. 3.9) се осъществява в модула `redis.module.ts`.

```
import { Module, Global } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import Redis from 'ioredis';

@Global()
@Module({
  providers: [
    {
      provide: 'REDIS',
      useFactory: (configService: ConfigService) => {
        return new Redis({
          host: configService.get<string>('REDIS_HOST'
/* 'REDIS_DEPLOYMENT_HOST' */),
          port: configService.get<number>('REDIS_PORT'),
        });
      },
      inject: [ConfigService],
    },
  ],
  exports: ['REDIS'],
})
export class RedisModule {}
```

фиг. 3.9 осъществяване на връзка със склад данни

`Global` декораторът прави обхвата на модула в рамките на кода глобален. Това означава, че модулет може да бъде използван в цялото приложение без да е нужно да бъде експортиран повторно. `Providers` представляват провайдерите, които модулет ще направи достъпни в рамките на приложението. С `provide: 'Redis'` е означен токенът, чрез който ще бъде inject-иран `Redis` провайдър от модула в компонентите и service-ите, които ще го използват. Чрез `exports` са определени провайдерите, които ще бъдат експортирани. В случая това е `Redis` провайдър, което означава, че всеки модул, който импортира `Redis`

модула може да използва токена на провайдъра, за да inject-ира инстанцията му.

```
BullModule.forRootAsync({
  inject: [ConfigService],
  useFactory: async (configService: ConfigService) => ({
    redis: {
      host: configService.get<string>('REDIS_HOST',
/*REDIS_DEPLOYMENT_HOST*/),
      port: configService.get<number>('REDIS_PORT'),
    },
  })
}),
```

фиг. 3.10 Bull модул

Във фигура 3.10 може да бъде разгледана конфигурацията на Bull модулът за връзка със склада данни. Това е модулът, който се използва за опашката за процеси. Конфигурацията е аналогична на тази за връзка със склада.

3.3.3 Опашка за процеси

Опашката за процеси в приложението се използва за насрочване на имейли, за насрочване на промяна на статус на акаунт и за капсулиране на натоварващи системата процеси с цел подобряване на издръжливостта ѝ под натоварване.

```
@Injectable()
export class QueueService {
  private logger = new Logger(QueueService.name);
  queues: { [key: string]: Bull.Queue } = {};
  opts: any;

  constructor(private readonly configService: ConfigService) {
    let client: any;
    let subscriber: any;
    this.opts = {
      // redisOpts here will contain at least a property of connectionName which
      // will identify the queue based on its name
      createClient: function (type: any, redisOpts: any) {
        switch (type) {
          case 'client':
            if (!client) {
```

```

        client = new Redis(
            configService.get<string>('REDIS_HOST'
/*'REDIS_DEPLOYMENT_HOST'*/) as string, {
                ...redisOpts,
                maxRetriesPerRequest: null,
                enableReadyCheck: false,
            });
    }
    return client;
case 'subscriber':
    if (!subscriber) {
        subscriber = new Redis(
            configService.get<string>('REDIS_HOST'
/*'REDIS_DEPLOYMENT_HOST'*/) as string, {
                ...redisOpts,
                maxRetriesPerRequest: null,
                enableReadyCheck: false,
            });
    }
    return subscriber;
case 'bclient':
    return new Redis(
        configService.get<string>('REDIS_HOST'
/*'REDIS_DEPLOYMENT_HOST'*/) as string, {
            ...redisOpts,
            maxRetriesPerRequest: null,
            enableReadyCheck: false,
        });
default:
    throw new Error('Unexpected connection type: ');
}
},
};
}

```

фиг. 3.11 конструктор на опашката

`QueueService` е класът, който ще бъде използван за менажиране на всички опашки за процеси. Към него има закачен `Logger`, който ще бъде използван за съобщения в конзолата при грешки в процесите. Има масив `queues`, който ще бъде използван като lookip таблица за отделните опашки за процеси. `Opts` представляват опции, които се използват за конфигурация на отделните опашки. Конструктора (фиг. 3.11) инициализира `opts`, като създава функцията `createClient`. Тази

функция се използва от Bull за създаване на клиенти за връзка с Redis хранилището от данни, което се използва като хранилище за опашките и процесите в тях. Връзката се осъществява с хост (в първия аргумент на Redis инициализацията), определен от .env файла. Нужно е и поне името на опашката, с която се осъществява връзка, подадено като .redisOpts. EnableReadyCheck е флаг, който определя дали Redis клиента да проверява дали Redis сървър е готов за приемане на заявки. По подразбиране този флаг е true, но тъй като в случая е гарантирано, че Redis сървърът е готов за приемане на заявки, може да бъде зададен като false, което намалява натоварването на системата, тъй като при връзка няма всяка опашка да изпраща ping към Redis сървър. EnableReadyCheck е флаг, който определя максималния брой опити за изпращане на заявка към Redis сървър в случай на проблеми с връзката или сризове. В случая, ако една заявка е неуспешна, тя няма да бъде повторена.

```
getQueue(name: string): Bull.Queue {
  if (this.queues[name]) {
    return this.queues[name];
  }

  this.queues[name] = new Bull(name, this.opts);

  return this.queues[name];
}

async add(
  queue: Bull.Queue,
  name: string,
  data: any,
  opts: Bull.JobOptions = {},
) {
  if (!queue) {
    return;
  }
  return queue.add(name, data, {
    removeOnComplete: true,
    removeOnFail: true,
  });
}
```



```

        ...opts,
    });
}

async remove(queue: Bull.Queue, jobID: string){
    if(queue){
        const job = await queue.getJob(jobID);
        if(job){
            await job?.remove();
            return true;
        }
        return false;
    }
    return false;
}

```

фиг. 3.12 Вземане на опашка ; добавяне на задача ; премахване на задача

Фигура 3.12 представлява функциите :

- `getQueue` - вземане на опашка,
- `add` - добавяне на задача към опашката
- `remove` - премахване на процес от опашката.

`GetQueue` функцията проверява дали в масива от опашки вече съществува опашка с име подадено в аргументите. Ако такава опашка съществува, то тя бива върната. В противен случай е създадена и след това върната.

`Add` функцията проверява дали подадената опашка съществува. Ако съществува връща резултата от `queue.add` функцията. `Queue.add` използва аргументите име, по което да бъде открит процеса, с който е асоциирана, данни, които процеса да приеме за изпълнението си и опции за задачата. При успешно добавяне на задача за изпълнение в опашката, тази функция връща инстанция на `job` (задача от процес в опашката) с полета `id`, `data` и други. Флаговете `removeOnFail` и

`removeOnComplete` определят дали задачата да бъде премахната от опашката, когато се провали или когато обработката ѝ е приключила.

`Remove` функцията проверява дали подадената опашка от аргументите съществува. Ако съществува, опитва да намери задача в опашката с идентификационен номер `jobID`. Ако успешно е намерена задачата, е извикан метода `remove`, който премахва задачата от опашката, след което функцията връща `true`, за индикация, че премахването е протекло успешно. Във всеки друг случай функцията връща `false`.

```
createProcess(queue: Bull.Queue, name: string, callback: Function,
onCompleteCallback?: Function) {
  if(queue){
    queue.process(
      name,
      this.processJobWrapper.bind(this, {callback,
      name,
      onCompleteCallback}),
    );
  }
}

async processJobWrapper(
  data: { callback: Function; onCompleteCallback?: Function; name: string },
  job: Bull.Job,
) {
  try {
    await data.callback(job);
    if (data.onCompleteCallback) {
      await data.onCompleteCallback(job);
    }
  } catch (err) {
    this.logger.error(
      `Error in job ${data.name} occurred with args ${JSON.stringify(
        job.data,
      )}\n Error: ${err}`,
      err.stack,
    );
    throw err;
  }
}
```

фиг. 3.13 Създаване на процес ; обработка на задача от процес

Фиг. 3.13 представлява функциите:

- `createProcess` - създаване на процес в опашката
- `processJobWrapper` - обработка на задача, обвързана с процеса

В `createProcess`, чрез функцията `queue.process`, се създава нов процес с име, подадено в аргументите на функцията. Втория аргумент `queue.process` функцията представлява функция, която ще бъде използвана за обработка на задачата, обвързана с процеса. Това е `processJobWrapper`. `Callback` аргументът, подаден в `createProcess`, предаден към `processJobWrapper`, е функцията, която изпълнява задачата при обработка. `OnCompleteCallback` аргументът е опционална функция, която се изпълнява след завършване на изпълнението на основната функция на задачата. Ако при обработка се срещне грешка, задачата бива изтрита от опашката (спрямо горезададената конфигурация) и в конзолата се регистрира грешката, която е довела до края на обработката.

3.3.4 Изпращане на имейли

```
@Module({
  imports: [
    MailerModule.forRootAsync({
      useFactory: async (config: ConfigService) => ({
        transport: {
          host: config.get<string>('MAIL_HOST'),
          secure: true,
          auth: {
            user: config.get<string>('MAIL_USER'),
            pass: config.get<string>('MAIL_PASSWORD'),
          },
        },
      },
    ),
    defaults: {
      from: `CRM automated message`,
    },
  },
  template: {
```

```

    dir: join(__dirname, 'templates'),
    adapter: new HandlebarsAdapter(),
    options: {
      strict: true,
    },
  },
  }),
  inject: [ConfigService],
  )),
],
providers: [MailService],
exports: [MailService],
})
export class MailModule {}

```

фиг. 3.14 *модул за имейли*

Помощната функционалност за изпращане на имейли ще бъде използвана за обратна връзка с клиентите (потребителите) на системния клиент при промяна на статуса на акаунта им. В `MailModule` (фиг. 3.14) е реализирана конфигурацията за връзка с имейл клиента, за да може успешно да бъдат изпращани имейли. Използва се библиотеката `'@nestjs-modules/mailer'`. Конфигурирането на хоста е аналогично с модула за базата данни и този за хранилището. Разликата е, че хоста представлява `SMTP` сървър, като например `smtp.gmail.com`. Полето `auth` представлява инициалите, с които се конфигурира профилът, през който да бъдат изпращани имейли по `SMTP` сървъра. `Secure` флага определя дали транспортният протокол ще използва сигурна връзка, обикновено по `SSL/TLS`. `Defaults` полето дефинира опции по подразбиране за имейлите. В случая определя полето `from` на имейлите. `Templates` полето конфигурира шаблоните за имейли : определя в коя директория да бъдат търсени (`dir`) и адаптера, който да бъде използван за тях, в случая `Handlebars`.

```

export class MailDto {
  @IsEmail()

```

```

    receiver: string;

    @IsString()
    title: string;

    @IsOptional()
    @IsString()
    name?: string;

    @IsString()
    client: string;

    @IsOptional()
    @IsString()
    reason?: string;

    @IsOptional()
    @IsString()
    duration?: string;

    @IsEnum(AccountStatus)
    status: AccountStatus;
}

```

фиг. 3.15 *обект за предаване на информация към MailService*

Във фигура 3.15 може да бъде разгледан обектът за предаване на информация към service-ът, който отговаря за изпращането на имейли.

Задължителни са полетата :

- `receiver` - получател на имейла
- `title` - заглавие на имейла
- `client` - име на системния клиент, към който принадлежи потребителя
- `status` - статус на акаунта, тъй като за момента имейлите са предназначени за изпращане само когато е променен статусът

Всички останали полета са маркирани като незадължителни чрез декторатора `IsOptional`. Това са полетата :

- `name` - име на потребителя от системния клиент
- `reason` - причина за промяната на статуса

- `duration` - време, за което този статус е в сила

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Account Status Update Notification</title>
  <style>
    body {
      font-family: Arial, sans-serif;
    }
    .message {
      background-color: #f8f8f8;
      padding: 20px;
      border-radius: 5px;
    }
    .reason {
      margin-top: 20px;
    }
  </style>
</head>
<body>
  <div class="message">
    <p>Dear {{ name }},</p>
    <p><b>Your status in {{ client }} has been updated to </b>{{ status }}</p>
    {{#if reason}}
      <p><b>Description:</b></p>
      <p class="reason">{{ reason }}</p>
    {{/if}}
    {{#if duration}}
      <p><b>Duration:</b> {{ duration }}</p>
    {{/if}}
    <p>If you have any questions or concerns, please contact the
  administrator.</p>
  </div>
</body>
</html>
```

фиг. 3.16 шаблон за съобщение за променен статус на акаунт

Във фигура 3.16 може да бъде разгледан шаблонът за имейл при променен статус на акаунт. Чрез `if` клаузите се проверява дали конкретния параметър е подаден на шаблона и ако е, се добавя тази част, която е оградена от клаузите, към него. В `<style></style>` се намират всички

стилове на отделните части на шаблона, а в `<body></body>` - структурата на шаблона.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Account Deletion Notification</title>
  <style>
    body {
      font-family: Arial, sans-serif;
    }
    .message {
      background-color: #f8f8f8;
      padding: 20px;
      border-radius: 5px;
    }
  </style>
</head>
<body>
  <div class="message">
    <p>Dear {{ name }},</p>
    <p>We regret to inform you that your account in {{ client }} has been
    deleted due to inactivity for the last 30 days.</p>
    <p>If you have any questions or concerns, please contact the
    administrator.</p>
  </div>
</body>
</html>
```

фиг. 3.17 шаблон за съобщение при деактивация на акаунт

Във фигура 3.17 може да бъде разгледан шаблонът за имейл при изтичане на срока за влизане в акаунт след деактивация. Двата шаблона се различават, тъй като деактивацията има различна функционалност от другите статуси. Напълно възможно е под една `if` клауза за статуса в предишния шаблон да бъде оградено съобщението от този, така че да се използва само един, но с оглед на четимостта на кода, ще бъдат разделени в два отделни файла.

```
@Injectable()
export class MailService {
  constructor(private mailerService: MailerService) {}
```

```

    async sendStatusUpdate(mailDto: MailDto) {
      await this.mailerService.sendMail({
        to: mailDto.receiver,
        subject: mailDto.title,
        template: './status',
        context: {
          name: mailDto.name ?? 'user',
          client: mailDto.client,
          reason: mailDto?.reason,
          duration: mailDto?.duration,
          status: mailDto.status
        },
      });
    }

    async sendAccountDeactivationUpdate(mailDto: MailDto) {
      await this.mailerService.sendMail({
        to: mailDto.receiver,
        subject: 'Account Deletion Notification',
        template: './deactivation',
        context: {
          name: mailDto.name ?? 'user',
          client: mailDto.client,
        },
      });
    }
  }
}

```

фиг. 3.18 MailService

Функционалността за изпращане на имейли може да бъде разгледана в класа MailService (фиг. 3.18). Класът използва service-ът от библиотеката '@nestjs-modules/mailer', за изпращане на имейли. Функцията `sendStatusUpdate` отговаря за изпращането на имейли, когато е променен статусът на акаунта. Извиква асинхронната функция на MailerService-а за изпращане на имейл с параметрите от `mailDto`. Във функцията е определено кой шаблон да се използва чрез полето `template`, а в `context` се подават стойностите към шаблона. Операцията `mailDto.name ?? 'user'` проверява дали стойността на полето `name` от `mailDto` е различна от `null` или `undefined` и ако не е, подава символният низ

"user" като име на потребителя. Достъпването на полета от обекта с ?. осигурява, че ако полето не е дефинирано, ще бъде подадена стойност null. Функцията `sendStatusUpdate` работи по аналогичен начин с единствени разлики параметрите, които се подават и шаблонът.

3.4 Основни модули в приложението

3.4.1 Системен клиент и ключ за достъп към приложно-програмния интерфейс

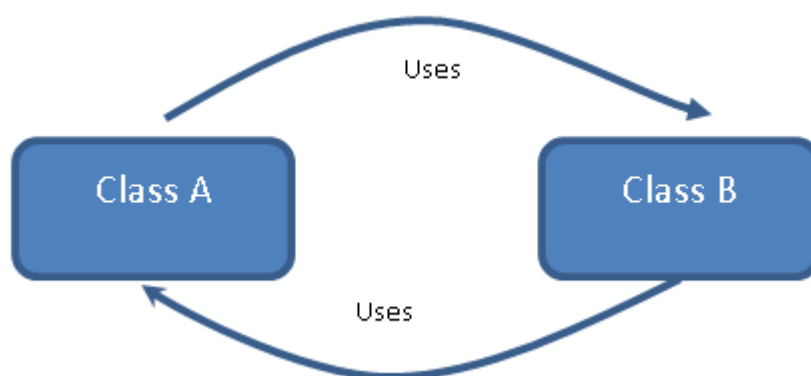
Системните клиенти обособяват отделните инстанции на системата. Всяка една инстанция има обособени оператори (потребители), които имат контрол върху клиентите на инстанцията, роли, с които са снабдени операторите и клиенти, които се явяват потребителите на външното приложение на инстанцията (на системния клиент).

```
@Module({
  imports: [
    ConfigModule,
    forwardRef(() => OperatorModule),
    forwardRef(() => UserModule),
    forwardRef(() => ClientApiKeyModule),
    TypeOrmModule.forFeature([Client, Operator, User, Role, ClientApiKey])
  ],
  controllers: [ClientController],
  providers: [ClientService],
  exports: [ClientService]
})
export class ClientModule {}
```

фиг. 3.19 Client модул

Във фигура 3.19 е разгледан модулът на системния клиент. В него са импортирани модулът за операторите, модулът за потребителските акаунти на операторите и модулът за ключовете за достъп на системния клиент. Тъй като трите изброени модула са взаимно зависими един от друг, тоест

например модулет на клиента импортира модула на оператора, но модулет на оператора също импортира модулет на клиента, се получават кръгови зависимости (фиг. 3.20). За справянето с кръговите зависимости се използва техника, наречена препращане напред или `forwardRef()`. Препращането напред има за цел да позволи на Nest да реферира към класове, които все още не са дефинирани. Така, въпреки че модулет на клиента и модулет на оператора зависят един от друг, чрез препращане напред и двата модула могат да разрешат кръговата зависимост. С оглед на подобие то между модулите, нататък в документацията ще бъдат показани само такива, които имат големи разлики помежду си.



фиг. 3.20 *Кръгова зависимост*

```

@Controller('client')
export class ClientController {
    constructor(private readonly clientService: ClientService) {}

    @RequireSuperuser()
    @Post('/create')
    create(@Body() createClientDto: CreateClientDto) {
        return this.clientService.create(createClientDto);
    }

    @RequireSuperuser()
    @Get('/all')
    findAll() {
        return this.clientService.findAll();
    }

    @RequireSuperuser()
    @Get('/:id')
    findOne(@Param('id') id: string) {
        return this.clientService.findById(+id);
    }

    @RequireSuperuser()
    @Patch('/update/:id')
    update(@Param('id') id: string, @Body() updateClientDto: UpdateClientDto) {
        return this.clientService.update(+id, updateClientDto);
    }

    @RequireSuperuser()
    @Delete('/delete/:id')
    remove(@Param('id') id: string) {
        return this.clientService.remove(+id);
    }
}

```

фиг. 3.21 Контролер на системния клиент

Във фигура 3.21 може да бъде разгледан контролерът за системния клиент. Контролерът разполага с 5 възможни заявки, като всяка една е декорирана с `@RequireSuperuser()`, което означава, че това са заявки, до които имат достъп само админските акаунти. Възможните типове на заявките тук, както и във всички контролери в приложението, са 4:

- **Get** - метод за получаване на ресурс. Тези единствено и само извличат данни.

- `Post` - метод, който изпраща обект към сървъра, който се използва за промяна на състоянието на сървъра или на обект/ресурс от сървъра (базата данни)
- `Patch` - метод, който прилага частични промени върху конкретен обект/ресурс.
- `Delete` - метод, който изтрива посочен обект/ресурс

Всички заявки работят с методи, дефинирани в service-а на системния клиент. Отговорите от заявките представляват резултатите, които съответните методи са върнали.

```
async create(createClientDto: CreateClientDto) {
  try{
    const client = this.clientRepository.create(createClientDto);
    const savedClient = await this.clientRepository.save(client);
    const keyAndToken = await this.apiKeyService.createKey({clientId:
savedClient.id});
    return {api_key: keyAndToken.token, client: savedClient};
  } catch (error) {
    this.logger.error(`Failed to create client: ${error.message}`,
error.stack )
    throw error;
  }
}
```

фиг. 3.22 Функция за създаване на системен клиент

Фигура 3.22 репрезентира функцията от service-а на системния клиент за създаване създаване на нова инстанция. Тази функция се извиква при получаване на `@Post('/create')` заявка в контролера. Цялата функционалност е заключена в `try-catch` блок с цел улавяне на грешки, ако изникнат такива. Чрез `@Body()` `createClientDto: CreateClientDto` данните, изпратени с Post заявката биват запазени в променливата `createClientDto` от тип обект за пренос на данни за създаване на нов системен клиент (фиг 3.23). Чрез изпратения обект за

пренос на данни от Post заявката се създава нова инстанция. След като е създадена, тя бива запазена в базата данни, а идентификационния номер на новата инстанция се препраща към service-а за ключове за достъп, за да бъде генериран и запазен нов ключ за достъп до приложно-програмния интерфейс (фиг. 3.24)

```
import { IsArray, IsOptional, IsString } from "class-validator";
import { Operator } from "src/operator/entities/operator.entity";

export class CreateClientDto {
  @IsString()
  public readonly name: string;

  @IsOptional()
  @IsArray()
  public readonly operators: Partial<Operator>[];
}
```

фиг. 3.23 *Обект за пренос на данни за създаване на нов системен клиент*

```
async createKey(createClientApiKeyDto: CreateClientApiKeyDto) {
  const clientID = createClientApiKeyDto.clientID;
  const keys = await this.constructKeys(clientID);

  const apiKey = this.apiKeyRepository.create({
    client: {id: clientID},
    token: keys.hashToken
  });

  const key = await this.apiKeyRepository.save(apiKey);

  return {key, token: keys.token};
}

async constructKeys(clientID: number){
  const token = this.authService.constructApiKey(clientID);
  const hashedToken = await bcrypt.hash(token, 10);
  return {token, hashedToken};
}
```

фиг. 3.24 *Функции за генерация и запазване на ключ за достъп до приложно-програмния интерфейс*

Аналогично на функцията за създаване на системен клиент функцията `createKey` е предназначена за създаването на ключ за достъп до приложно-програмния интерфейс. Чрез подадения идентификационен номер в обекта за пренос на данни (фиг. 3.25), се създава ключа за достъп във функцията `constructKeys`. Този ключ бива генериран от функцията на service-а за автентикация `constructApiKey` (фиг. 3.86). След генерация, ключът е хеширан чрез библиотеката `bcrypt` с оглед на сигурността на данните в базата. `Bcrypt` използва шифърният алгоритъм `Blowfish`. След хеширане “суровият” и хешираният вариант на ключа са върнати към `createKey` и се създава и запазва в базата данни нова инстанция на ключ, обвързан със системния клиент, с токен хешираният ключ. Поради обвързаността на системния клиент и ключа за достъп до приложно-програмния интерфейс чрез `OneToOne` релацията, не е нужно да се правят промени по записа на системния клиент в базата - `TypeORM` сам ще оправи зависимостите. След успешното създаване на ключа за достъп биват върнати още един път “суровият” и хешираният вариант на ключа, като “суровият” е върнат заедно с новият системен клиент като отговор на заявката за създаване.

```
export class CreateClientApiKeyDto {
    @IsNumber()
    public readonly clientId: number;
}
```

фиг. 3.25 *Обект за пренос на данни за създаване на ключ за достъп до приложно-програмния интерфейс*

```
async findAll() {
    return this.clientRepository
        .createQueryBuilder('client')
        .leftJoinAndSelect('client.operators', 'operators')
        .leftJoinAndSelect('operators.roles', 'roles')
        .leftJoinAndSelect('operators.user', 'user')
        .leftJoinAndSelect('client.api_key', 'api_key')
        .getMany();
}
```

```

}

async findByName(name: string){
  return this.clientRepository
    .createQueryBuilder('client')
    .leftJoinAndSelect('client.operators', 'operators')
    .leftJoinAndSelect('operators.roles', 'roles')
    .leftJoinAndSelect('operators.user', 'user')
    .leftJoinAndSelect('client.api_key', 'api_key')
    .where('client.name = :name', {name})
    .getOneOrFail();
}

async findById(id: number) {
  return this.clientRepository
    .createQueryBuilder('client')
    .leftJoinAndSelect('client.operators', 'operators')
    .leftJoinAndSelect('client.roles', 'room_roles')
    .leftJoinAndSelect('operators.roles', 'roles')
    .leftJoinAndSelect('operators.user', 'user')
    .leftJoinAndSelect('client.customers', 'room_customers')
    .leftJoinAndSelect('client.api_key', 'api_key')
    .where('client.id = :id', {id})
    .getOneOrFail();
}

```

фиг. 3.26 Функция за вземане на всички инстанции от базата ; функция за намиране на инстанция по име ; функция за намиране на инстанция по идентификационен номер

Фигура 3.26 репрезентира функциите `findById` и `findAll`, които се извикват при получаване на `@Get('/all')` и `@Get('/:id')`. `CreateQueryBuilder` инициализира заявката, `LeftJoinAndSelect` взима релациите за оператори, роли, потребители на операторите и ключа за достъп, след което се определя критерии за търсене чрез `where` (във `findById` съответно идентификационния номер, а във `findById` - името), ако трябва такъв, след което се извиква функцията за екзекуция на заявката. `GetOneOrFail` осигурява, че ако заявката към базата не успее да намери клиент по критерия, то ще бъде върната грешка.

`GetMany` връща всички записи в базата, които отговарят на заявката. В случая това са абсолютно всички записи, тъй като няма критерий, по който се търси. `':id'` в `@Get(':id')` представлява параметър, който се добавя към заявката чрез този синтаксис. `@Param('id') id: string` взима параметъра `id` от заявката и го записва в променлива, в случая със същото име като него.

```
async update(id: number, updateClientDto: UpdateClientDto) {
  const client = await this.findById(id);
  const operators = updateClientDto.operators;
  if(client){
    if(operators && operators?.length){
      this.addOperators(client, operators);
    }
  }else{
    this.logger.error(`Client with an id of ${id} not found`);
    throw new NotFoundException('Client not found');
  }
}
```

фиг. 3.27 Функция за ъпдейт на инстанция

Функцията за ъпдейт на инстанция (фиг. 3.27) се извиква при получаване на `@Patch('/update/:id')`. Чрез идентификационния номер, подаден като аргумент на функцията и параметър на заявката се търси инстанция на системен клиент. Ако тя не е намерена е хвърлена грешка. В противен случай от `updateClientDto` (фиг 3.28), взето от тялото на заявката, се взима полето за масива от оператори и ако масивът е валиден и има поне един оператор в него, се извиква функцията `addOperators` на `service-a` за добавяне на операторите към инстанцията и промяната ѝ.

```
export class UpdateClientDto extends PartialType(CreateClientDto) {
  @IsOptional()
  @IsString()
  name?: string;
```



```

@IsOptional()
@IsArray()
operators?: Operator[] | undefined;
}

```

фиг. 3.28 обект за пренос на данни за промяна системен клиент

```

async addOperators(client: Partial<Client>, operators:
Partial<Operator>[]){

    for (const operator of operators){
        try{
            const {client : operatorClient, ...clientOperator} = await
this.operatorService.assignClient(operator.id as number, client);
            if(!client.operators?.includes(operator as Operator)){
                client.operators?.push(clientOperator)
            }
        }catch(error){
            this.logger.error(`Error occurred adding operator ${operator.id} in
client ${client.id} : ${error.message}`, error.stack);
            return false;
        }
    }
    return this.clientRepository.save(client);
}

```

фиг. 3.29 добавяне на оператори към системен клиент

Функцията за добавяне на оператори към системния клиент (фиг. 3.29) приема като аргумент **непълна** (означена с *Partial*; т.е. непълен тип) инстанция на системния клиент и масив от **непълни** инстанции на оператори. Тя минава през масива от оператори и за всеки от тях вика функцията на `OperatorService` за обвързване на системен клиент с оператор `assignClient` (фиг. 3.44). Ако операторът не се съдържа в масива от оператори на системния клиент, той бива добавен, като се изключи неговият системен клиент, за да не стават кръгови референции. Ако се случи грешка в някоя от операциите, то тя е хваната и регистрирана в конзолата.

```

async addRole(clientID: number, role: Role){
    const {operators, client: roleClient,...sanitizedRole} = role;
    const client = await this.findById(clientID);

    if(roleClient && roleClient.id !== clientID){
        this.logger.error(`Error occurred adding role ${role.id} in client
        ${client.id}`,
        'role belongs to another client');
        throw new Error('Role belongs to another client');
    }

    if(client){

        if(!client.roles?.find(role => role.name === sanitizedRole.name)){
            client.roles?.push(sanitizedRole);
            return this.clientRepository.save(client);
        }else{
            this.logger.error(`Error occurred adding role ${role.id} in client
            ${client.id}`,
            'role already exists within this client');
            throw new Error('Role already exists within this client')
        }

    }else{
        this.logger.error(`Error occurred finding client ${clientID}`);
        throw new NotFoundException('Client not found')
    }
}

```

фиг. 3.30 добавяне на роля към системен клиент

Функцията за добавяне на роля към системния клиент (фиг. 3.30) работи на подобен принцип като тази за добавяне на оператори. Разликата е, че тази функция се извиква от service-а за роли и заради това приема роля като аргумент.

```

remove(id: number) {
    return this.clientRepository.delete({id});
}

```

фиг. 3.31 премахване на системен клиент по идентификационен номер

Фигура 3.31 репрезентира функцията `remove`, която се извиква при получаване на `@Delete('/:id')`. Ако подадения идентификационен номер не сочи към нито един системен клиент, то няма да се случи нищо. В противен случай системният клиент ще бъде изтрит, заедно с ключа, благодарение на `OneToOne` релацията.

```
@Controller('client-api')
export class ClientApiKeyController {
  constructor(private readonly clientApiKeyService: ClientApiKeyService) {}

  @RequireSuperuser()
  @Get('refresh/:id')
  refreshKey(@Param('id') id: number) {
    return this.clientApiKeyService.refreshKey({clientId: id} as
UpdateClientApiKeyDto);
  }

  @RequireSuperuser()
  @Delete('delete/:id')
  deleteKey(@Param('id') id: number) {
    return this.clientApiKeyService.deleteKey(id);
  }
}
```

фиг. 3.32 контролер за ключове за достъп до приложно-програмния интерфейс

Във фигура 3.32 е изложен кодът за контролера за ключовете за достъп до приложно програмния интерфейс. Заявките на този контролер могат да достъпени единствено и само от админски акаунти.

`@Get('refresh/:id')` е заявка, която се използва за опресняване на ключа за достъп, тъй като те имат едномесечна валидност.

`@Delete('delete/:id')` се използва за изтриването на ключа за достъп.

```
async refreshKey(updateClientDto: UpdateClientApiKeyDto){
  const clientId = updateClientDto.clientID;
  try{

    const apiKey = await this.apiKeyRepository.findOneByOrFail({
      client: {id: updateClientDto.clientID}
```

```

    });

    const keys = await this.constructKeys(clientID);
    apiKey.token = keys.hashedToken;

    const key = await this.apiKeyRepository.save(apiKey);
    this.clientService.updateApiKey(clientID, key);
    return {token: keys.token};

  }catch(error){
    return this.createKey(updateClientDto);
  }
}

async deleteKey(clientID: number){
  return this.apiKeyRepository.delete({client: {id: clientID}})
}

```

фиг. 3.33 функция за опресняване на ключ ; функция за изтриване на ключ

`RefreshKey` (фиг. 3.33 ; ф-ция 1) се използва от заявката за опресняване на ключа за достъп. Тя опитва да намери ключ за достъп по идентификационния номер на системния клиент, подаден като параметър на заявката и предаден към функцията в обект за пренос на данни за промяна на ключа за достъп (аналогичен на този от фиг. 3.25). Ако успее, извиква функцията за създаване на ключове (фиг. 3.24), задава новият хеширан ключ, запазва променения обект и извиква функцията от service-a на системния клиент за промяна на ключа за достъп (фиг. 3.34).

```

async updateApiKey(id: number, apiKey: ClientApiKey){
  const client = await this.findById(id);
  if(client){
    if(!client.api_key ||
      (client.api_key && client.api_key.id == apiKey.id)){
      client.api_key = apiKey;
      await this.clientRepository.save(client);
    }else{
      this.logger.error(`API Key ${apiKey.id} does not belong to client
        ${client.id}`)
      return `API Key ${apiKey.id} does not belong to client ${client.id}`
    }
  }
}

```

```

    }else{
        throw new NotFoundException('Client not found');
    }
}

```

фиг. 3.34 функция за промяна на ключ за достъп

Функцията работи аналогично на тази за добавяне на роля (фиг. 3.30) с разликата, че сравнява дали системният клиент има ключ и дали идентификационният номер на този ключ отговаря на идентификационния номер на подадения.

`DeleteKey` (фиг. 3.33 ; ф-ция 2) се използва от заявката за изтриване на ключа за достъп. Функцията работи по аналогичен начин като тази в системния клиент, с разликата, че когато ключът бъде изтрит системният клиент ще продължи да съществува, защото от страната на системния клиент `OneToOne` релацията няма `cascade` опция при изтриване.

3.4.2 Потребител и оператор

Операторите представляват потребителите в системния клиент, които има управление над неговите клиенти. Операторите са обвързани с потребителски акаунти в системата.

```

@Controller('operator')
export class OperatorController {
    constructor(private readonly operatorService: OperatorService) {}

    @checkAbilities({ action: SubjectActions.CREATE, subject: 'operator' })
    @Post()
    create(@Body() createOperatorDto: CreateOperatorDto, @Req() request:
    UserRequest) {
        return this.operatorService.createOperator(createOperatorDto,
        request.user);
    }

    @checkAbilities({ action: SubjectActions.READ, subject: 'operator' })
    @Get('/all')
    findAll(@Req() request: UserRequest) {
        return this.operatorService.findAllOperators(request.user);
    }
}

```

```

}

@RequireSuperuser()
@Get('/all/:client_id')
findAllInClient(@Param('client_id') clientID: string){
    return this.operatorService.findAllInClient(+clientID);
}

@checkAbilites({ action: SubjectActions.READ, subject: 'operator' })
@Get('/:id')
findOne(@Param('id') id: string, @Req() request: UserRequest) {
    return this.operatorService.findOneOperator(+id, request.user);
}

@checkAbilites({ action: SubjectActions.UPDATE, subject: 'operator' })
@Patch('update/:id')
update(@Param('id') id: string, @Body() updateOperatorDto:
UpdateOperatorDto, @Req() request: UserRequest) {
    return this.operatorService.update(+id, updateOperatorDto,
request.user);
}

@checkAbilites({ action: SubjectActions.DELETE, subject: 'operator'})
@Delete('delete/:id')
remove(@Param('id') id: string, @Req() request: UserRequest) {
    return this.operatorService.removeOperator(+id, request);
}
}

```

фиг. 3.35 контролер за оператори

Крайните точки на контролера за операторите (фиг. 3.35) са почти напълно аналогични с тези на контролера за системните клиенти. Различават се декораторите на поставени над всеки един от контролерите. `@checkAbilites({ action: <action>, subject: <subject>})` е декоратор, чрез който се индикират нужните права на оператора, за да може успешно да бъде направена заявка на конкретната крайна точка. Проверката на правата е обяснена в част 3.5. Възможните операции, които могат да се извършват към различните обекти могат да бъдат разгледани на фигура 3.6 от част 3.2.2. Ако операторът не е админ, при липса на нужните права за достъпване на заявката, тя няма да мине.

```
createOperator(createOperatorDto: CreateOperatorDto, user:any) {
    return user.is_admin ? this.createOperatorGlobally(createOperatorDto) :
    this.createOperatorInInstance(createOperatorDto, user.client_id);
}
```

фиг. 3.36 функция за създаване на оператор

Фигура 3.36 показва функцията за създаване на оператор, извикана при заявка `@Post()`. При по-обстойно разглеждане на заявките, се забелязва, че почти всяка от тях има във функцията си `@Req()` `request: UserRequest`. `@Req` декоратора се използва за създаване на параметър, съдържащ заявката, изпратена от клиента. Типът използван за заявката (фиг. 3.7) не е по подразбиране, а е направен, за да отговаря на изискванията на приложението. Полето `user` на заявката се задава по време на автентикацията и оторизацията на потребителя преди пристъпването към нея. Функцията `createOperator` приема това поле. В нея се проверява дали потребителят има атрибута `is_admin` и ако го има се пристъпва към функцията `createOperatorGlobally` (фиг. 3.37), която създава оператор без системен клиент или с клиент зададен в `createOperatorDto` (фиг. 3.39). В противен случай се пристъпва към функцията `createOperatorInInstance` (фиг. 3.38), която създава оператор в инстанцията на потребителя (оператора), който е извикал заявката. Инстанцията се намира чрез атрибута `client_id`, който е присъщ само за операторите.

```
createOperatorGlobally(createOperatorDto: CreateOperatorDto){
    const operator = this.operatorRepository.create(createOperatorDto);
    return this.operatorRepository.save(operator);
}
```

фиг. 3.37 функция за създаване на оператор без системен клиент или с клиент зададен в createOperatorDto

```
createOperatorInInstance(createOperatorDto: CreateOperatorDto, clientID:
```

```

number){
  const {client, ...sanitizedDto} = createOperatorDto;
  const operator = this.operatorRepository.create({
    client: {
      id: clientID
    }, ...sanitizedDto})
  return this.operatorRepository.save(operator);
}

```

фиг. 3.38 функция за създаване на оператор в инстанцията на потребителя (оператора), който е извикал заявката

В `createOperatorInInstance` се отделя клиента `createOperatorDto`, след което се създава оператор останалите параметри в обекта за пренос на данни и съответния идентификационен номер на инстанцията.

```

export class CreateOperatorDto {
  @IsOptional()
  public readonly client: Client;
  @IsOptional()
  public readonly user: User;

  @IsOptional()
  @IsArray()
  public readonly roles: Role[];
}

```

фиг. 3.39 обект за пренос на данни за създаване на оператор

Функциите за вземане на оператор, асоциирани със заявките `@Get('/:id')`, `@Get('/all')` и `@Get('/all/:client_id')` са почти аналогични на тези в модула на системния клиент.

```

async findAllInClient(clientID: number){
  return this.operatorRepository.find({where: {
    client : {
      id: clientID
    }
  }, relations: ['roles', 'client']})
}

async findOne(id: number) {
  return this.operatorRepository
    .createQueryBuilder('operator')

```



```

        .leftJoinAndSelect('operator.user', 'user')
        .leftJoinAndSelect('operator.client', 'client')
        .leftJoinAndSelect('operator.roles', 'roles')
        .where('operator.id = :id', { id })
        .getOneOrFail();
    }

    async findOneInClient(id: number, clientID: number){
        return this.operatorRepository
            .createQueryBuilder('operator')
            .leftJoinAndSelect('operator.user', 'user')
            .leftJoinAndSelect('operator.client', 'client')
            .leftJoinAndSelect('operator.roles', 'roles')
            .where('operator.id = :id', { id })
            .andWhere('operator.client.id = :clientID', { clientID })
            .getOneOrFail();
    }

    async findOneOperator(id: number, user: any){
        return user.is_admin ? this.findOne(id) : this.findOneInClient(id,
            user.client_id);
    }

```

фиг. 3.40 функции за вземане на един или много оператори

Тънката разлика е, че функцията за вземане на всички оператори и тази за вземане на един оператор, прилагат същата практика като функцията за създаване. `@Get('/:client_id')` е заявка само за админски акаунти, тъй като е предназначена за вземане на всички оператори от конкретна инстанция, която и да е тя, докато нормалните оператори са ограничени до вземане на всички оператори от тяхната инстанция.

```

async update(id: number, updateOperatorDto: UpdateOperatorDto, user: any) {
    const operatorUser = updateOperatorDto.user;
    const role = updateOperatorDto.role;

    try{
        if(user.is_admin){
            if(operatorUser){
                try{
                    this.userService.update(user.id, {operator: await
this.findOne(id)}, user)
                }catch(error){
                    console.error(error);
                }
            }
        }
    }
}

```

```

    }
  }
}
if(role){
  await this.addRole(id, role, user)
}
}catch(error){
  this.logger.error(`An error occurred updating operator ${id}: `,
error.message, error.stack);
  return 'An error occurred updating operator';
}
}

```

фиг. 3.41 функция за промяна на оператор

Функцията за ъпдейт на оператор (фиг. 3.41) се извиква при получаване на `@Patch('update/:id')`. Ако потребителят е админ, функцията може да бъде използвана за обвързване на потребителски акаунт с оператор. Това се случва от страна на service-а за потребителските акаунти, чрез извикване на функцията `update` (фиг. 3.53). Функцията може да бъде използвана и за добавяне на роля към оператора, което извиква `addRole` метода (фиг. 3.42).

```

async addRole(id: number, role: Role, user: any){

  const operator = await this.findOneOperator(id, user);
  const {operators, client: roleClient, ...sanitizedRole} = role;

  if(operator){
    if(!operator.roles?.find(role => role.name === sanitizedRole.name)){
      if(roleClient?.name === operator.client?.name && sanitizedRole.name
!= SUPERUSER){
        operator.roles?.push(sanitizedRole);
        return this.operatorRepository.save(operator);
      }
    }else{
      this.logger.error(`Error occurred adding role ${role.id} to
operator ${operator.id} : `,
        'Role client does not match operator client')
      throw new Error('Role client does not match operator client')
    }
  }else{
    this.logger.error(`Error occurred adding role ${role.id} to operator
${operator.id} : `,

```

```

        'Role already exists within this operator')
        throw new Error('Role already exists within this operator')
    }
    }else{
        this.logger.error(`Cannot find operator`);
        throw new NotFoundException('Operator not found')
    }
}

```

фиг. 3.42 *функция за добавяне на роля към оператор*

Функцията за добавяне на роля към оператор е аналогична с тази за добавяне роля към системен клиент. Разликата е че тук се проверява дали добавената роля е за админ, тъй като никой няма право да добавя тази роля, освен разработчиците. Функциите за обвързване на оператор със системен клиент (фиг. 3.43) и обвързване на оператор с потребителски акаунт (фиг 3.44) са аналогични.

```

async assignClient(id: number, client: Partial<Client>){
    const operator = await this.findOne(id);
    if(operator){
        if(operator.client){
            this.logger.error(`Error occurred adding client ${client.id} to operator
${operator.id} : `,
                'Cannot assign client to operator already associated with another client')
            throw new Error('Cannot assign client to operator already associated with
another client');
        }
        const {operators: clientOperators, ...sanitizedClient} = client;
        if(clientOperators?.includes(operator)){
            this.logger.error(`Error occurred adding client ${client.id} to operator
${operator.id} : `,
                'Operator already associated with client')
            throw new Error('Operator already associated with client');
        }
        operator.client = sanitizedClient as Client;
        return this.operatorRepository.save(operator)
    }else{
        this.logger.error(`Cannot find operator`);
        throw new NotFoundException('Operator not found');
    }
}

```

фиг. 3.43 *функция за обвързване на оператор със системен клиент*

```

async assignUser(id:number, user: Partial<User>){
    const operator = await this.findOne(id);
    if(operator){
        if(operator.user && operator.user.id !== user.id){
            this.logger.error(`Error occurred adding user ${user.id} in operator
${operator.id} : `,
                'Cannot assign user to operator already associated with another
user')
            throw new Error('Cannot assign user to operator already associated
with another user');
        }
        const {operator: userOperator, email, password, ...sanitizedUser} =
user;
        if(userOperator && userOperator.id !== operator.id){
            this.logger.error(`Error occurred adding user ${user.id} in operator
${operator.id} : `,
                'Cannot assign operator to user already associated with another
operator')
            throw new Error('Cannot assign operator to user already associated
with another operator');
        }
        operator.user = sanitizedUser as User;
        return this.operatorRepository.save(operator)
    }else{
        this.logger.error(`Cannot find operator`);
        throw new NotFoundException('Operator not found');
    }
}
}

```

фиг. 3.44 *функция за обвързване на оператор със потребителски акаунт*

```

removeByClientAndId(id: number, clientID: number){
    return this.operatorRepository.delete({
        id,
        client : {
            id: clientID
        }
    })
}

removeById(id: number) {
    return this.operatorRepository.delete(id);
}

removeOperator(id: number, user:any) {
    return user?.is_admin ? this.removeById(id) : this.removeByClientAndId(id,
user.client_id);
}

```

фиг. 3.45 функция за премахване на оператор

Функцията за премахване на оператор (фиг. 3.45 ; `removeOperator`) се извиква при получаване на `@Delete('delete/:id')`. Начинът и на работа е аналогичен с другите функции, които използват полетата на потребителя от заявката. При изтриване на оператор, бива изтрит и потребителският акаунт, обвързан с него.

```
@Controller('user')
export class UserController {
    constructor(private readonly userService: UserService) {}

    @checkAbilities({action: SubjectActions.CREATE, subject: 'user'})
    @Post('/create')
    create(@Body() createUserDto: CreateUserDto, @Req() request: UserRequest)
    {
        return this.userService.create(createUserDto, request.user);
    }

    @checkAbilities({action: SubjectActions.READ, subject: 'user'})
    @Get('/:id')
    findOne(@Param('id') id: string, @Req() request: UserRequest) {
        return this.userService.findUserById(+id, request.user);
    }

    @checkAbilities({action: SubjectActions.READ, subject: 'user'})
    @Get()
    findByEmail(@Query('email') email: string, @Req() request: UserRequest){
        return this.userService.findUserByEmail(email, request.user);
    }

    @checkAbilities({action: SubjectActions.READ, subject: 'user'})
    @Get('/all')
    findAll(@Req() request: UserRequest) {
        return this.userService.findUsers(request.user);
    }

    @RequireSuperuser()
    @Get('/all/:client_id')
    findAllInClient(@Param('client_id') clientID: string) {
        return this.userService.findAllInClient(+clientID)
    }
}
```

```

    @checkAbilities({action: SubjectActions.UPDATE, subject: 'user'})
    @Patch('/update/:id')
    async update(@Param('id') id: string, @Body() updateUserDto:
UpdateUserDto, @Req() request: UserRequest) {
        return await this.userService.update(+id, updateUserDto, request.user);
    }

    @checkAbilities({action: SubjectActions.DELETE, subject: 'user'})
    @Delete('/delete/:id')
    remove(@Param('id') id: string, @Req() request: UserRequest) {
        return this.userService.removeUser(+id, request.user);
    }
}

```

фиг. 3.46 контролер за потребителски акаунта

Във фигура 3.46 може да бъде разгледан контролерът потребителски акаунти. Функцията, за създаване на потребителски акаунт (фиг. 3.47), която бива извикана при заявка `@Post('/create')`, е аналогична по логика с функциите за създаване на оператор.

```

    async create(createUserDto: CreateUserDto, u:any) {
        const {password: rawPassword, ...userDto} = createUserDto;
        const hashedPassword = await bcrypt.hash(rawPassword, 10);
        try{
            return u.is_admin ? this.createWithOperator({password: hashedPassword,
...userDto}) :
            this.createUserAndOperator({password: hashedPassword, ...userDto},
u.client_id)
        }catch(error){
            Logger.error(error);
            return error.message;
        }
    }
}

```

фиг. 3.47 функция за създаване на потребителски акаунт

Паролата, зададена в `createUserDto` (фиг. 3.48), е хеширана чрез `bcrypt`. От там ако потребителят, изпратил заявката е админ се създава акаунт с оператора, подаден в `createUserDto` (ако е подаден такъв) чрез `createWithOperator` (фиг. 3.49), а ако не е - се създава акаунт

заедно с оператор, обвързан с него в конкретната инстанция на системата чрез `createUserAndOperator` (фиг. 3.50).

```
export class CreateUserDto {
  @IsString()
  public readonly username: string;
  @IsEmail()
  public readonly email: string;
  @IsString()
  public readonly password: string;
  @IsOptional()
  public readonly operator: Partial<Operator>
}
```

фиг. 3.48 *обект за пренос на данни за създаване на потребителски акаунт*

```
async createWithOperator(createUserDto: CreateUserDto){
  const user = this.userRepository.create(createUserDto);
  return this.userRepository.save(createUserDto);
}
```

фиг. 3.49 *функция за създаване на потребителски акаунт с всички данни от `createUserDto`*

```
async createUserAndOperator(createUserDto: CreateUserDto, clientID:
number){
  const {operator, ...sanitizedDto} = createUserDto
  const user = this.userRepository.create(sanitizedDto);
  const savedUser = await this.userRepository.save(user);
  this.operatorService.createOperator({user: savedUser} as
CreateOperatorDto, {client_id : clientID});
  return savedUser;
}
```

фиг. 3.50 *функция за създаване на потребителски акаунт, заедно с оператор, обвързан с него*

Функциите, извиквани от `@Get()` заявките са идентични като имплементация с тези при service-а за операторите. Заявката за откриване на потребител по имейл взима query параметър чрез `@Query('email')`. Query параметрите изглеждат по следния начин :

```
<request>/?parameter=<value>
```

Фигура 3.51 показва функциите за вземане на един или много потребителски акаунти.

```
async findByEmail(email: string){
    return this.userRepository
        .createQueryBuilder('user')
        .leftJoinAndSelect('user.operator', 'operator')
        .where('user.email = :email', { email })
        .getOneOrFail();
}

findByEmailAndClient(email: string, clientID: number){
    return this.userRepository
        .createQueryBuilder('user')
        .leftJoinAndSelect('user.operator', 'operator')
        .where('user.email = :email', { email })
        .andWhere('operator.client.id = :clientID', { clientID })
        .getOneOrFail();
}

async findById(id: number) {
    return this.userRepository
        .createQueryBuilder('user')
        .leftJoinAndSelect('user.operator', 'operator')
        .where('user.id = :id', { id })
        .getOneOrFail();
}

async findByIdAndClient(id: number, clientID: number){
    return this.userRepository
        .createQueryBuilder('user')
        .leftJoinAndSelect('user.operator', 'operator')
        .where('user.id = :id', { id })
        .andWhere('operator.client.id = :clientID', { clientID })
        .getOneOrFail();
}

async findUserById(id: number, user: any){
    return user?.is_admin ? this.findById(id) : this.findByIdAndClient(id,
user.client_id)
}

async findUserByEmail(email: string, user: any){
    return user?.is_admin ? this.findByEmail(email) :
this.findByEmailAndClient(email, user.client_id)
}
```

фиг. 3.51 вземане на един потребителски акаунт


```

findAllInClient(clientID: number) {
    return this.userRepository
        .createQueryBuilder('user')
        .leftJoinAndSelect('user.operator', 'operator')
        .andWhere('operator.client.id = :clientID', { clientID })
        .getMany();
}

findAll() {
    return this.userRepository
        .createQueryBuilder('user')
        .leftJoinAndSelect('user.operator', 'operator')
        .getMany();
}

async findUsers(user: any){
    return user?.is_admin ? this.findAll() :
    this.findAllInClient(user.client_id);
}

```

фиг. 3.52 вземане на много потребителски акаунти

```

async update(id: number, updateUserDto: UpdateUserDto, u: any) {
    const user = await this.findById(id);
    const operator = updateUserDto.operator;
    if(user){
        if(operator &&
            (u.is_admin || (operator.client
                && u.client_id == operator.client.id) )){
            const userOperator = await this.assignOperator(user, operator);
            user.operator = userOperator;
        }
        if(updateUserDto.refreshToken || updateUserDto.refreshToken == null)
            user.refresh_token = updateUserDto.refreshToken as string;

        return this.userRepository.save(user);
    }else{
        this.logger.error(`User ${id} not found`)
        throw new NotFoundException('User not found');
    }
}

```

фиг. 3.53 функция за промяна на на потребителски акаунт

Във фигура 3.53 е изложена функцията за промяна на потребителски акаунт, която се извиква при получаване на `@Patch('update/:id')`. Тя е

логически близка с функцията за промяна на оператор. Атрибутът `refreshToken` от `updateUserDto` (фиг. 3.55) се използва за опресняване на JWT токена за достъп. Това ще бъде разгледано в част 3.5. Потребителският акаунт може да бъде обвързан с произволен оператор единствено и само при заявка от админски акаунт. Това се случва посредством функцията `assignOperator` (фиг. 3.54).

```
async assignOperator(user:User, operator:Partial<Operator>){
    try{
        const userOperator = await
this.operatorService.assignUser(operator?.id as number, user);
        return userOperator;
    }catch(error){
        this.logger.error(`An error occurred assigning user ${user.id} to
operator ${operator.id}: ${error.message}`, error.stack)
        throw new Error(error.message);
    }
}
```

фиг. 3.54 функция за обвързване на потребителски акаунт с оператор

```
export class UpdateUserDto extends PartialType(CreateUserDto) {

    @IsEmail()
    @IsOptional()
    public readonly email?: string;

    @IsString()
    @IsOptional()
    public readonly password?: string;

    @IsOptional()
    public readonly operator?: Partial<Operator>;

    @IsOptional()
    public readonly refreshToken?: string | null;
}
```

фиг. 3.55 обект за пренос на информация за промяна на потребителски акаунт

Функцията използва метода на `OperatorService assignUser` (фиг. 3.44) и връща операторът от него. Този оператор се задава като оператор на потребителския акаунт.

```
removeUser(id:number, user: any){
    return user?.is_admin ? this.removeById(id) :
    this.removeByClientAndId(id, user.client_id);
}

removeByClientAndId(id: number, clientID: number){
    return this.userRepository.delete({
        id,
        operator : {
            client: {
                id: clientID
            },
            user:{
                id
            }
        }
    })
}

removeById(id: number) {
    return this.userRepository.delete(id);
}
```

фиг. 3.56 функции за изтриване потребителски акаунт

Логиката зад функцията за изтриване на потребителски акаунт (фиг. 3.56), извиквана от `@Delete('/delete/:id')`, е аналогична с функцията за изтриване на оператор. При изтриване на потребителския акаунт операторът **ще бъде запазен**.

3.4.3 Права

Правата, с които разполагат ролите, могат да бъдат контролирани само от админските акаунти. Причината за това е, че правата, които съществуват, са определен брой. Фигура 3.57 разглежда контролера за права.

```

@Controller('permissions')
export class PermissionsController {
    constructor(private readonly permissionsService: PermissionsService) {}

    @RequireSuperuser()
    @Post()
    create(@Body() createPermissionDto: CreatePermissionDto) {
        return this.permissionsService.create(createPermissionDto);
    }

    @RequireSuperuser()
    @Get('/all')
    findAll() {
        return this.permissionsService.findAll();
    }

    @RequireSuperuser()
    @Get('/:id')
    findOne(@Param('id') id: string) {
        return this.permissionsService.findOne(+id);
    }

    @RequireSuperuser()
    @Delete('/:id')
    remove(@Param('id') id: string) {
        return this.permissionsService.remove(+id);
    }
}

```

фиг. 3.57 контролер за права

```

async create(createPermissionDto: CreatePermissionDto) {
    const subjectExists = await
this.findSubject(createPermissionDto.subject);
    if(subjectExists){
        try{
            await this.findOneWithAttributes(createPermissionDto.subject,
createPermissionDto.action, createPermissionDto.conditions)
        }catch(error){
            const permission =
this.permissionRepository.create(createPermissionDto);
            return this.permissionRepository.save(permission);
        }
        this.logger.error(`Permission with subject
${createPermissionDto.subject} and action ${createPermissionDto.action}
already exists`)
    }
}

```

```

        throw new Error('Permission already exists');
    }else{
        this.logger.error(`Permission subject ${createPermissionDto.subject}
not found`)
        throw new Error('Permission subject not found ')
    }
}

```

фиг. 3.58 създаване на право

Функцията за създаване на право (фиг. 3.58), извикана от `@Post()`, Използва функцията `findSubject` (фиг. 3.59), за да провери дали обектът за право е легитимна таблица от базата данни. Макар и да представлява “сурова” SQL заявка, има много малък риск за инжекция, тъй като името на таблицата, която ще бъде търсена е параметризирано. Ако таблицата е легитимна търси право със същите полета като тези от `createPermissionDto` (фиг. 3.60). Ако не намери такава, създава правото успешно. Причината за търсенето е, защото атрибутите на правото не могат да бъдат уникални, а например ще съществуват няколко права с еднакви `action` атрибути, но различни `subject` атрибути, няколко с еднакви `subject` атрибути, но различни `action` атрибути и така нататък.

```

async findSubject(subject: string){
    const table = await this.permissionRepository.manager.query(
        `SELECT EXISTS (
            SELECT 1
            FROM information_schema.tables
            WHERE table_name = $1
        )`,
        [subject.toLowerCase()]
    );
    return table[0].exists;
}

```

фиг. 3.59 откриване на таблица в базата данни

```

export class CreatePermissionDto {
    @IsString()
    action: string;

    @IsString()

```

```

    subject: string;

    @IsString()
    conditions: string;

    @IsOptional()
    roles?: Role[];
}

```

фиг. 3.60 *обект за пренос на данни за създаване на право*

Макар и да представлява “сурова” SQL заявка, има много малък риск за инжекция, тъй като името на таблицата, която ще бъде търсена е параметризирано. Функциите за вземане на едно или много права, за изтриване на право и за обвързване на роля с право са аналогични на тези за системния клиент (фиг. 3.61).

```

async findOneWithAttributes(subject: string, action: string, conditions?:
string){
    try{
        const permission = await this.permissionRepository.findOneByOrFail({
subject, action, conditions});
        return permission;
    }catch(error){
        throw error;
    }
}

findAll() {
    return this.permissionRepository.findAndCount();
}

async findOne(id: number) {
    return this.permissionRepository
        .createQueryBuilder('permission')
        .leftJoinAndSelect('permission.roles', 'roles')
        .where('permission.id = :id', { id })
        .getOneOrFail();
}

remove(id: number) {
    return this.permissionRepository.delete({id});
}

```

фиг. 3.61 *Функции за вземане на едно или много права, за изтриване на право, за обвързване на роля с право*

Функцията за добавяне на роля към право (фиг. 3.62) е аналогична на тази в service-а на операторите.

```
async addRole(id: number, role: Role){
  const permission = await this.findOne(id);
  const {permissions, ...sanitizedRole} = role;

  if(permission){
    if(!permission.roles?.find(role => role.id == sanitizedRole.id)){
      permission.roles?.push(sanitizedRole);
      return this.permissionRepository.save(permission);
    }else{
      this.logger.error(`Role ${sanitizedRole.id} already has permission
${id}`)
      throw new Error('Role already has this permission')
    }
  }else{
    this.logger.error(`Permission ${id} not found`)
    throw new NotFoundException('Permission not found')
  }
}
```

фиг. 3.62 добавяне на роля към право

3.4.4 Роли

Във фигура 3.63 може да бъде разгледан контролерът за ролите.

```
@Controller('roles')
export class RolesController {
  constructor(private readonly rolesService: RolesService) {}

  @checkAbilities({action: SubjectActions.CREATE, subject: 'role'})
  @Post('/create')
  create(@Body() createRoleDto: CreateRoleDto, @Req() request: UserRequest)
  {
    return this.rolesService.createRole(createRoleDto, request.user);
  }

  @checkAbilities({action: SubjectActions.READ, subject: 'role'})
  @Get('/all')
  findAll(@Req() request: UserRequest) {
    return this.rolesService.findRoles(request.user);
  }
}
```

```

@RequireSuperuser()
@Get('/all/:client_id')
findAllInInstance(@Param('client_id') clientID: string){
    return this.rolesService.findAllInClient(+clientID);
}

@checkAbilities({action: SubjectActions.READ, subject: 'role'})
@Get('/:id')
findOne(@Param('id') id: string, @Req() request: UserRequest) {
    return this.rolesService.findRole(+id, request.user);
}

@checkAbilities({action: SubjectActions.UPDATE, subject: 'role'})
@Patch('/update/:id')
update(@Param('id') id: string, @Body() updateRoleDto: UpdateRoleDto,
@Req() request: UserRequest) {
    return this.rolesService.update(+id, updateRoleDto, request.user);
}

@checkAbilities({action: SubjectActions.DELETE, subject: 'role'})
@Delete('/:id')
remove(@Param('id') id: string, @Req() request: UserRequest) {
    return this.rolesService.removeRole(+id, request.user);
}
}

```

фиг. 3.63 контролер за роли

Функциите за създаване, вземане и изтриване на роля (фиг. 3.64) са аналогични с тези в service-а за оператори.

```

createRole(createOperatorDto: CreateRoleDto, user:any) {
    return user.is_admin ? this.createOperatorGlobally(createOperatorDto) :
    this.createRoleInInstance(createOperatorDto, user.client_id);
}

createRoleInInstance(createRoleDto: CreateRoleDto, clientID: number){
    const {client, ...sanitizedDto} = createRoleDto;
    const role = this.roleRepository.create({
        client: {
            id: clientID
        }, ...sanitizedDto})
    return this.roleRepository.save(role);
}

createOperatorGlobally(createRoleDto: CreateRoleDto){
    const role = this.roleRepository.create(createRoleDto);
}

```



```

    return this.roleRepository.save(role);
}

findAllInClient(clientID: number) {
    return this.roleRepository.find({
        where: {
            client: {id : clientID}
        }, relations:['permissions', 'client'])
}

findAll() {
    return this.roleRepository.find({
        relations:['permissions', 'client']
    });
}

async findRoles(user: any){
    return user?.is_admin ? this.findAll() :
this.findAllInClient(user.client_id);
}

async findByIdAndClient(id: number, clientID: number){
    return this.roleRepository.createQueryBuilder('role')
        .leftJoinAndSelect('role.client', 'client')
        .leftJoinAndSelect('role.operators', 'operators')
        .leftJoinAndSelect('role.permissions', 'permissions')
        .where('role.id = :id', { id })
        .andWhere('role.client.id = :clientID', { clientID })
        .getOne()
}

async findById(id: number) {
    return this.roleRepository.createQueryBuilder('role')
        .leftJoinAndSelect('role.client', 'client')
        .leftJoinAndSelect('role.operators', 'operators')
        .leftJoinAndSelect('role.permissions', 'permissions')
        .where('role.id = :id', { id })
        .getOne()
}

async findRole(id: number, user: any){
    return user?.is_admin ? this.findById(id) : this.findByIdAndClient(id,
user.client_id)
}

findByName(name: string){
    return this.roleRepository.createQueryBuilder('role')

```

```

        .leftJoinAndSelect('role.client', 'client')
        .leftJoinAndSelect('role.operators', 'operators')
        .leftJoinAndSelect('role.permissions', 'permissions')
        .where('role.name = :name', { name })
        .getOne();
    }
    removeRole(id:number, user: any){
        return user?.is_admin ? this.removeById(id) :
        this.removeByClientAndId(id, user.client_id);
    }

    removeByClientAndId(id: number, clientID: number){
        return this.roleRepository.delete({
            id,
            client : {
                id: clientID
            }
        })
    }

    removeById(id: number) {
        return this.roleRepository.delete(id);
    }

```

фиг. 3.64 функции за създаване, вземане и изтриване на роля

Това, с което ролите се различават от останалите компоненти на системата е функцията за промяна на роля (фиг. 3.65).

```

async update(id: number, updateRoleDto: UpdateRoleDto, user: any) {
    let role = await this.findRole(id, user);
    if (!role) {
        throw new NotFoundException('Role not found');
    }

    try {
        if(user.is_admin){
            role = await this.updateClient(role, updateRoleDto.client);
        }

        await this.enqueueUpdateEntities(
            role,
            updateRoleDto.operators,
            EntityService.OPERATOR,
            role.client ? role.client.id : undefined
        );
    }
}

```

```

    await this.enqueueUpdateEntities(
        role,
        updateRoleDto.permissions,
        EntityService.PERMISSION,
        role.client ? role.client.id : undefined
    );

    } catch (error) {
        this.logger.error(`An error occurred updating role ${id} :
        ${error.message}`, error.stack);
        throw new InternalServerErrorException(error.message);
    }
}

```

фиг. 3.65 функция за промяна на роля

Началната логика на функцията е същата - роля може да бъде добавена към клиент единствено и само когато потребителят е админ. Различното е начинът, по който се добавят оператори и права към ролята. Тъй като при добавяне и на оператори, и на права към ролята процесът става натоварващ, е използвана опашката за процеси за олекотяване и разпределяне на работата.

```

private async enqueueUpdateEntities(role: Role, entitiesDto: any[] |
undefined,
    service: EntityService, clientID: number | undefined) {

    await this.queueService.add(
        this.queueService.getQueue(`role.${role.id}`),
        `role.${role.id}.addToEntityProcess`,
        {
            role,
            entitiesDto,
            entityService: service,
            clientID
        }
    );
}

```

фиг. 3.65 добавяне на задача за добавяне на права/оператори към процес в опашката

Функцията във фигура 3.64 добавя задача към процесът ``role.${role.id}.addToEntityProcess`` в опашката. Задачата приема за аргументи :

- `role` - ролята
- `entitiesDto` - масивът от обекти за пренос на информация, които ще бъдат добавени
- `entityService` - енумерация за типа на service-a, който се използва
- `clientID` - идентификационният номер на системния клиент

Конфигурацията на процесът е реализирана във класът `RolesListener` (фиг. 3.66).

```
@Injectable()
export class RolesListener {
  constructor(
    @Inject('REDIS')
    private readonly redis: Redis,
    private queueService: QueueService,
    private operatorService: OperatorService,
    private premissionService: PermissionsService,
    private roleService: RolesService) {
    this.roleService.findAll().then((roles: any) => {
      roles.forEach((role: any) => {
        const queue = this.queueService.getQueue(`role.${role.id}`);
        this.queueService.createProcess(queue,
          `role.${role.id}.addToEntityProcess`,
          this.addToEntityProcess.bind(this),
        )
      })
    })
  }

  async addToEntityProcess(job: Job<{
    role: Role, entitiesDto:
    any, entityService:
    EntityService, clientID: number }>){

    const { role, entitiesDto, entityService, clientID } = job.data;;
```

```

let service: any;
let addRole: Function;
switch (entityService){
  case EntityService.OPERATOR:
    service = this.operatorService;
    addRole = (role: Role, entity: Operator) => role.operators?.push(entity)
    break;
  default :
    case EntityService.PERMISSION:
      service = this.premissionService;
      addRole = (role: Role, entity: Permission) =>
role.permissions?.push(entity)
      break;
}

try{

  //TODO : insert operations
  if (entitiesDto && entitiesDto.length > 0) {
    const promises = entitiesDto.map(async (entity: any) => {
      let roleEntity;

      if(entityService == EntityService.OPERATOR)
        roleEntity = await service.addRole(entity.id, role, {client_id:
clientID});

      else
        roleEntity = await service.addRole(entity.id, role);

      addRole(role, roleEntity);
    });
    await Promise.all(promises).then(async () => {
      return await this.roleService.saveOne(role);
    });
  }
}catch(error){
  console.error(error);
  throw new InternalServerErrorException(error.message);
}
}
}

```

фиг. 3.66 RolesListener клас

В конструктора на `RolesListener` се вземат всички роли и се създава процес за всяка една от тях, при който при обработка задачата ще изпълнява функцията `addToEntityProcess`. `AddToEntityProcess` взима данните от задачата и базирано на подадената `entityService` енумерация избира дали да използва `service`-ът на операторите или този на

ролите, след което преминава през масива от обекти. При случай на service-ът на операторите, се подава и идентификационния номер на инстанцията, за да може да бъде намерен операторът в нея. След добавянето на ролята към конкретния обект, се извиква lambda функцията `addRole`, чрез която към масива от обекти (оператори или права) на ролята се добавя конкретният обект. Всички тези асинхронни операции се капсулират в карта от promise-и, която след това бива обработена чрез `Promise.all()`. След обработка на promise-ите е запазена променената роля.

3.4.5 Клиенти на системните клиенти

Във фигура 3.67 може да бъде разгледан контролерът за потребителите (клиентите) на системния клиент.

```
@Controller('customer')
export class CustomerController {
    constructor(private readonly customerService: CustomerService) {}

    @RequireApiKey()
    @Post('auth/register')
    register(@Body() createCustomerDto: CreateCustomerDto) {
        return this.customerService.register(createCustomerDto);
    }

    @RequireApiKey()
    @Post('auth/login')
    login(@Body() loginCustomerDto: LoginCustomerDto) {
        return this.customerService.login(loginCustomerDto);
    }

    @RequireApiKey()
    @Patch('/deactivate/:id')
    deactivate(@Param('id') id: string){
        return this.customerService.deactivate(+id);
    }

    @checkAbilities({action: SubjectActions.READ, subject: 'customer'})
```

```

@Get('/all')
findAll(@Req() request: UserRequest) {
    return this.customerService.findCustomers(request.user);
}

@RequireSuperuser()
@Get('/all/:client_id')
findAllInClient(@Param('client_id') clientID: string) {
    return this.customerService.findAllInClient(+clientID)
}

@checkAbilities({action: SubjectActions.READ, subject: 'customer'})
@Get('/:id')
findOne(@Param('id') id: string, @Req() request: UserRequest) {
    return this.customerService.findCustomerById(+id, request.user);
}

@checkAbilities({action: SubjectActions.UPDATE, subject: 'customer'})
@Patch('ban/:id')
ban(@Param('id') id: string, @Body() statusDto: StatusDto, @Req() request:
UserRequest){
    return this.customerService.ban(+id, request.user, statusDto)
}

@checkAbilities({action: SubjectActions.UPDATE, subject: 'customer'})
@Patch('update/:id')
update(@Param('id') id: string, @Body() updateCustomerDto:
UpdateCustomerDto, @Req() request: UserRequest) {
    return this.customerService.update(+id, request.user,
updateCustomerDto);
}

@checkAbilities({action: SubjectActions.DELETE, subject: 'customer'})
@Delete('delete/:id')
remove(@Param('id') id: string, @Req() request: UserRequest) {
    return this.customerService.removeCustomer(+id, request.user);
}
}

```

фиг. 3.67 контролер за потребителите (клиентите) на системния клиент

Функциите за вземане, изтриване и променяне на клиент (фиг. 3.68) са аналогични с тези в service-ите за оператори и роли.

```

async findByEmailAndClient(email: string, clientID: number){
    return await this.customerRepository.findOneOrFail({

```

```

        where: {
            email: email,
            client : { id: clientID }
        },
        relations: ['client']
    });
}

async findByEmail(email: string){
    return await this.customerRepository.findOneOrFail({
        where: {
            email: email,
        },
        relations: ['client']
    });
}

async findByIdAndClient(id: number, clientID: number){

    return await this.customerRepository.findOneOrFail({
        where: {
            id,
            client: {id : clientID}
        },
        relations: ['client']
    })
}

async findById(id: number){
    return await this.customerRepository.findOneOrFail({
        where: {
            id,
        },
        relations: ['client']
    })
}

async findCustomers(user: any){
    return user?.is_admin ? this.findAll() :
this.findAllInClient(user.client_id);
}

async findCustomerById(id: number, user: any){
    return user?.is_admin ? this.findById(id) : this.findByIdAndClient(id,
user.client_id)
}

```



```

    async findCustomerByEmail(email: string, user: any){
      return user?.is_admin ? this.findByEmail(email) :
this.findByEmailAndClient(email, user.client_id);
    }

    findAllInClient(clientID: number) {
      return this.customerRepository.find({
        where: {
          client: {id : clientID}
        })
    }

    findAll() {
      return this.customerRepository.find({
        relations: ['client']
      });
    }
    async update(id: number, user: any, updateCustomerDto: UpdateCustomerDto)
    {

      try{
        const customer = await this.findCustomerById(id, user);
        const {id: customerID, client: customerClient, email: customerEmail,
...sanitizedCustomer} = customer;
        if(updateCustomerDto.email && updateCustomerDto.email !==
customerEmail){
          try{
            await this.findCustomerByEmail(updateCustomerDto.email, user);
          }catch(error){console.error(error)}
          finally{
            throw new Error(`User with email ${updateCustomerDto.email}
already exists under client`);
          }
        }else{
          updateCustomerDto.email = customerEmail;
        }

        const updatedCustomer = {
          id: customerID,
          client: customerClient,
          ...updateCustomerDto
        }

        return this.customerRepository.save(updatedCustomer)

      }catch(error){
        throw new NotFoundException(`Could not find user

```

```

    ${updateCustomerDto.email} within client`);
    }

    }
    removeCustomer(id:number, user: any){
        return user?.is_admin ? this.removeById(id) :
    this.removeByClientAndId(id, user.client_id);
    }

    removeByClientAndId(id: number, clientID: number){
        return this.customerRepository.delete({
            id,
            client : {
                id: clientID
            }
        })
    }

    removeById(id: number) {
        return this.customerRepository.delete(id);
    }
}

```

фиг. 3.68 функции за вземане, изтриване и променяне на клиент

За създаване на клиент се използва обект за пренос на данни. Той може да бъде разгледан във фиг. 3.69.

```

export class CreateCustomerDto {

    @IsEmail()
    email: string;

    @IsString()
    public readonly password: string;

    @IsOptional()
    firstName?: string;

    @IsOptional()
    lastName?: string;

    @IsOptional()
    number: string;

    @IsOptional()
    notes: string;
}

```

```

@IsObject()
public readonly client: Partial<Client>;

}

```

фиг. 3.69 *обект за пренос на данни за създаване на обект*

Отличаващите се функции в контролера са под заявките

```

@Post('auth/register'), @Post('auth/login'),
@Patch('/deactivate/:id'), @Patch('ban/:id').

```

```

async ban(id: number, user: any, statusDto: StatusDto) {
  try{
    const customer = await this.update(id, user, {
      account_status: AccountStatus.BANNED,
      notes : statusDto.reason,
    } as UpdateCustomerDto);

    const queue = this.queueService.getQueue(`customer.${id}`);

    await this.queueService.add(
      queue,
      `customer.${id}.sendStatusUpdateMessageProcess`,
      {
        mailDto: {
          receiver: customer.email,
          title: 'Your account has been banned',
          client: customer.client.name,
          reason: customer.notes,
          status: AccountStatus.BANNED,
          duration: DURATION_WORD_KEYS[statusDto.duration]
        } as MailDto
      }
    )

    await this.enqueueStatusChange(queue,
      customer, AccountStatus.PENDING_ACTIVATION,
      `Previously banned for ${statusDto.reason}`,
      CHANGE_STATUS_LENGTHS[statusDto.duration]
    )

    return `Customer ${customer.id} banned successfully`
  }catch(error){
    this.logger.log(`An error occurred banning customer ${id} :

```

```

    ${error.message}`, error.stack);
    throw error;
  }
}

```

фиг. 3.70 функция за блокиране на клиент

Във фигура 3.70 е изложена функцията за блокиране на клиент. Бива променен статуса на акаунта към блокиран посредством update, биват подадени само нужни параметри от UpdateCustomerDto (фиг. 3.71), след което обектът е cast-нат към този тип. Взима се опашката за процеси, свързани с конкретния клиент, и се добавя задача от тип процеса за изпращане на имейл за промяна на статуса му. Този процес се изпълнява в опашка с оглед скалируемостта на системата и натоварването. Чрез функцията enqueueStatusChange (фиг. 3.72) се добавя процес за промяна на статуса на акаунта от блокиран към изчакващ активация, след като премине подадения период от време.

```

export class UpdateCustomerDto extends PartialType(CreateCustomerDto) {

    @IsOptional()
    @IsEmail()
    email: string;

    @IsOptional()
    @IsString()
    public readonly password: string;

    @IsOptional()
    number: string;

    @IsOptional()
    account_status: AccountStatus;

    @IsOptional()
    notes: string;

}

```

фиг. 3.71 обект за пренос на данни за промяна на потребител на системния клиент

```

async enqueueStatusChange(queue: Bull.Queue,
  customer: Customer,
  status: AccountStatus,
  reason: string,
  delay: number){
  await this.queueService.add(
    queue,
    `customer.${customer.id}.changeAccountStatusProcess`,
    {
      customer,
      account_status: status,
      notes: reason
    },
    {delay}
  )
}

```

фиг. 3.72 функция за добавяне на задача от тип процес за промяна на статус на акаунта към опашката

```

async deactivate(id: number){
  try{
    const customer = await this.update(id, {is_admin: true}, {
      account_status: AccountStatus.DEACTIVATED,
    } as UpdateCustomerDto);

    const queue = this.queueService.getQueue(`customer.${id}`);

    await this.queueService.add(
      queue,
      `customer.${id}.sendStatusUpdateMessageProcess`,
      {
        mailDto: {
          receiver: customer.email,
          title: 'Your account has been deactivated',
          client: customer.client.name,
          reason: customer.notes,
          status: AccountStatus.DEACTIVATED,
        } as MailDto
      }
    )

    const deactivationJob = await this.queueService.add(
      queue,
      `customer.${id}.deactivationProcess`,
      {
        customer,

```

```

        mailDto: {
            receiver: customer.email,
            client: customer.client.name,
        } as MailDto
    },
    {delay: CHANGE_STATUS_LENGTHS["MONTH"]}
)

    await
    this.redis.set(`customer.${customer.client.id}.${customer.id}.deactivationJob`, deactivationJob?.id as number);

    return `Customer account ${customer.id} deactivated successfully`
} catch (error) {
    throw error;
}
}

```

фиг. 3.73 *функция за деактивация на акаунт*

Функциите за деактивация и блокиране на акаунт работят по аналогичен начин. Разликата между двете е, че докато функцията за блокиране насрочва връщането на акаунта към статус изчакващ активация след като изтече съответно подадения период от време, то функцията за деактивиране насрочва акаунта за изтриване след месец. Идентификационният номер на задачата бива запазен в хранилището за данни под ключ `customer.<идентификационен номер на системния клиент>.<идентификационен номер на клиента>.deactivationJob`, с цел при нужда да може да бъде намерен процесът за изтриване и да бъде спрял преди изпълнението му.

```

async register(createCustomerDto: CreateCustomerDto) {

    const {password: rawPassword, ...customerDto} = createCustomerDto;
    const hashedPassword = await bcrypt.hash(rawPassword, 10);
    const customer = this.customerRepository.create({password:
hashedPassword, ...customerDto});
    customer.account_status = AccountStatus.PENDING_ACTIVATION;
    try{
        await this.findByEmailAndClient(customer.email, customer.client.id);
    } catch (error){
        const savedCustomer = await this.customerRepository.save(customer);
    }
}

```

```

    await this.enqueueStatusChange(
      this.queueService.getQueue(`customer.${customer.id}`),
      savedCustomer,
      AccountStatus.ACTIVE,
      'initial account activation',
      CHANGE_STATUS_LENGTHS["MINUTE"]
    )

    return this.authService.constructCustomerToken(customer);
  }
  throw new Error('Customer with this email already registered under
client')
}

```

фиг. 3.74 функция за регистрация (създаване на клиент)

Фигура 3.74 излага функцията за регистрация (създаване на клиент). Тя използва `CreateClientDto` (фиг. 3.69). Създаването и записването на акаунт в базата е аналогично на това за операторски акаунти. Отличаващото се тук са началният статус на акаунта и промяната му. С цел ограничение на натоварването върху приложението на системния клиент и евентуални бъдещи валидации на данни и проверки, началният статут на акаунта е зададен като изчакващ активация. Бива добавен процес за промяна на статуса към активен за минута след създаването. Отговорът на заявката за регистрация е JWT token. Информацията която носи е за :

- идентификационен номер на клиента
- идентификационен номер на инстанцията в системата за управление на клиенти
- статус на акаунта на клиента

```

async login(loginCustomerDto: LoginCustomerDto){
  try{
    const customer = await
this.findByEmailAndClient(loginCustomerDto.email, loginCustomerDto.client.id
as number);
    if(customer.account_status === AccountStatus.DEACTIVATED){
      const activeJob = await

```

```

this.redis.get(`customer.${customer.client.id}.${customer.id}.deactivationJob`);
    if(activeJob){
        await
this.redis.del(`customer.${customer.client.id}.${customer.id}.deactivationJob`);

        const jobRemoved = await this.queueService.remove(
            this.queueService.getQueue(`customer.${customer.id}`),
            activeJob as string
        )
        if(jobRemoved){
            await this.update(customer.id, {is_admin: true}, {
                account_status: AccountStatus.ACTIVE,
            } as UpdateCustomerDto);
        }
    }
}
if(await bcrypt.compare(loginCustomerDto.password,
customer.password)){
    return this.authService.constructCostumerToken(customer);
}else{
    throw new UnauthorizedException('Invalid customer credentials for client');
}
}catch(error){
    throw new NotFoundException(`Could not find user ${loginCustomerDto.email} within client ${loginCustomerDto.client.id}`);
}
}
}

```

фиг. 3.74 функция за вписване на клиент

Във фигура 3.74 може да бъде разгледана функцията за вписване на клиент. Тя работи аналогично на функцията за вписване на оператор. Използва `LoginCustomerDto` (фиг. 3.75). Различава се с това, че при намиране на акаунта се извършва проверка дали е деактивиран. Ако е се взима записаният в хранилището данни идентификационен номер на задачата от процеса за деактивация, след което тя бива изтрита от опашката. Ако успешно е изтрита, статусът се променя на активен. Така успешно се предотвратява изтриването на акаунта, тъй като процесът на деактивация вече не е сила.


```
export class LoginCustomerDto {

    @IsEmail()
    email: string;

    @IsString()
    public readonly password: string;

    @IsObject()
    public readonly client: Partial<Client>;

}
```

фиг. 3.75 обект за пренос на данни за вписване на клиент

Всички тези функции използват процеси от опашката. Тези процеси са регистрирани в `StatusListener` (фиг. 3.76) - клас, подобен на `RolesListener` (фиг. 3.66).

```
@Injectable()
export class StatusListener {
    private logger: Logger = new Logger(StatusListener.name)
    constructor(
        @Inject('REDIS')
        private readonly redis: Redis,
        private queueService: QueueService,
        private customerService: CustomerService,
        private mailService: MailService){
        this.customerService.findAll().then((customers: any) => {
            customers.forEach((customer: any) => {
                const queue =
this.queueService.getQueue(`customer.${customer.id}`);

                this.queueService.createProcess(queue,
                    `customer.${customer.id}.changeAccountStatusProcess`,
                    this.changeAccountStatusProcess.bind(this),
                );

                this.queueService.createProcess(queue,
                    `customer.${customer.id}.sendStatusUpdateMessageProcess`,
                    this.sendStatusUpdateMessageProcess.bind(this),
                );

                this.queueService.createProcess(queue,
                    `customer.${customer.id}.deactivationProcess`,
                    this.deactivationProcess.bind(this),
                );
            });
        });
    }
}
```

```

        async () => {
            await
this.redis.del(`customer.${customer.client.id}.${customer.id}.deactivationJob`);

            this.logger.log(`Customer ${customer.id} deleted`);
        }
    );

    })
}

}

async changeAccountStatusProcess(job: Job<{
    customer: Customer,
    account_status: AccountStatus,
    notes?: string;
}>){
    const {customer, account_status, notes} = job.data;
    try{

        await this.customerService.update(customer.id, {client_id:
customer.client.id}, {
            account_status,
            notes: notes ?? ''
        } as UpdateCustomerDto)

        await this.mailService.sendStatusUpdate({
            receiver: customer.email,
            title: 'Account status change',
            name: `${customer?.first_name} ${customer?.last_name}`,
            client: customer.client.name,
            reason: notes ?? '',
            status: account_status
        } as MailDto)
        Logger.log(`operation ${job.id} of changing account status
finished on customer ${customer.id}`)

    }catch(error){
        Logger.error(error);
        throw error;
    }
}

}

async deactivationProcess(job: Job<{
    customer: Customer,
    mailDto: MailDto

```

```

    }>){
      try{
        const {customer, mailDto} = job.data;
        await this.customerService.removeById(customer.id);
        await this.mailService.sendAccountDeactivationUpdate(mailDto);
      }catch(error){
        Logger.error(error);
        throw error;
      }
    }
  }

  async sendStatusUpdateMessageProcess(job: Job<{ mailDto: MailDto }>){
    const {mailDto} = job.data;
    await this.mailService.sendStatusUpdate(mailDto);
  }
}

```

фиг. 3.76 StatusListener клас

Процесите използват функции, които са разгледани и обяснени. Процесът за деактивация на акаунт се използва от опционалния `onCompleteCallback`, за да може след успешното изпълнение на задачата за деактивация да бъде изтрят идентификационният ѝ номер от хранилището за данни.

Обектът за пренос на данни за статус на акаунт, използван във функцията за блокиране (фиг. 3.70) може да бъде разгледан във фигура 3.77

```

export class StatusDto {

  @IsString()
  reason: string;

  @IsDurationKeyOfStatusLengths()
  duration: keyof typeof CHANGE_STATUS_LENGTHS;

}

```

фиг. 3.77 обект за пренос на данни за статус на акаунт

Обектът използва custom декоратор за проверка дали подадения период време е ключ от константата `CHANGE_STATUS_LENGTHS` (фиг. 3.2), който може да бъде разгледан на фигура 3.78.

```

export function IsDurationKeyOfStatusLengths(validationOptions?:
ValidationOptions) {
  return function (object: Record<string, any>, propertyName: string) {
    registerDecorator({
      name: 'isDurationKeyOfStatusLengths',
      target: object.constructor,
      propertyName: propertyName,
      options: validationOptions,
      validator: {
        validate(value: any, args: ValidationArguments) {
          return Object.keys(CHANGE_STATUS_LENGTHS).includes(value);
        },
        defaultMessage(args: ValidationArguments) {
          return `${args.property} must be a valid key of
CHANGE_STATUS_LENGTHS`;
        },
      },
    });
  };
}

```

фиг. 3.78 проверка дали подадения период време е ключ от константата `CHANGE_STATUS_LENGTHS`.

3.5 Автентикация и оторизация

Автентикацията и оторизацията са два често бъркани, но напълно различни в същността си процеси.

Автентикацията представлява процесът на проверка на самоличността на потребител или система, които се опитват да получат достъп до ресурс или услуга. Тя гарантира, че потребителят или системата са тези, за които се представят, преди да се предостави достъп до защитени ресурси.

Оторизацията се основава на идентичността, ролите и разрешенията на автентифицирания потребител, свързани с неговия акаунт или токен. Тя представлява процесът на определяне на това дали автентифицираният потребител или система имат достъп до определени ресурси или за извършване на определени операции.

3.5.1 Автентикация

В рамките на описанието на реализацията може да бъде забелязано, че при създаването на ключ за достъп до приложно програмния интерфейс, при вписването на потребителски акаунти, както и при регистрацията и вписването на системния клиент, се използват функции от `AuthService`. `AuthService` представлява service-ът, който държи логиката за автентикацията. Заедно с него чрез имплементацията на `authGuard` и `clientApiKeyGuard` е постигната автентикацията в приложението.

```
@Module({
  imports: [
    PassportModule,
    JwtModule.registerAsync({
      inject: [ConfigService],
      useFactory: async (configService: ConfigService) => ({
        secret: configService.get<string>('JWT_SECRET'),
        signOptions: { expiresIn:
configService.get<string>('JWT_EXPIRATION') },
      }),
    ],
    controllers: [AuthController],
    providers: [AuthService, JwtStrategy],
    exports: [AuthService, JwtModule],
  })
```

фиг. 3.79 модул за автентикация

С цел по-добро разглеждане на service-ът за автентикация, първо ще бъдат разгледани модулът (фиг. 3.79), контролерът (фиг. 3.80) и съответните функции от service-а, използвани за имплементацията му. Модулът за автентикация конфигурира JWT модулът, използван за генерирането на токени чрез `JwtModule.registerAsync`. `Secret` полето дефинира думата, с която ще бъде криптиран токенът, а `signOptions` - в случая чрез полето `expiresIn` дефинира след колко време ще изтече токенът за достъп по подразбиране.

```

@Controller('auth')
export class AuthController {
    constructor(private authService: AuthService) {}

    @Post('login')
    @AllowUnauthorizedRequest()
    login(@Body() authUserDto: AuthUserDto): Promise<any> {
        return this.authService.loginOperator(authUserDto);
    }

    @Get('refresh')
    refresh(@Req() request: UserRequest){
        return this.authService.refreshTokens(request.user.sub,
            request.user.refreshToken)
    }

    @Post('logout')
    logout(@Req() request: UserRequest){
        return this.authService.logout(request.user.sub);
    }
}

```

фиг. 3.80 контролер за автентикация

Контролерът за автентикация (фиг. 3.80) е предназначен за вписването на оператори в системата. Липсва функционалност за регистрация въз основа на структурата на системата, тъй като акаунти могат да бъдат създадени само от служители на конкретния системен клиент. Контролерът разполага със заявки :

- `@Post('login')` - за вписване на акаунта
- `@Get('refresh')` - за опресняване на токена за достъп
- `@Post('logout')` - за отписване на акаунта

```

async loginOperator(authUserDto: AuthUserDto): Promise<{}> {
    try{
        let user = await this.validateUser(authUserDto);

        if(user){
            return this.generateTokens(user);
        }else{
            throw new ForbiddenException(`User with specified email or password
not found`)
        }
    }
}

```

```

    }
  }catch(error){
    this.logger.log(`An error occurred logging in user
    ${authUserDto.email} with password ${authUserDto.password} :
    ${error.message}`,
    error.stack);
    throw new ForbiddenException(`User with specified email or password
    not found`)
  }
}

```

фиг. 3.81 *вписване на оператор*

Във фигура 3.81 е изложена функцията за вписване на оператор. Тя използва функциите `validateUser` (фиг. 3.82) за да валидира съществуването на подадения потребител и `generateTokens` (фиг. 3.82) за генерирането на токени за достъп, които да бъдат върнати в отговор на заявката.

```

async validateUser(authUserDto: AuthUserDto): Promise<User | null> {
  try{
    const user = await this.userService.findByEmail(authUserDto.email);
    if (user && (await bcrypt.compare(authUserDto.password,
    user.password))) {
      return user;
    }
    return null;
  }catch(error){
    throw error;
  }
}

```

фиг. 3.82 *Валидиране на потребител*

Валидацията на потребител във фигура 3.82 се осъществява чрез търсенето на акаунт по подадения имейл в заявката. Ако е открит такъв акаунт е сравнена хешираната парола от базата с подадената. При съответствие е върнат намереният потребител, в противен случай не бива върнато нищо или бива хвърлена грешка.

```

async generateTokens(user: User){

  let payload: any = { sub: user?.id };

```

```

const refreshTokenOptions = {
  secret: this.configService.get<string>('JWT_REFRESH_SECRET'),
  expiresIn: '7d',
}

const refreshToken = this.jwtService.sign(
  payload,
  refreshTokenOptions
);

payload = {
  ...payload,
  refreshToken
};

const hashedToken = await bcrypt.hash(refreshToken, 10);
user = await this.userService.update(user?.id as number, {refreshToken:
hashedToken}, {}) as User;

return {
  access_token: this.jwtService.sign(payload),
  refresh_token: refreshToken,
  user: user
};
}

```

фиг. 3.83 генерация на токени

Фигура 3.83 разглежда функцията за генерация на JWT токени. `Payload` променливата представлява информацията, която ще се съдържа в JWT токена. Токенът за опресняване на токенът за достъп е с валидност 7 дни. Целта му е да бъде използван малко преди да изтече токенът за достъп, за да може да се продължи успешното протичане на потребителската сесия. Чрез `jwtService.sign()` се създават токенът за опресняване и токенът за достъп : те са подписани дигитално и са криптирани с думата, подадена в `secret` полето. Secret-ите между токенът за достъп и токенът за опресняване се различават. Токенът за опресняване бива хеширан и запазен в потребителя.

```

async refreshTokens(userId: number, refreshToken: string){
  try{
    const user = await this.userService.findById(userId);

```



```

    if (!user || !user.refresh_token)
        throw new ForbiddenException('Access Denied');

    this.jwtService.verify(refreshToken);
    if (!(await bcrypt.compare(refreshToken, user.refresh_token)))
        throw new ForbiddenException('Access Denied');

    return this.generateTokens(user);
} catch (error) {
    this.logger.error(`An error occurred refreshing token for user
    ${userId} : ${error.message}`, error.stack)
    throw new ForbiddenException('Access Denied');
}
}

```

фиг. 3.84 опресняване на токен за достъп

Функцията във фиг 3.84 служи за опресняване на токенът за достъп. Тя използва токенът за опресняване подаден в заявката, валидира го и го сравнява с този на потребителя от заявката. При успешна валидация биват генерирани нови токени. Така се постига запазването на потребителската сесия за повече време от това, за което ще изтече токенът за достъп.

```

async logout(userID: number){
    return this.userService.update(userID, {refreshToken: null}, {});
}

```

фиг. 3.85 отписване на потребител

Фигура 3.85 разглежда функцията за отписване на потребител. Тя премахва токенът за опресняване от акаунта, като така успешно възпира генерирането на нови токени и достъпът до сървъра без вписване наново.

```

constructCostumerToken(customer: Customer){
    const payload = { sub: {id: customer.id, client_id: customer.client.id, status:
    customer.account_status} };
    return {
        token: this.jwtService.sign(payload),
        user: customer
    }
}

constructApiKey(clientID: number){
    const payload = { sub : clientID }
}

```

```

const apiKeyOptions = {
  secret: this.configService.get<string>('JWT_API_KEY_SECRET'),
  expiresIn: '30d',
}
return this.jwtService.sign(payload, apiKeyOptions)
}

```

фиг. 3.86 конструиране на токен за достъп до приложно програмния интерфейс и токен за потребителите на системния клиент

Във фигура 3.86 са разгледани помощни функции, реализирани в service-а за автентикация, използвани за генериране на токени за достъп до приложно-програмния интерфейс и за потребителите на системния клиент. Реализацията на автентикирането на потребители обаче не свършва до тук. За завършване са нужни да бъдат имплементирани два guard - а :

- `AuthGuard` - за проверка на токените за достъп и опресняване на потребителя
- `ClientApiKeyGuard` - за проверка на токените за достъп на системните клиенти за приложно-програмния интерфейс

```

export class AuthGuard implements CanActivate {
  constructor(private jwtService: JwtService,
    private userService: UserService,
    private reflector: Reflector,
    private readonly configService: ConfigService) {}

  async canActivate(context: ExecutionContext): Promise<boolean> {
    const request = context.switchToHttp().getRequest();
    const allowUnauthorizedRequest =
this.reflector.getAllAndOverride<boolean>(DecoratorMetadata.UNAUTHORIZED_REQUEST_DECORATOR,
    [context.getHandler(),
    context.getClass()]);
    const allowClientAPIRequest =
this.reflector.getAllAndOverride<boolean>(DecoratorMetadata.REQUIRE_API_KEY,
    [context.getHandler(),
    context.getClass()]);

    if(allowClientAPIRequest){
      request.user = {...request.user, is_api: true};
    }
  }
}

```

```

    }

    if(allowUnauthorizedRequest){
        request.user = {...request.user, is_authorized: true};
    }

    return allowClientAPIRequest || allowUnauthorizedRequest || (await
this.validateRequest(request));
}

async validateRequest(request: any): Promise<boolean> {
    const authHeader = request.headers['authorization'];

    if(!authHeader){
        return false;
    }

    const [bearer, token] = authHeader.split(' ');

    if(!bearer || bearer !== 'Bearer'){
        return false;
    }

    try {
        const decodedToken = this.jwtService.verify(token);
        request.user = decodedToken;
        console.log('refreshToken', request.user);

        this.jwtService.verify(request.user.refreshToken, {
            secret: this.configService.get<string>('JWT_REFRESH_SECRET')
        });

        const user = await this.userService.findById(request.user.sub);
        if(!user || !user.refresh_token){
            return false;
        }

        return true;
    } catch (error) {
        return false;
    }
}
}

```

фиг. 3.87 проверка на токените за достъп и опресняване на потребителя

Във фиг 3.87 може да бъде разгледан `authGuard`. Чрез `context.switchToHttp().getRequest();` се взема подадената заявка, след това чрез

```
this.reflector.getAllAndOverride<boolean>(<името на метаданните от декоратора>
    [context.getHandler(),
    context.getClass()]);
```

се проверява дали на заявката е позволено да премине без автентикация (чрез `@AllowUnauthorizedRequest`) или че е заявка, която трябва да бъде валидирана от другия `guard` за автентикация (`@RequireApiKey` - за токена за достъп до приложно-програмния интерфейс). Ако е такава заявка, `guard`-ът директно пропуска потребителя. Във функцията `validateToken` от `authorization` header-а се взима токенът за достъп. Ако успешно бива верифициран, се търси потребител с идентификационен номер, взет от данните, предадени през токена, проверява се дали потребителят съществува и има токен за опресняване (тоест дали не е отписан) и ако ги има този `guard` успешно пропуска достъпа.

```
@Injectable()
export class ClientApiKeyGuard implements CanActivate {
    private logger = new Logger(ClientApiKeyGuard.name);
    constructor(private reflector: Reflector,
        private readonly jwtService: JwtService,
        private readonly configService: ConfigService){}
    async canActivate(
        context: ExecutionContext,
    ): Promise<boolean> {

        const requireApiKey =
this.reflector.getAllAndOverride<boolean>(DecoratorMetadata.REQUIRE_API_KEY,
        [context.getHandler(),
        context.getClass()]);
        const request = context.switchToHttp().getRequest();
        const user = request.user;
        const clientApiKey = request.headers['authorization'];
```

```

    if(user?.is_admin || user?.is_authorized){
        return true;
    }

    if(requireApiKey){
        const [bearer, token] = clientApiKey.split(' ');
        if(!bearer || bearer !== 'Bearer'){
            return false;
        }

        try{
            this.jwtService.verify(token, {
                secret: this.configService.get<string>('JWT_API_KEY_SECRET')
            });
        }catch(error){
            this.logger.error(`An error occurred validating token:
${error.message}`, error.stack);
            return false;
        }
        const apiKeysMatch = await bcrypt.compare(token, user.api_key);
        if(apiKeysMatch){
            request.user = {...request.user, is_authorized: true};
        }
        return apiKeysMatch;
    }

    return true;
}
}

```

фиг. 3.88 проверка на токени за достъп на системните клиенти за приложно-програмния интерфейс

Фигура 3.88 излага имплементацията на guard-а за проверка на токени за достъп на системните клиенти за приложно-програмния интерфейс. Разглежда се дали потребителят е допуснат без токен (зададено в `AuthGuard` - фиг. 3.87) или дали е админ (`SuperuserGuard` - фиг. 3.89). Ако заявката изисква ключ за достъп до приложно програмния интерфейс се сравнява този от `authorization header`-а с полето `api_key` на потребителя на заявката (зададено в `ClientGuard` - фиг. 3.90).

3.5.2 Оторизация

Оторизацията в рамките на системата се осъществява чрез три guard-a :

- `SuperuserGuard` - за проверка дали един акаунт е админски
- `ClientGuard` - за успешното разделение на инстанциите и проверяването на съответствието им
- `AbilityGuard` - за проверяване на потребителските роли

```
export class SuperuserGuard implements CanActivate {
  private logger: Logger = new Logger(SuperuserGuard.name);
  constructor(private reflector: Reflector,
    @Inject(forwardRef(() => OperatorService))
    private readonly operatorService: OperatorService){}

  async canActivate(
    context: ExecutionContext
  ) {

    const request = context.switchToHttp().getRequest();
    const user = request.user;

    if(user?.is_authorized || user?.is_api){
      return true;
    }

    try{
      const operator = await
this.operatorService.findOneByUser(user.sub);
      if(operator.roles && operator.roles.some((role) => {
        return role.name === SUPERUSER;
      })){
        request.user = {...request.user, is_admin: true};
      }
    }catch(error){
      this.logger.error(`An error occurred during superuser checking :
${error}`, error.stack);
      return false;
    }
  }
}
```

```

        return true;
    }
}

```

фиг. 3.89 проверка за админски акаунти

`SuperuserGuard` реализира проверката дали един акаунт е админски. Основата на функционалността му седи зад това, че чрез service-а за оператори намира оператор, отговарящ на идентификационния номер на потребителя, след което минава през ролите му и ако някоя от тях отговаря на ролята за админ, слага `is_admin` флаг.

```

@Injectables()
export class ClientGuard implements CanActivate {
    private logger: Logger = new Logger(ClientGuard.name);
    constructor(private clientService: ClientService, private reflector:
Reflector){}
    async canActivate(
        context: ExecutionContext,
    ) {

        const request = context.switchToHttp().getRequest();
        const user = request.user;
        const client = request.headers['x-client'];
        let clientApiKey = '';
        const requireApiKey =
this.reflector.getAllAndOverride<boolean>(DecoratorMetadata.REQUIRE_API_KEY,
[context.getHandler(), context.getClass()]);

        if(user?.is_authorized || user?.is_admin){
            return true;
        }

        if(!client){
            return false;
        }

        try{
            const foundClient = await this.clientService.findByName(client);
            if(requireApiKey){
                clientApiKey = request.headers['authorization'];
                if(!foundClient.api_key || !foundClient?.api_key.token){
                    return false;
                }
            }
        }
    }
}

```

```

        }
        request.user = {...request.user, api_key:
foundClient.api_key.token};
    }

    request.user = {...request.user, client_id : foundClient.id};
}catch(error){
    this.logger.error(`An error occurred during client checking :
${error}`, error.stack);
    return false;
}

return true;
}
}

```

фиг. 3.90 *успешното разделяне на инстанциите и проверяването на съответствието им*

Във фигура 3.90 е показана реализацията на guard-a за системните клиенти. От custom header-a `x-client` се взима името на инстанцията за която се отнася заявката, след което ако заявката изисква ключ за достъп до приложно-програмния интерфейс се проверява дали конкретния системен клиент е снабден с такъв. Ако не, стига да е намерен, се добавя идентификационния му номер към заявката.

```

@Inject()
export class AbilityGuard implements CanActivate {
    private logger: Logger = new Logger(AbilityGuard.name);

    constructor(
        private reflector: Reflector,
        @InjectRepository(Operator)
        private operatorRepository: Repository<Operator>,
    ) {}

    async canActivate(context: ExecutionContext): Promise<boolean> {

        const requireSuperuser =
this.reflector.getAllAndOverride<boolean>(DecoratorMetadata.REQUIRE_SUPERUSE
R_ROLE,
        [context.getHandler(), context.getClass()]);
    }
}

```



```

const rules: any =
  this.reflector.get<RequiredRule[]>(DecoratorMetadata.CHECK_ABILITY,
context.getHandler()) ||
  [];
const request = context.switchToHttp().getRequest();

for(const rule of rules){
  if(!(await this.findSubject(rule?.subject))){
    this.logger.error(`Permission subject ${rule.subject} does not
exist`);
    throw new NotFoundException(`Permission subject ${rule.subject} does
not exist`);
  }

  if (!(rule.action as SubjectActions in SubjectActions)) {
    this.logger.error(`Permission action ${rule.action} does not
exist`);
    throw new NotFoundException(`Permission action ${rule.action} does
not exist`);
  }
}

const user = request.user;

if(requireSuperuser || user?.is_admin){
  return user?.is_admin;
}

if(user?.is_authorized){
  return true;
}

const operator = await
this.operatorRepository.createQueryBuilder('operator')
  .leftJoinAndSelect('operator.roles', 'roles')
  .leftJoinAndSelect('roles.permissions', 'permissions')
  .leftJoinAndSelect('operator.client', 'client')
  .where('operator.user.id = :id', { id: user.sub })
  .getOneOrFail();

if(!operator?.roles || !operator?.roles?.length){
  return false;
}

```

```

    if(operator?.client && operator?.client?.id !== user?.client_id){
        return false;
    }

    let seenIds = new Set<number>();

    const ability = defineAbility((can, cannot) => {
        if(operator.roles){
            for (const role of operator.roles) {
                for (const permission of role.permissions as Permission[]) {
                    if (!seenIds.has(permission.id)) {
                        if(permission.conditions){
                            can(permission.action, permission.subject,
permission.conditions);
                        }else{
                            can(permission.action, permission.subject);
                        }
                        seenIds.add(permission.id);
                    }
                }
            }
        }
    });

    for(const rule of rules){
        ForbiddenError.from(ability)
            .setMessage('You are not allowed to perform this action')
            .throwUnlessCan(rule.action, rule.subject);
    }

    return true;
}

async findSubject(subject: string){
    if(!subject){
        return true;
    }
    const table = await this.operatorRepository.manager.query(
        `SELECT EXISTS (
            SELECT 1
            FROM information_schema.tables
            WHERE table_name = $1
        )`,
        [subject.toLowerCase()]
    );
    return table[0].exists;
}

```

```
}
```

фиг. 3.91 guard за проверка на роли

Фигура 3.91 показва имплементацията на guard-a за проверка на роли. Вземат се подадените нужни права и се записват в масива `roles`. Подадените метаданни са предадени от декоратора `checkAbilities` (фиг. 3.92)

```
//needed interface for permission rules
export interface RequiredRule {
  action: SubjectActions;
  subject: string;
  conditions?: any;
}

export const checkAbilities = (...requirements: RequiredRule[]) =>
  SetMetadata(DecoratorMetadata.CHECK_ABILITY, requirements);
```

фиг. 3.92 декоратор за проверка на роли

При преминаването през `rules` се проверява дали обекта, за който се отнася правилото е валиден обект от базата данни. След всички проверки за потребителя, включително дали идентификационния номер на неговата клиентска инстанция съвпада с този от `x-client` header-a, се минава през ролите на оператора, асоцииран с него, и се вземат всички права на тези роли. Използва се `Set` от идентификационните номера на правата, тъй като потребителят може да има няколко роли с еднакви права и няма смисъл да има повече от една инстанция на правото. По тези права се изгражда `CASL ability`, за което след това бива проверено дали има атрибутите, нужни за изпълнение на заявката.

3.6 Скалируемост и висока отказоустойчивост

3.6.1 Какво е висока отказоустойчивост?

Високата отказоустойчивост се представлява способността на дадена система или компонент да остане оперативна и достъпна през възможно най-голям процент от времето. В една високо отказоустойчива система отказите и сривовете са сведени до минимум, като се гарантира, че услугите остават достъпни за потребителите дори в случай на хардуерни повреди, софтуерни повреди или други смущения. Високата отказоустойчивост обикновено се постига чрез толерантност към грешки и проактивни мерки, като например балансиране на натоварването и автоматизирани процеси за възстановяване. Едни от най-често прилаганите методи за реализация на висока отказоустойчивост са клъстериране и репликация.

3.6.2 Какво е хоризонтално скалиране?

Хоризонталното скалиране представлява увеличаване на капацитета и производителността на дадена система чрез добавяне на повече ресурси, като например сървъри, възли или инстанции, за да се разпредели работното натоварване между няколко машини. Добавя повече сървъри към системата, което ѝ позволява да се справя с нарастващия трафик и изискванията за натоварване. Така подобрява мащабируемостта и производителността, като позволява на системата да обработва повече едновременни потребители, заявки и задачи за обработка на данни, без да претоварва отделните компоненти. Контейнерните среди често използват хоризонтално мащабиране, за да се адаптират към динамичните работни натоварвания.

3.6.3 Реализация

За реализация на висока отказоустойчивост и скалируемост на приложението са използвани следните конфигурационни файлове :

```
FROM node:16

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

COPY typeOrm.config.mjs ./

RUN npm run build

EXPOSE 5000

CMD ["npm", "run", "start"]
```

фиг. 3.93 dockerfile за построяване на изображение на сървър

- Dockerfile за построяване на изображение на сървър (фиг. 3.93). Използва се `node:16` за nodejs среда. Задава се работна директория `/app`, копира се `package.json` файла, след което в контейнера се инсталират пакетите. Копира се всичко останало, построява се апликацията, излага се порт и се стартира сървър. Това изображение ще бъде използвано в конфигурацията на `kubernetes service-a`, който ще скалира сървърното приложение.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: server-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: server
  strategy:
```

```

    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  template:
    metadata:
      labels:
        app: server
    spec:
      containers:
        - name: server
          image: crm-system
          imagePullPolicy: Never
          ports:
            - containerPort: 5000
---
apiVersion: v1
kind: Service
metadata:
  name: server-service
spec:
  type: LoadBalancer
  selector:
    app: server
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000

```

фиг. 3.94 конфигурация за service на сървърната апликация

- Конфигурация за service на сървърната апликация - създава се service от тип LoadBalancer, за да може да се балансира товара между двете инстанции на сървърното приложение (индикирани чрез броя реплики в полето replicas) и да бъде изложен на порт 80. Стратегията за опресняване е rollingUpdate, което ще рече, че pod-овете са постепенно опресняващи се, за да се избегне downtime. Чрез задаване на targetPort се постига препращането на заявки от порт 80 на машината към порт 5000 в рамките на deployment-a. MaxSurge: 1 позволява да бъде създаден допълнителен pod по време на опресняването, а

`maxUnavailable: 0` индикира, че по време на него не трябва да има недостъпни pod-ове.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres-statefulset
spec:
  replicas: 0
  serviceName: postgres-service
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15
          env:
            - name: POSTGRES_USER
              value: user
            - name: POSTGRES_PASSWORD
              value: password
            - name: POSTGRES_DB
              value: database
          volumeMounts:
            - name: shared-storage
              mountPath: /var/lib/postgresql/data
      volumeClaimTemplates:
        - metadata:
            name: shared-storage
          spec:
            accessModes: [ "ReadWriteOnce" ]
            resources:
              requests:
                storage: 1Gi
---
apiVersion: v1
kind: Service
metadata:
  name: postgres-service
spec:
  selector:
    app: postgres
```

```
ports:
  - protocol: TCP
    port: 5432
    targetPort: 5432
```

фиг. 3.95 конфигурация на service за базата данни

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis-statefulset
spec:
  replicas: 0
  serviceName: redis-service
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:latest
          ports:
            - containerPort: 6379
          volumeMounts:
            - name: redis-storage
              mountPath: /data
      volumeClaimTemplates:
        - metadata:
            name: redis-storage
          spec:
            accessModes: ["ReadWriteOnce"]
            resources:
              requests:
                storage: 1Gi
---
apiVersion: v1
kind: Service
metadata:
  name: redis-service
spec:
  selector:
    app: redis
```

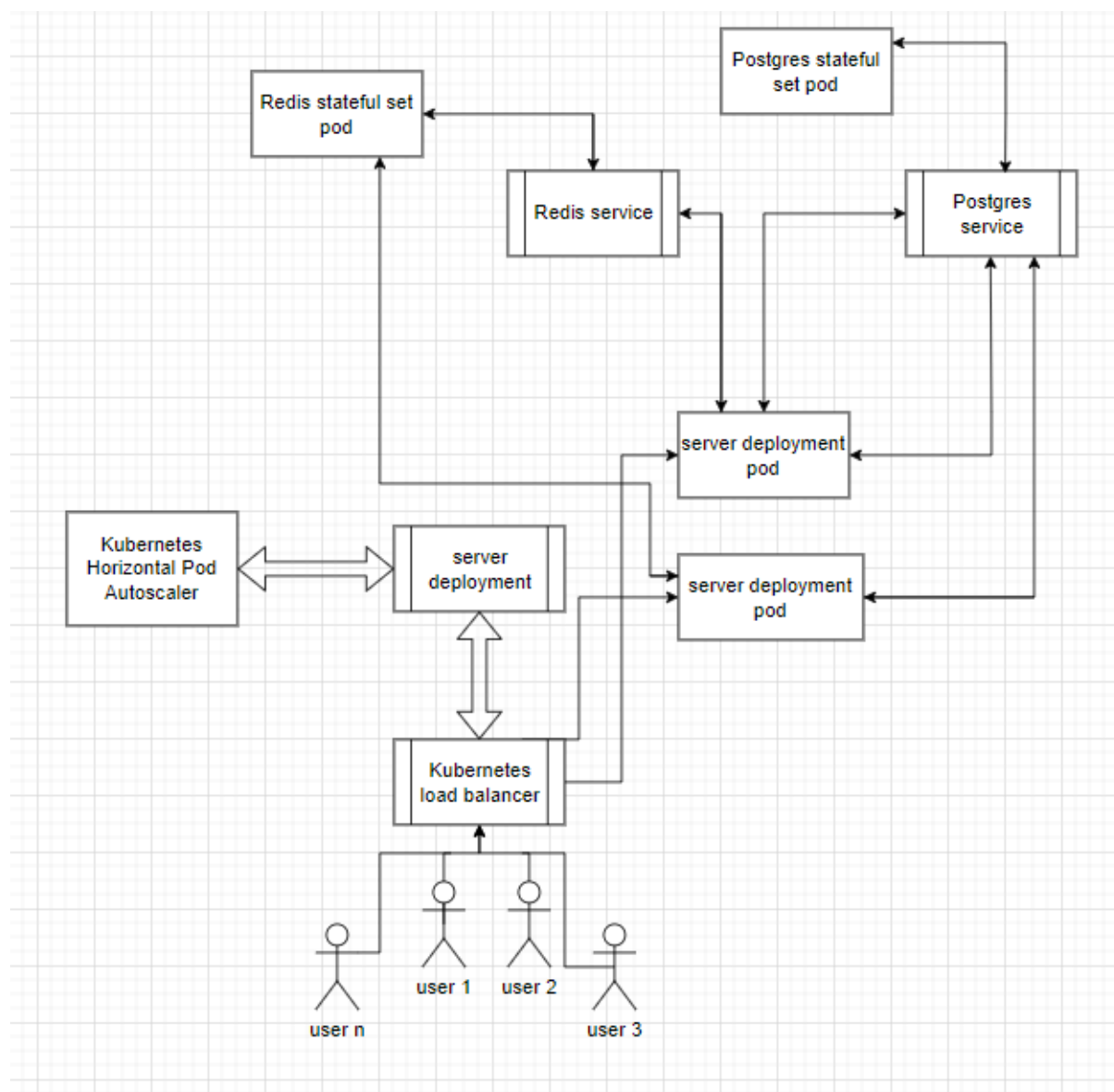


```
ports:
  - protocol: TCP
    port: 6379
    targetPort: 6379
```

фиг. 3.96 конфигурация на service за базата данни

Конфигурациите за базата данни Postgres и хранилището за данни Redis са едни и същи. Те са от тип `StatefulSet` - специализиран ресурс на Kubernetes, пригоден за управление на приложения с променливо състояние. `StatefulSet`-овете предоставят стабилност и уникални мрежови идентичности за отделните pod-ове, като гарантират целостта и наличността на данните. `VolumeClaimTemplate`-овете са включени в конфигурациите, за да се определят шаблони за поддържане на данни. Това гарантира, че данните остават непокътнати във всяка инстанция, дори в случай на рестартиране на pod-овете. Полето `AccessModes` е зададено на `["ReadWriteOnce"]`, което позволява операции за четене и запис само от един node във всеки един момент.

3.6.4 Схема на постигнатата конфигурация



Глава IV.

Ръководство за потребители и разработчици

4.1 Инсталации

За да бъде инсталиран приложно-програмният интерфейс трябва да бъде клонирана github репозиторията му чрез :

```
git clone https://github.com/emildoychinov/CRM-Diploma-BE.git
```

След успешното клониране на репозиторията, трябва да бъде свален Docker на вашата система. На [линкка https://docs.docker.com/engine/install/](https://docs.docker.com/engine/install/) е налична всичката нужна информация за инсталацията. След успешното инсталиране на docker имате две опции :

4.1.1 Пускане на сървъра директно с контейнеризирана база данни

За инсталация на нужните пакети ще се нуждаете от npm и nodejs. Информация за инсталациите им може да бъде намерена на <https://docs.npmjs.com/downloading-and-installing-node-js-and-npm>. След успешната инсталация на npm можете да насочите към директорията, в която сте запазили репозиторията. За най-новата версия на приложно-програмния интерфейс се насочете към branch-a `dev` чрез `git checkout dev`. В директорията създайте файл с име `.env` и сложете данни по следния шаблон във фигура 4.1 :

```
DEV_PORT=<some-type-of-port>
PROD_PORT=<some-type-of-port>

JWT_SECRET="<some-type-of-secret>"
JWT_REFRESH_SECRET="<some-type-of-secret>"
JWT_API_KEY_SECRET="<some-type-of-secret>"
JWT_EXPIRATION="<some-type-of-expiration>"

DB_HOST=<some-type-of-host>
DB_DEPLOYMENT_HOST="<some-type-of-host>"
DB_PORT=<some-type-of-port>
DB_USER="<some-type-of-user>"
DB_PASSWORD="<some-type-of-password>"
DB_NAME="<some-type-of-database>"

REDIS_HOST=<some-type-of-host>
REDIS_DEPLOYMENT_HOST="<some-type-of-host>"
REDIS_PORT=<some-type-of-port>

MAIL_HOST="<some-type-of-host>"
MAIL_USER="<some-type-of-user>"
MAIL_PASSWORD="<some-type-of-password>"
MAIL_FROM="<some-type-of-email>"
```

фиг. 4.1 *примерен .env файл*

Осигурете се, че всяко едно от полетата е попълнено с валидни данни. В противен случай интерфейсът или някои части от него може да не работят. Препоръчително е да се работи с хоста за имейли на gmail smtp.gmail.com. Можете да използвате за потребител свои имейл. За информация по предоставяне на парола към имейла се насочете към <https://support.google.com/mail/answer/185833?hl=en>. Тъй като базата данни и хранилището вървят на вашата машина, можете да зададете хост полетата на localhost. Когато конфигурацията е готова, можете да се насочите към изпълнението на следните команди :

- `docker compose up -d` - за стартиране на Postgres базата данни и Redis хранилището. Осигурете се, че Docker апликацията е пусната на вашата машина предварително.
- `npm install` - за инсталация на всички нужни пакети за пускането на сървъра.
- `npm run build && npm run typeorm:migration:create ./src/migrations/NewMigration` - построяване на миграция към базата данни, за да се направят нужните таблици за функционирането на приложението.
- `npm run build` - построяване на приложението.
- `npm run typeorm:migration:up` - пускане на миграцията
- `npm run start` или `npm run start:debug` (debug или dev - при нужда за промени по кода на сървъра в реално време) - стартиране на приложението.

След изпълняването на тези команди можете да се насочите към сървъра на localhost:<порта, зададен в .env файла>

4.1.2 Пускане на Kubernetes deployment

Пускането на K8s deployment лишава нуждата от инсталацията на node. За да може апликацията да бъде пусната по този начин, трябва предварително да конфигурирате Kubernetes за Docker. Повече информация за това можете да откриете на <https://docs.docker.com/get-started/orchestration/#turn-on-kubernetes>. Можете да пристъпите към командата `./setup.sh`, която изпълнява shell скрипта `setup.sh` (фиг. 4.2) за пускане на deployment-a.

```
#!/bin/bash
kubectl apply -f postgres.yaml
```

```

kubect1 wait --for=condition=Ready pod -l app=postgres
kubect1 apply -f redis.yaml
kubect1 wait --for=condition=Ready pod -l app=redis
docker build -t crm-system .
kubect1 apply -f server.yaml
kubect1 wait --for=condition=Ready pod -l app=server
SERVER=$(kubect1 get pods -l app=server -o
jsonpath='{.items[0].metadata.name}')
kubect1 exec -it $SERVER -- bash -c "npm run build && npm run
typeorm:migration:create ./src/migrations/NewMigration"
kubect1 exec -it $SERVER -- bash -c "npm run build"
kubect1 exec -it $SERVER -- bash -c "npm run typeorm:migration:up"
kubect1 autoscale deployment server-deployment --cpu-percent=50 --min=1
--max=4

```

фиг. 4.2 скрипт за пускане на deployment-a.

Когато искате да го спрете можете да изпълните скрипта за “сваляне” на deployment-a `drop_routine.sh` (фиг. 4.3) чрез `./drop_routine.sh`.

```

#!/bin/bash
kubect1 scale deployment server-deployment --replicas=0
kubect1 scale statefulset redis-statefulset --replicas=0
kubect1 scale statefulset postgres-statefulset --replicas=0
sleep 20s
kubect1 delete hpa server-deployment

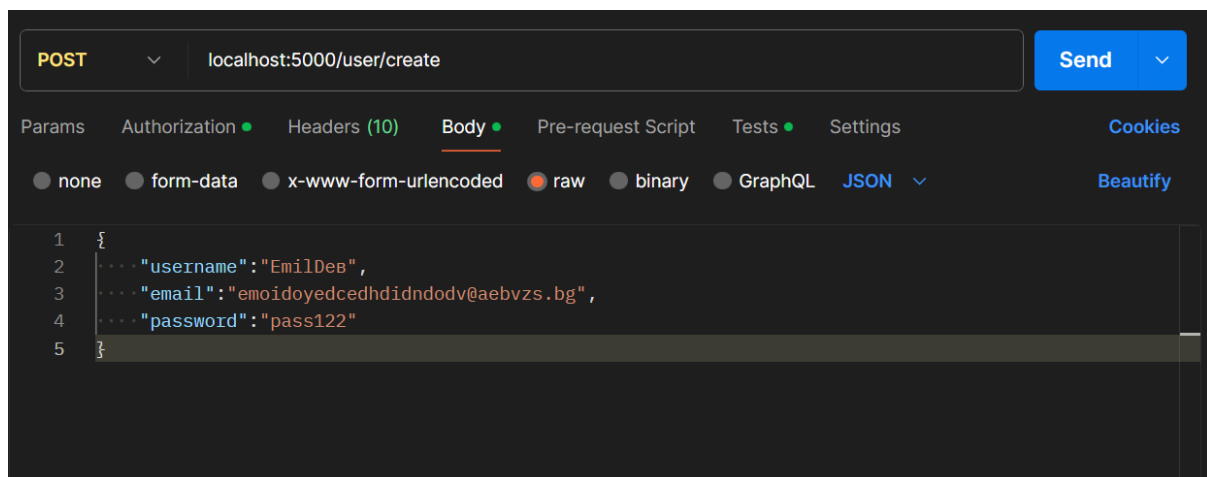
```

фиг. 4.3 скрипт за сваляне на deployment-a.

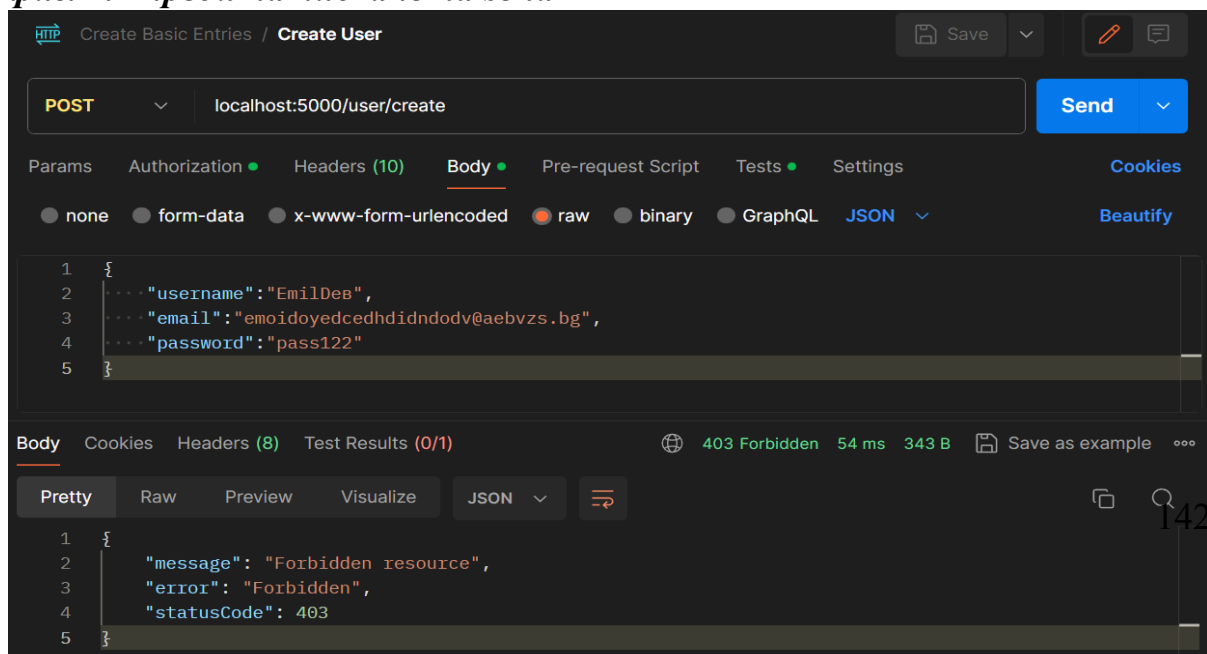
За конфигурация на данните за достъп отново се използва `.env` файл, като този в част 4.1.1. По подразбиране полетата за потребител, парола и база данни в Kubernetes конфигурацията за базата данни са зададени като `user`, `password` и `database`. Ако използвате тях, се осигурете, че са зададени така и в `.env` файла. Осигурете се, че портовете в `.env` файла са същите като тези, използвани в конфигурационните файлове.

4.2 Инструкции за потребители (разработчици) при използване (тестване)

На https://app.getpostman.com/join-team?invite_code=e8776d09236d86b69462ea36d111e2c5&target_code=674878be4591f2ef0c5cbf10d760e819 можете да достъпите Postman работното пространство в което може да бъдат тествани различните заявки. При насочване към заявка можете да натиснете бутона send за получаване на резултат от сървъра. Фигура 4.4 и 4.5 показват процеса



фиг. 4.4 преди натискане на send



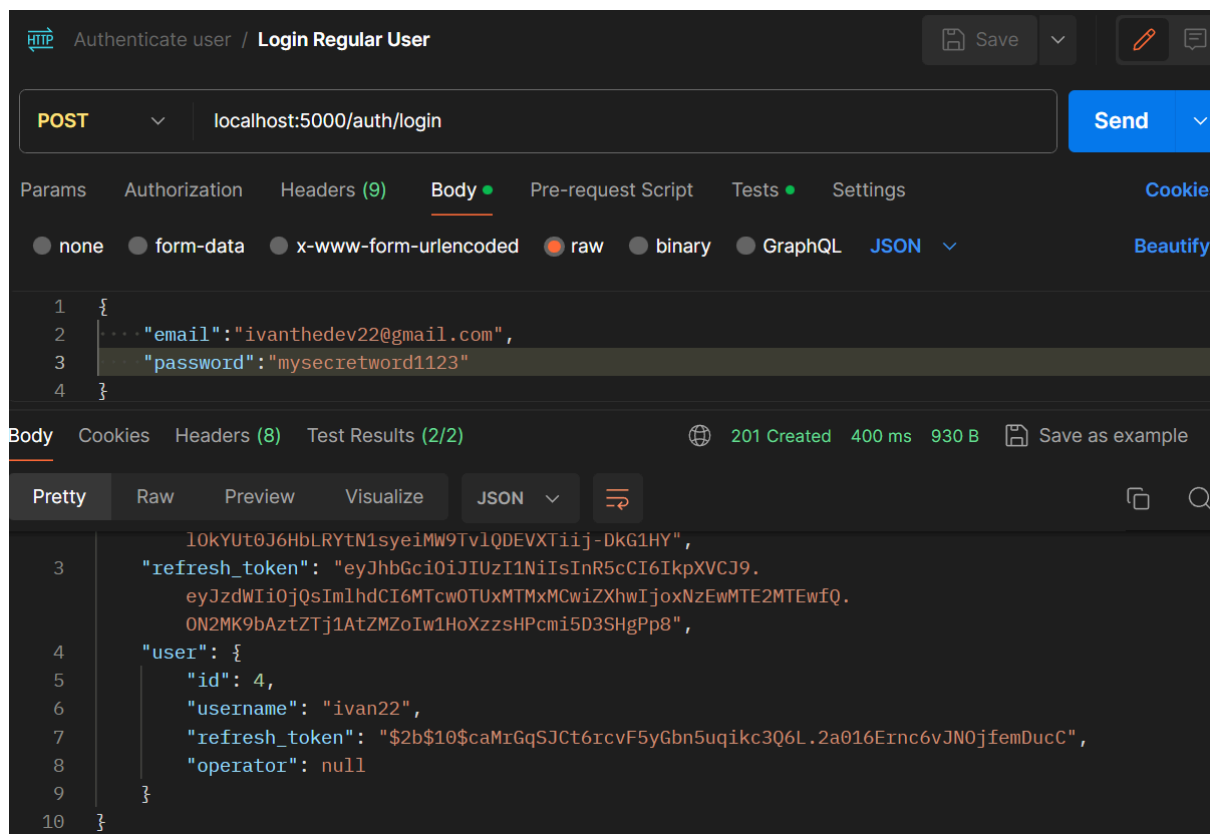
фиг. 4.5 след натискане на send

Тук може да бъде такъв отговорът на заявката, защото е подадена без нужните права или без потребителят да е вписан. При началното пускане на апликацията няма никакви записи в базата данни. За да не се нарушава цялостната структура на приложението, в рамките на конструкторите на service-ите можете да пускате заявки към базата данни, за да бъде напълнена с обекти, с които да се тества. Такъв пример е изложен на фигура 4.6

```
constructor(  
    @InjectRepository(User)  
    private userRepository: Repository<User>,  
    @Inject(forwardRef(() => OperatorService))  
    private operatorService: OperatorService,  
    ) {  
    this.create({  
        username: 'ivan22',  
        email: 'ivanthede22@gmail.com',  
        password: 'mysecretword1123'  
    } as CreateUserDto, {is_admin:true})  
    }
```

фиг. 4.6 създаване на запис в базата данни

След успешният запис на потребител в базата данни, както може да бъде разгледано във фигура 4.7, при вписване вече получаваме следният отговор:



фиг. 4.7 *успешен отговор от заявка*

За улеснение, можете да направите една инстанция на системен клиент (фиг. 4.8), потребителски акаунт (фиг. 4.6), оператор, обвързан с клиента (фиг. 4.9) и потребителския акаунт и роля за админ, която да бъде зададена към този оператор (фиг 4.10).

```

constructor(
    @InjectRepository(Client)
    private readonly clientRepository: Repository<Client>,
    @Inject(forwardRef(() => OperatorService))
    private readonly operatorService: OperatorService,
    @Inject(forwardRef(() => ClientApiKeyService))
    private readonly apiKeyService: ClientApiKeyService
) {
    this.create({
        name: 'testTenant'
    }) as CreateClientDto
}

```

фиг. 4.8 *създаване на системен клиент*

```

export class OperatorService {
  private logger = new Logger(OperatorService.name);
  constructor(
    @InjectRepository(Operator)
    private operatorRepository: Repository<Operator>,
    @Inject(forwardRef(() => UserService))
    private userService: UserService
  ) {
    this.createOperatorGlobally({
      user : {
        id: 4
      } as User,
      client :{
        id: 1
      } as Client,
    } as CreateOperatorDto)
  }
}

```

фиг. 4.9 създаване на оператор

```

export class RolesService {
  private logger = new Logger(RolesService.name);
  constructor(
    @InjectRepository(Role)
    private roleRepository: Repository<Role>,
    @Inject(forwardRef(() => ClientService))
    private clientService: ClientService,
    private readonly queueService: QueueService,
  ) {
    this.createRoleInInstance({
      name: SUPERUSER,
      operators:[
        {id: 3}
      ]
    }, 1)
  }
}

```

фиг. 4.10 създаване на роля

Чрез тези инструкции повече няма да се налага да се пускат заявки към базата данни през кода. За да можете да изпращате заявки, се осигурете, че първо сте преминали през login заявката - така ще се добави токенът за достъп. В полето Headers (фиг. 4.11) добавете header-a x-client с името на създадената инстанция (или инстанцията, за която искате да тествате).

Headers 8 hidden			
	Key	Value	Descript...
☒	x-client	testTenant	
	Key	Value	Description

фиг. 4.11 Header за системен клиент

Чрез тези конфигурации вече всички заявки могат да бъдат тествани, като например

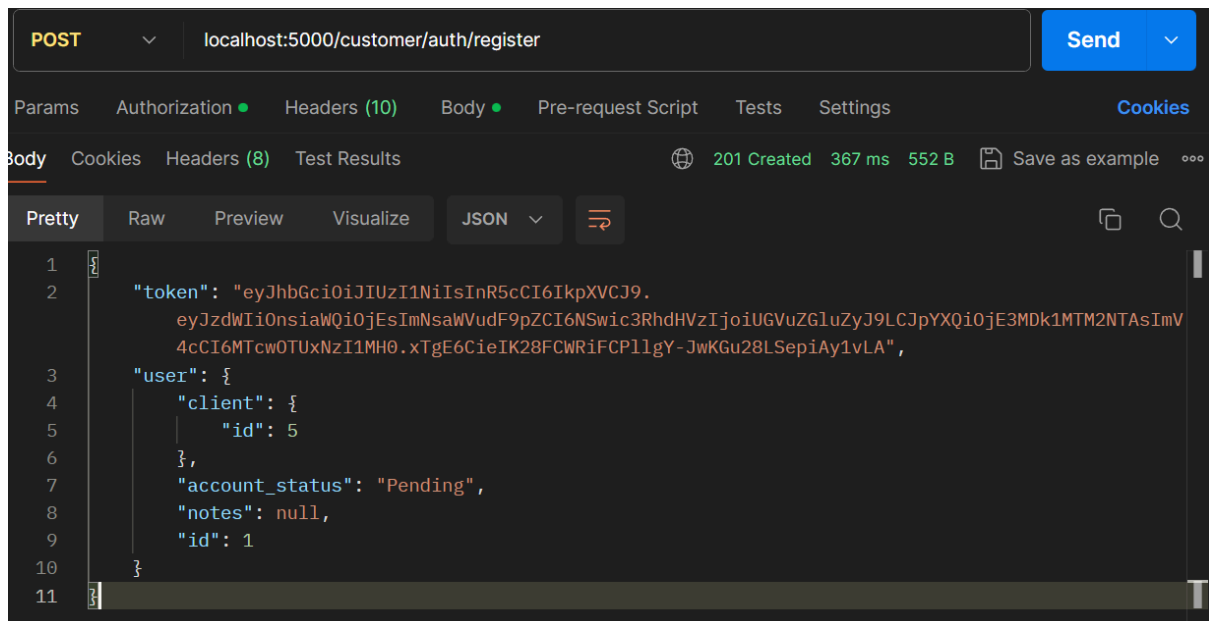
The screenshot shows the Postman interface for a POST request to `localhost:5000/client/create`. The request body is a JSON object: `{ "name": "testTenant123" }`. The response is a 201 status code, indicating successful creation. The response body is a JSON object containing an `api_key` and a `client` object with `name` and `id` fields.

```

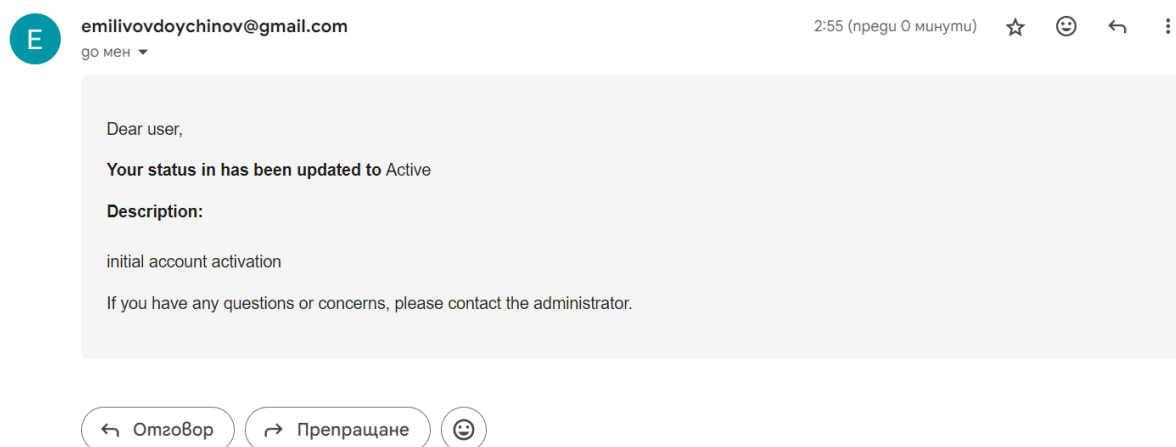
{
  "api_key": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJsIm1hdCI6MTcwOTUxMzMxMywiZGlhZjoxNzEyMTA1MzEzZfQ.MV2F-vWUMDwATX1wDv7H4fMtUoYJWga5ACMh-2iCJFc",
  "client": {
    "name": "testTenant123",
    "id": 5
  }
}

```

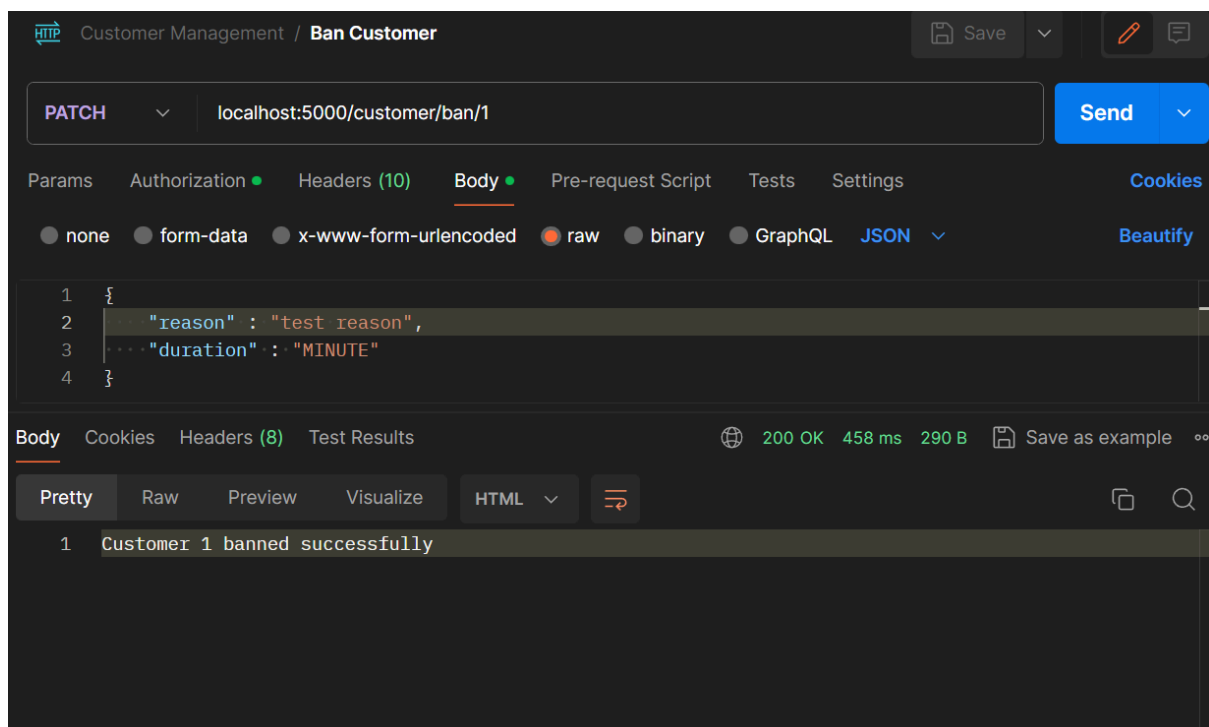
фиг. 4.12 създаване на системен клиент



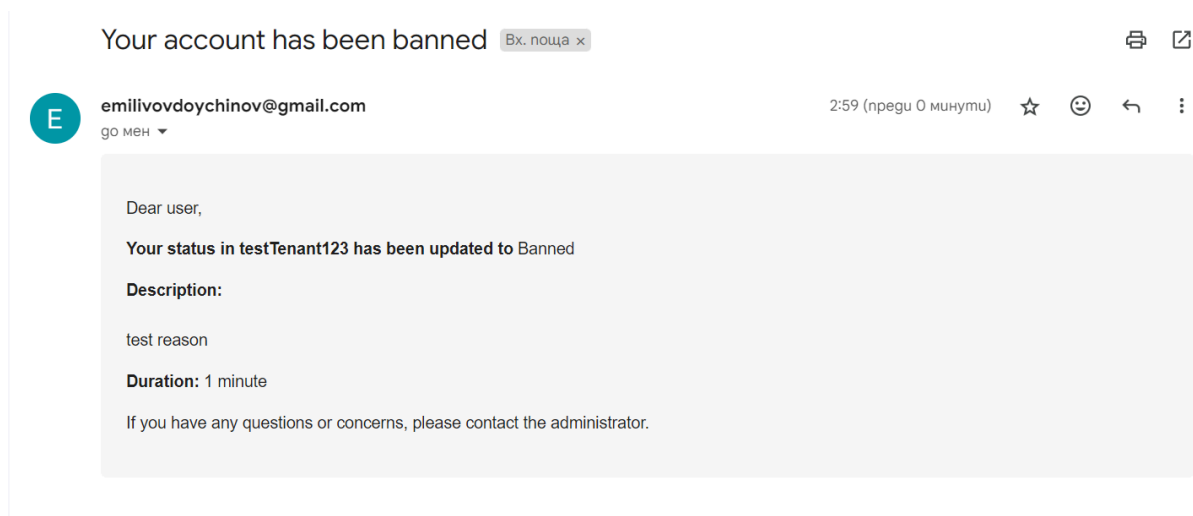
фиг. 4.13 регистриране на потребител на приложението на системен клиент



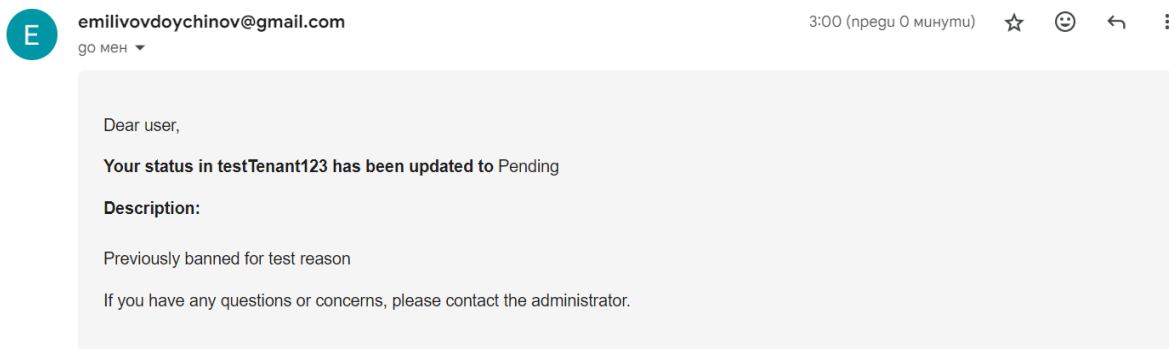
фиг. 4.14 - имейл след регистрация и активация на акаунта



фиг. 4.15 блокиране на потребител



фиг. 4.16 имейл след блокиране на потребител



фиг. 4.17 имейл след изтичане на блокирането

По този начин следва да бъдат тествани и всички други заявки.

Заклучение

В изготвената дипломна работа всички заложиени изисквания за функционалности са покрити. Системата бе наградена и с други изисквания. Тяхната цел бе да я направи възможно решение за корпоративни среди и да постави основата за работещ проект в бизнес бранша. Създаването на система, която е разделена на инстанции не е лека и бърза работа - проверките и валидациите са много, а възможните проблеми, изскачащи покрай тях повече. Въпреки срещаните трудности, особено при изграждането на структура за deployment, е създаден работещ прототип за приложно-програмен интерфейс за система за управление на клиенти. Основен успех на дипломната работа е способността системата да бъде интегрирана с външни клиенти и приложно-програмни интерфейси. Разработката ѝ не спира до тук. Нужни са допълнителни функционалности и промени като :

1. Подобряване на логиката зад някои функционалности с цел подобряване на производителността на приложението, което пряко корелира със скалируемостта
2. Потребителски front-end интерфейс със всички нужни страници за тестване и използване на системата
3. Статистики за потребители на клиентското приложение: за активност, за транзакции, за често посещавани страници ; проследяване на потребителската сесия
4. Известия за срещи с нови клиенти, за които да бъде изготвена инстанция
5. Известия за срещи с потребителите при някои системни клиенти
6. Прикачване на файлове към информацията за потребителите на клиентските приложения

7. Проследяване на дейностите на операторите в рамките на конкретна инстанция
8. Система за статус на акаунти на оператори

Исползвана литература

- [1] - <https://www.cloudflare.com/learning/serverless/glossary/client-side-vs-server-side/>
- [2] - <https://www.rust-lang.org/>
- [3] - <https://www.python.org/>
- [4] - <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [5] - <https://www.typescriptlang.org/>
- [6] - <https://www.oracle.com/java/>
- [7] - <https://docs.microsoft.com/en-us/dotnet/csharp/>
- [8] - <https://golang.org/>
- [9] - <https://www.ruby-lang.org/en/>
- [10] - <https://www.php.net/>
- [11] - <https://www.postgresql.org/>
- [12] - <https://docs.microsoft.com/en-us/sql/?view=sql-server-ver15>
- [13] - <https://redis.io/>
- [14] - <https://www.docker.com/>
- [15] - <https://kubernetes.io/>
- [16] - <https://www.postman.com/>

Съдържание

Увод	1
Глава I.	3
1.1 Методи за разработка на уеб приложения	3
1.2 Технологии и развойни средства за разработка на уеб приложения	4
1.2.1 Rust	4
1.2.2 Python	5
1.2.3 JavaScript	5
1.2.4 TypeScript	6
1.2.5 Java	6
1.2.6 C#	7
1.2.7 Go	7
1.2.8 Ruby	8
1.2.9 PHP	8
1.2.10 PostgreSQL	9

1.2.11 MSSQL	9
1.2.12 Redis	10
1.2.13 Docker	11
1.2.14 Kubernetes	12
1.2.15 Postman	13
1.3 Оглед на съществуващи решения	13
1.3.1 Salesforce	13
1.3.2 Zoho CRM	15
1.3.3 HubSpot CRM	15
Глава II.	17
2.1 Функционални изисквания	17
2.2 Избор на развойни средства	20
2.2.1 Visual Studio Code	20
2.2.2 NestJS	21
2.2.3 JWT	22
2.2.4 CASL	23
2.2.5 Bull	25
2.2.6 Redis	26
2.2.7 Handlebars	26
2.2.8 JSON и DTO	27
2.2.9 Docker	28
2.2.10 Kubernetes	28
2.2.11 Postman	28
2.3 База данни	28
2.3.1 Релационни бази данни	28
2.3.1 Нерелационни бази данни	30
2.3.2 Избор на база данни	30
2.3.3 Осъществяване на връзка с база данни	30
2.3.4 Структура на базата данни	32
2.4 Структура на приложението	40
Глава III.	42
3.1 Файлова структура	42
3.2 Помощни константи, енумерации, интерфейси	44
3.2.1 Константи	44
3.2.2 Енумерации	46

3.2.3 Интерфейси	49
3.3 Помощни функционалности	50
3.3.1 Връзка с базата данни	50
3.3.2 Връзка със склада данни	51
3.3.3 Опашка за процеси	53
3.3.4 Изпращане на имейли	58
3.4 Основни модули в приложението	64
3.4.1 Системен клиент и ключ за достъп към приложно-програмния интерфейс	64
3.4.2 Потребител и оператор	76
3.4.3 Права	90
3.4.4 Роли	94
3.4.5 Клиенти на системните клиенти	101
3.5 Автентикация и оторизация	115
3.5.1 Автентикация	116
3.5.2 Оторизация	125
3.6 Скалируемост и висока отказоустойчивост	131
3.6.1 Какво е висока отказоустойчивост?	131
3.6.2 Какво е хоризонтално скалиране?	131
3.6.3 Реализация	132
3.6.4 Схема на постигнатата конфигурация	137
Глава IV.	138
4.1 Инсталации	138
4.1.1 Пускане на сървър директно с контейнеризирана база данни	138
4.1.2 Пускане на Kubernetes deployment	140
4.2 Инструкции за потребители (разработчици) при използване (тестване)	142
Заклучение	150
Използвана литература	152
Съдържание	153